
Table of Contents

Section	Content
00	Introduction & How to Use
01	C++ STL
02	Time & Space Complexity
03	Arrays
04	Strings
05	Searching
06	Sorting
07	Hashing
08	Two Pointers
09	Sliding Window
10	Linked Lists
11	Stacks & Queues / Deques
12	Recursion
13	Backtracking
14	Trees
15	Binary Search Trees
16	Heaps / Priority Queues
17	Greedy Algorithms
18	Graphs
19	Dynamic Programming
20	Tries
21	Bit Manipulation
22	Disjoint Set Union
23	Fenwick Trees
24	Segment Trees
25	Advanced Graph Algorithms
26	Problem-Solving Patterns
27	Practice Solutions & Hints
28	DSA Checklist & Complexity Cheat Sheet

DSA Notes in C++ — From Beginner to Advanced

A complete, self-contained study guide for Data Structures & Algorithms in C++. Written for someone who knows **basic C++** (variables, loops, functions, classes, pointers, references) but is **new to DSA**.

How to use these notes 1. Work through the chapters **in order**. Each builds on the previous one. 2. **Do not just read the code — type it, run it, modify it**. Understanding comes from doing. 3. Each chapter ends with practice problems. Try them yourself **before** looking at [27-Practice-Solutions.md](#). Solutions exist so you can verify, not to be read first. 4. Keep the cheatsheet ([28-DSA-Checklist-and-Cheatsheet.md](#))

open while practicing. 5. Compile examples with any C++17 compiler, e.g. `g++ -std=c++17 file.cpp -o prog`.

Chapter Index

#	Chapter	Core idea
01	C++ STL	The tools you will use daily: vectors, maps, sets, priority queues
02	Time & Space Complexity	Big-O, analyzing loops, recursion, space analysis
03	Arrays	The most basic data structure; subarray patterns
04	Strings	Character arrays + built-in operations
05	Searching	Linear search, binary search, search on answer
06	Sorting	$O(n^2)$, $O(n \log n)$, and linear-time sorts; stability
07	Hashing	$O(1)$ lookups via hash tables
08	Two Pointers	Squeezing $O(n^2)$ down to $O(n)$
09	Sliding Window	Fixed & variable size windows over arrays/strings
10	Linked Lists	Dynamic memory, node-based chains
11	Stacks & Queues / Deques	LIFO and FIFO; monotonic stacks
12	Recursion	Functions calling themselves; base + recursive case
13	Backtracking	Systematic search with undo
14	Trees	Hierarchical data; traversals; BFS/DFS
15	Binary Search Trees	Ordered trees with $O(\log n)$ ops
16	Heaps / Priority Queues	Fast min/max extraction
17	Greedy Algorithms	Take the best local choice, prove it global
18	Graphs	BFS, DFS, shortest paths, MST, topological sort
19	Dynamic Programming	Overlapping subproblems; memoization & tabulation
20	Tries	Fast string prefix lookups
21	Bit Manipulation	Bitwise tricks for speed and compact state
22	Disjoint Set Union	Merge/set membership in near $O(1)$
23	Fenwick Trees	Prefix sums with point updates
24	Segment Trees	Range queries with updates (general Fenwick)
25	Advanced Graph Algorithms	Dijkstra, Bellman-Ford, Floyd-Warshall, SCC, bridges, flows
26	Problem-Solving Patterns	The meta-skills: how to attack any problem
27	Practice Solutions & Hints	Hints first, then full solutions
28	DSA Checklist & Complexity Cheat Sheet	Pre-interview refresher

Learning Paths

- **If you have limited time (interview prep):** 01 → 02 → 03 → 05 → 07 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 18 → 19 → 26 → 28.
- **If you are a student learning everything:** go in order, 01 → 28.
- **If you already know the basics:** skim 01–07, then start from 08.

Conventions Used in These Notes

- `//` comments explain *why*, the code shows *what*.

- Time/space complexity is shown in a box under every implementation, e.g.

Time: $O(n \log n)$ | **Space:** $O(1)$

- Practice problems are tagged **[Easy]**, **[Medium]**, **[Challenging]**.
- Diagrams use ASCII art so they render everywhere.

How Problems Are Chosen

Easy problems teach the *one core idea* of the chapter. Medium problems combine two ideas. Challenging problems require creativity and multi-step reasoning — they may use techniques from later chapters. If a challenging problem feels too hard, come back after finishing the whole book.

Chapter 01 — C++ STL

1. What is the STL and why learn it?

The **Standard Template Library (STL)** is the standard toolbox that ships with C++. It contains ready-made, battle-tested data structures (containers), algorithms, and iterators.

Why it matters for DSA: In competitive programming and interviews you almost never hand-roll a hash map or a priority queue. You use `std::unordered_map` and `std::priority_queue`. Knowing the STL is a prerequisite for everything that follows.

Golden rule: Use the STL. Only implement your own data structure when the STL either (a) does not provide what you need, or (b) you are being asked to implement it.

2. The containers you'll use 95% of the time

2.1 `vector` — a dynamic array

A resizable array. Data sits contiguously in memory, so random access is $O(1)$.

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v;           // empty
    vector<int> v2(5, 3);    // five 3s: 3 3 3 3 3
    vector<int> v3 = {1, 2, 3}; // initializer list

    v.push_back(10);         // append to end (amortized O(1))
    v.push_back(20);
    v.pop_back();            // remove last element
    v[0] = 5;               // random access O(1)

    cout << v.size() << "\n"; // number of elements
    cout << v.empty() << "\n"; // 1 if empty

    // iteration
    for (int x : v) cout << x << " ";
    for (size_t i = 0; i < v.size(); i++) cout << v[i] << " ";

    // useful member functions
    v.clear();               // remove all elements
    v.insert(v.begin(), 99); // insert at position (O(n))
    v.erase(v.begin());      // erase at position (O(n))
    sort(v.begin(), v.end()); // O(n log n), needs <algorithm>
    reverse(v.begin(), v.end()); // reverse in place

    return 0;
}
```

Time: push_back amortized $O(1)$, random access $O(1)$, insert/erase middle $O(n)$.

Why it grows for "free": when `v` is full, the STL allocates a bigger block (usually 2x) and copies elements over. That copy happens rarely, so the average cost of `push_back` is $O(1)$. This is called **amortized** analysis.

2.2 string

A `std::string` is basically a `vector<char>` with extra string-specific helpers.

```
string s = "hello";
s += " world";           // concatenation
s.size(); s.length();    // both give length
s[0];                    // 'h'
s.substr(1, 3);           // "ell" - from index 1, length 3
s.find("ell");             // index of first match, or string::npos if absent
s.compare("hello");        // 0 if equal
reverse(s.begin(), s.end());

string a = "10", b = "20";
int x = stoi(a);           // string -> int
string t = to_string(x + 5); // int -> string

// character helpers
isalpha('a'); isdigit('7'); islower('b'); toupper('c');
```

Convert char to string: `string(1, c)` or use `s.push_back(c)`.

2.3 stack, queue, deque

```
stack<int> st;           // LIFO
st.push(1); st.push(2);
int t = st.top();        // 2
st.pop();                 // removes 2 (returns void!)

queue<int> q;             // FIFO
q.push(1); q.push(2);
int f = q.front();        // 1
q.pop();                  // removes 1

deque<int> dq;            // double-ended
dq.push_back(1); dq.push_front(0);
dq.pop_front(); dq.pop_back();
```

Common mistake: `stack::pop` and `queue::pop` return `void`. Read `top()` / `front()` before popping.

2.4 pair and tuple

```
pair<int, string> p = {3, "abc"};
p.first; p.second;

// comparison works lexicographically (first, then second)
pair<int, int> a = {1, 2}, b = {1, 5};
// a < b is true

tuple<int, int, int> t = {1, 2, 3};
auto [x, y, z] = t;           // structured binding (C++17)
```

2.5 map / set — ordered (Red-Black tree based)

`map` = ordered dictionary (key → value). `set` = ordered unique elements. Both maintain **sorted** order; iteration goes from smallest to largest.

```
map<string, int> mp;
mp["alice"] = 5;           // O(log n) insert
mp["bob"] = 3;
mp["alice"] += 1;           // now 6

if (mp.count("bob")) { /* exists */ }
auto it = mp.find("bob");   // iterator or mp.end()
mp.erase("bob");
```

```
for (auto &[key, val] : mp) { // ordered iteration
    cout << key << " " << val << "\n";
}

set<int> s;
s.insert(3); s.insert(1); s.insert(2); // {1,2,3}
s.erase(2);
if (s.count(1)) { /* present */ }
auto lb = s.lower_bound(2); // first element >= 2
auto ub = s.upper_bound(2); // first element > 2
```

Time: all ops $O(\log n)$. Iteration is sorted and $O(n)$.

`multiset` allows duplicates; `multimap` allows duplicate keys. Use sparingly — `multiset` is famous for slow `erase(value)` erasing *all* copies.

2.6 unordered_map / unordered_set — hash based

Same interface, but **no ordering** and **average $O(1)$** per operation. Use when you only need fast lookup, not sorted iteration.

```
unordered_map<string, int> cnt;
for (string w : {"a", "b", "a", "c"}) cnt[w]++;
cout << cnt["a"]; // 2

unordered_set<int> seen;
seen.insert(7);
if (seen.count(7)) { /* present */ }
```

Time: average $O(1)$, worst case $O(n)$ if many hash collisions.

When to pick which:

Need	Use
Fast lookups, no ordering needed	<code>unordered_map</code> / <code>unordered_set</code>
Iterate in sorted order, <code>lower_bound</code> / <code>upper_bound</code>	<code>map</code> / <code>set</code>
Count frequencies of keys	<code>unordered_map</code>
Duplicates allowed + order	<code>multiset</code> / <code>multimap</code>

2.7 priority_queue — a max-heap

By default the **largest** element is at the top. See Chapter 16 for the theory.

```
priority_queue<int> pq; // max-heap
pq.push(3); pq.push(10); pq.push(1);
cout << pq.top(); // 10
pq.pop(); // removes 10

// min-heap
priority_queue<int, vector<int>, greater<int>> minpq;
minpq.push(3); minpq.push(10); minpq.push(1);
cout << minpq.top(); // 1

// custom comparator (max-heap by second of pair)
priority_queue<pair<int, int>> pq2; // already compares lexicographically
```

Time: push/pop $O(\log n)$, top $O(1)$.

2.8 list and forward_list

Doubly- and singly-linked lists. In interviews you usually build linked lists by hand (Chapter 10) because questions test your understanding of pointers.

3. Iterators

An iterator is a "pointer into a container". Ranges are `[begin, end)` — `end` is one **past** the last element, so an empty container has `begin() == end()`.

```
vector<int> v = {1, 2, 3, 4};
auto it = v.begin() + 1;      // points at 2
cout << *it;                 // 2
auto last = v.end();          // one past 4
```

The `[begin, end)` **half-open convention is the most important iterator rule**. It's why `v.end() - v.begin() == v.size()` and why loops usually look like `for (i = 0; i < n; ++i)`.

4. Key algorithms from `<algorithm>`

```
sort(begin, end);                // O(n log n), introsort
stable_sort(begin, end);          // keeps equal elements in original order
sort(begin, end, greater<int>()); // descending

binary_search(begin, end, val);    // O(log n), requires sorted
lower_bound(begin, end, val);      // first position >= val
upper_bound(begin, end, val);      // first position > val

min_element / max_element(begin, end); // iterator to min/max
accumulate(begin, end, 0);         // sum (adds to init)
count(begin, end, val);             // how many times val appears
find(begin, end, val);              // iterator to first val, else end()
reverse(begin, end);               // reverse in place
unique(begin, end);                // removes adjacent duplicates (usually sort first!)
next_permutation(begin, end);      // next lexicographic permutation, returns bool
```

The **unique trap**: `unique` only collapses *adjacent* duplicates, and it does not resize the vector. Standard idiom:

```
v.erase(unique(v.begin(), v.end()), v.end());
```

Count frequency quickly:

```
sort(v.begin(), v.end());
int cnt = upper_bound(v.begin(), v.end(), x) - lower_bound(v.begin(), v.end(), x);
```

5. Two-step worked examples

Example A — Count each word, then print most frequent

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<string> words = {"cat", "dog", "cat", "bird", "dog", "cat"};
    unordered_map<string, int> freq;
    for (auto &w : words) freq[w]++;           // hash map counts in O(1) each

    string best; int bestCnt = 0;
    for (auto &[w, c] : freq)                  // scan all keys
        if (c > bestCnt) { bestCnt = c; best = w; }

    cout << best << " " << bestCnt << "\n";   // cat 3
    return 0;
}
```

Example B — Return indices of two numbers summing to target

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> pos;               // value -> index
    for (int i = 0; i < (int)nums.size(); i++) {
        int need = target - nums[i];
        if (pos.count(need))
            return {pos[need], i};
    }
}
```

```

    pos[nums[i]] = i; // store AFTER check to avoid self-match
}
return {};
}

```

Time: $O(n)$ | **Space:** $O(n)$

6. When to use & common mistakes

When to use the STL: always, unless the problem asks you to implement the structure itself. It is correct, fast, and accepted by every judge.

Common mistakes: 1. `vector<int> v(n);` gives `n` zeros, while `vector<int> v;` is empty — don't index into an empty vector. 2. `pop()` on `stack/queue` returns `void`. 3. `map[key]` **inserts** a default value if missing. Use `count/ find` to test presence first. 4. `string::npos` comparison: `if (s.find(c) != string::npos)`. 5. `sort` on iterators, but `.begin()/.end()` — forgetting `.begin()` is a classic. 6. `unordered_map` with pairs/tuples as keys needs a custom hash (use `map` instead). 7. Iterators are invalidated after `push_back` reallocates — re-fetch or use indices.

7. Practice

P1 [Easy] — Given an array, return `true` if any element appears more than once. (*LeetCode 217*)

P2 [Easy] — Merge two sorted arrays into one sorted array. (*LeetCode 88*)

P3 [Medium] — Given a string `s`, find the length of the longest substring without repeating characters. (*LeetCode 3 — hints in the sliding window chapter*)

P4 [Challenging] — Given an array of integers, group numbers by their number of divisors, and return the size of the largest group when counts are equal. (*LRU-style tuning: think about what map structure keeps a running max.*)

Check your solutions in `27-Practice-Solutions.md` only after trying.

Chapter 02 — Time & Space Complexity

1. What is it and why do we care?

Complexity analysis answers one question: *how does the runtime (or memory) of an algorithm grow as the input grows?*

We care because inputs can be huge. An algorithm that is instant on 10 elements might take hours on 10,000,000. Complexity analysis lets us predict this **before** running the code, and lets us compare two algorithms fairly, regardless of machine speed.

2. Big-O notation in plain language

Big-O describes the **upper bound** of growth rate. It ignores constants and smaller terms, focusing only on what happens for **very large inputs**.

- $O(1)$ — constant. Always the same number of steps.
- $O(\log n)$ — logarithmic. Grows very slowly (halving each step).
- $O(n)$ — linear. Steps grow proportionally with input.
- $O(n \log n)$ — linearithmic. Slightly super-linear (typical of good sorts).
- $O(n^2)$ — quadratic. Doubling input quadruples work.
- $O(2^n)$ — exponential. Doubling input squares work — unusable beyond $\sim n=25$.
- $O(n!)$ — factorial. Beyond $\sim n=10$, hopeless.

Size	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	10	33	100	1,024
100	100	664	10,000	$\sim 1e30$
1,000	1,000	9,966	$1e6$	impossible
$1e6$	$1e6$	$\sim 2e7$	$1e12$	impossible

Rules for simplifying: - Drop constants: $3n + 7 \rightarrow O(n)$. - Drop smaller terms: $n^2 + n + 1000 \rightarrow O(n^2)$. - Drop the base of logs: $\log_2 n = \log_{10} n = O(\log n)$ (they only differ by a constant factor).

3. How to analyze loops

Simple loop

```
int sum = 0;
for (int i = 0; i < n; i++) sum += a[i]; // n iterations, each O(1)
// O(n)
```

Nested loop (independent)

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        sum += 1;
// n * n = O(n^2)
```

Nested loop where the inner depends on outer

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        sum += 1;
// 0 + 1 + 2 + ... + (n-1) = n(n-1)/2 = O(n^2)
```

Halving loop (logarithmic)

```
while (n > 1) {
    n /= 2; // n: 64 → 32 → 16 → 8 → 4 → 2 → 1, that's log2(64) = 6 steps
}
// O(log n)
```

Loop whose index jumps in powers of two

```
for (int i = 1; i < n; i *= 2) { } // O(log n)
```

The "two halves + linear scan" pattern (merge sort style)

```
void f(int n) {
    if (n <= 1) return;
    f(n / 2);
    f(n / 2);
    for (int i = 0; i < n; i++) { } // O(n) work
}
// T(n) = 2T(n/2) + O(n) → O(n log n)
```

Master method (quick shortcut) for $T(n) = a \cdot T(n/b) + O(n^d)$: - If $a = b^d \rightarrow O(n^d \log n)$ (merge sort: $a=2, b=2, d=1 \rightarrow n \log n$) - If $a < b^d \rightarrow O(n^d)$ - If $a > b^d \rightarrow O(n^{\log_b a})$

4. Analyzing recursion

Recursive complexity is the sum of work done by all calls. Two useful shapes:

1. **One recursive call that shrinks by a constant** — linear recursion $\rightarrow O(n)$.
2. **One recursive call that halves** — $O(\log n)$ (like binary search).

3. **Two recursive calls that halve** — $O(n)$ if each node does $O(1)$ work (full binary tree has n leaves)... wait, careful: - $T(n) = 2T(n/2) + O(1) \rightarrow O(n)$ (each level totals $O(1) \cdot n/2^n$... actually $O(n)$ total). - $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.
4. **Two recursive calls that shrink by 1** — Fibonacci style $\rightarrow O(2^n)$, explosive.

```
// Fibonacci, naive
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2); // tree of calls, size ~2^n
}
// O(2^n) ← this is why DP (Chapter 19) exists
```

5. Space complexity

Space complexity counts **extra memory** the algorithm needs, besides the input itself.

- An array of size $n \rightarrow O(n)$.
- A 2D array $n \times n \rightarrow O(n^2)$.
- Recursion depth $d \rightarrow O(d)$ for the call stack.

```
int sumArray(vector<int>& a) { // passed by reference: no copy
    int s = 0;
    for (int x : a) s += x;
    return s;
}
// Time: O(n), Space: O(1) — no extra memory that grows with n
```

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
// Time: O(n), Space: O(n) — the recursion stack holds n frames
```

6. Estimating input size limits (rule of thumb)

A rough guide for what's solvable in ~1 second:

Complexity	Max input size
$O(n!)$	$n \leq 10$
$O(2^n)$	$n \leq 20-25$
$O(n^3)$	$n \leq \sim 500$
$O(n^2)$	$n \leq \sim 10^4-10^5$
$O(n \log n)$	$n \leq \sim 10^6-10^7$
$O(n)$	$n \leq \sim 10^7-10^8$
$O(\log n)$	anything

This is the **single most useful trick** in competitive programming: look at n , deduce the required complexity, and that tells you which algorithm to reach for.

7. Worked example — analyzing a full program

```
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i]; // O(n)

    sort(a.begin(), a.end()); // O(n log n)

    unordered_map<int,int> freq;
    for (int x : a) freq[x]++; // O(n) amortized
```

```

int ans = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (freq.count(a[i] + a[j])) ans++;    // O(n^2)

cout << ans << "\n";
return 0;
}

```

Total: $O(n) + O(n \log n) + O(n) + O(n^2) = O(n^2)$ time, $O(n)$ space.

8. Common mistakes

- Confusing best/average/worst case.** Big-O is usually worst-case. E.g., `unordered_map` is $O(1)$ *average* but $O(n)$ *worst*.
- Counting steps, not growth.** "This runs 1,000,000 times" is $O(1)$ if n is fixed.
- Forgetting input-reading/output.** Sometimes I/O dominates; use fast IO: `ios::sync_with_stdio(false); cin.tie(nullptr);`
- Assuming two nested loops are always $O(n^2)$.** If the inner loop is a hash-map lookup over a bounded alphabet, it might be $O(n)$.
- Ignoring the recursion stack in space analysis.**

9. Practice

P1 [Easy] — For each snippet, state time & space complexity and justify:

```

// (a)
for (int i = 1; i < n; i *= 2) for (int j = 0; j < n; j++) x++;
// (b)
for (int i = 0; i < n; i++) for (int j = 0; j < i*i; j++) x++;
// (c)
int f(int n){ if(n==0) return 0; return f(n/2) + 1; }

```

P2 [Medium] — Given $T(n) = 3T(n/3) + O(n)$, find the complexity using the master method. Then write the recursive function it describes.

P3 [Medium] — A function has complexity $O(n^2)$. If it takes 2 seconds for $n = 10,000$, estimate (roughly) how long for $n = 100,000$.

P4 [Challenging] — Find a single loop or nested-loop structure with complexity $O(n \log n)$ that is *not* merge sort, and justify its complexity precisely (hint: think of a loop that skips by powers of two *inside* a linear outer loop, and sum the logarithms).

Solutions in `27-Practice-Solutions.md`.

Chapter 03 — Arrays

1. What is an array?

An array is a **contiguous block of memory** holding elements of one type, addressed by index. `a[i]` means "jump i slots past the start" — that's why access is $O(1)$.

```

Memory:
+-----+-----+-----+-----+-----+
| 5  | 12 | -3  | 8  | 42 |      a[0]=5, a[1]=12, ...
+-----+-----+-----+-----+-----+
index 0      1      2      3      4

```

Why it's useful: it is the foundation of almost every data structure and problem. Random access in $O(1)$, cache friendly (elements sit side by side), and simple.

2. Core operations and complexity

Operation	Time
Access <code>a[i]</code>	$O(1)$
Search (unsorted)	$O(n)$
Search (sorted)	$O(\log n)$ via binary search
Insert at end (vector)	$O(1)$ amortized
Insert in middle	$O(n)$
Delete in middle	$O(n)$

3. The three most important array patterns

Pattern 1: Prefix sums

Precompute sums of prefixes so that **range sums become $O(1)$** .

`pref[i]` = sum of `a[0..i-1]`. Then `sum(l..r) = pref[r+1] - pref[l]`.

```
vector<int> prefixSum(const vector<int>& a) {
    int n = a.size();
    vector<int> pref(n + 1, 0);
    for (int i = 0; i < n; i++)
        pref[i + 1] = pref[i] + a[i];
    return pref;
}

int rangeSum(const vector<int>& pref, int l, int r) {
    return pref[r + 1] - pref[l];
}
```

Time: $O(n)$ to build, $O(1)$ per query | **Space:** $O(n)$

Worked example: `a = [2, 1, -1, 3]` → `pref = [0, 2, 3, 2, 5]`. `sum(1..3) = pref[4] - pref[1] = 5 - 2 = 3` ✓ (`1 + -1 + 3 = 3`).

Pattern 2: Kadane's algorithm (max subarray sum)

The famous $O(n)$ solution to "find the contiguous subarray with the largest sum".

Idea: as you scan left to right, keep the best sum that *ends at position i*. At each step, either extend the previous best (add `a[i]`) or start fresh (just `a[i]`). That's exactly `max(a[i], prevBest + a[i])`.

```
int kadane(const vector<int>& a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < (int)a.size(); i++) {
        cur = max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}
```

Time: $O(n)$ | **Space:** $O(1)$

Worked example: `a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]` - `i=1`: `cur = max(1, -2+1=-1) = 1`, `best = 1` - `i=2`: `cur = max(-3, 1-3=-2) = -2`, `best = 1` - `i=3`: `cur = max(4, 2) = 4`, `best = 4` - `i=4`: `cur = max(-1, 3) = 3`, `best = 4` - `i=5`: `cur = max(2, 5) = 5`, `best = 5` - `i=6`: `cur = max(1, 6) = 6`, `best = 6` - `i=7`: `cur = max(-5, 1) = 1`, `best = 6` - `i=8`: `cur = max(4, 5) = 5`, `best = 6` → **answer 6** ✓

Pattern 3: Two pointers on a sorted array (see also Ch. 08)

4. 2D arrays

```
int n = 3, m = 4;
vector<vector<int>> grid(n, vector<int>(m, 0)); // n x m grid of zeros

// fill
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        grid[i][j] = i * m + j;

// row-major iteration
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        cout << grid[i][j] << " \n"[j == m - 1];
```

Neighbors in a grid (up/down/left/right) — used constantly in graph/DP problems:

```
int di[4] = {-1, 1, 0, 0};
int dj[4] = {0, 0, -1, 1};
for (int k = 0; k < 4; k++) {
    int ni = i + di[k], nj = j + dj[k];
    if (ni < 0 || ni >= n || nj < 0 || nj >= m) continue; // bounds check
    // process (ni, nj)
}
```

5. Rotation / reversal tricks

Reverse a whole subarray — the building block for rotations:

```
reverse(a.begin(), a.end()); // reverse whole array
reverse(a.begin(), a.begin() + k); // reverse first k
```

Rotate right by k (LeetCode 189) in $O(n)$:

```
k %= n;
reverse(a.begin(), a.end());
reverse(a.begin(), a.begin() + k);
reverse(a.begin() + k, a.end());
```

6. When to use & common mistakes

When to use: when you need indexed random access, or when the problem screams "subarray", "prefix", "sliding window". Arrays are the default choice for any sequence.

Common mistakes: 1. Off-by-one on n vs $n-1$. Make the `pref` array size $n+1$ to avoid it. 2. Forgetting to `%` the rotation amount (rotate by $k > n$). 3. Integer overflow in prefix sums — use `long long`. 4. Using $O(n^2)$ when prefix sums make it $O(n)$. 5. Confusing indices when passing 2D arrays; pass by reference. 6. Kadane on an all-negative array: initialize `best` with `a[0]`, not 0.

7. Practice

P1 [Easy] — Given an array, return the running sum (`pref` itself). (LC 1480)

P2 [Easy] — Rotate an array to the right by k steps in place with $O(1)$ extra space. (LC 189)

P3 [Medium] — Given an integer array, return all subarrays whose sum equals k , counted. (LC 560 — the *hashmap + prefix sum pattern*)

P4 [Medium] — Best Time to Buy and Sell Stock: one transaction max profit. (LC 121)

P5 [Challenging] — Product of Array Except Self in $O(n)$ without division. (LC 238)

P6 [Challenging] — Given an array, find the longest subarray with at most k distinct values. (*sliding window — combine with Ch. 09*)

Solutions in `27-Practice-Solutions.md`.

Chapter 04 — Strings

1. What is a string?

A string is a **sequence of characters**, usually implemented as an array of `char`. In C++, `std::string` is a dynamic array of `char` with convenience operations.

Why it matters: strings appear in ~30% of interview problems — parsing, pattern matching, palindromes, anagrams, substrings, encodings. The skills from arrays carry over, plus a few string-specific ideas.

2. C++ string essentials (quick recap)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s = "algorithm";
    cout << s.size() << "\n";           // 9
    cout << s[0] << "\n";               // 'a'
    cout << s.substr(1, 4) << "\n";      // "lgor"

    s.push_back('s');                    // append char
    s += "!";                            // append string
    s.insert(s.begin() + 2, 'X');         // insert
    s.erase(2, 1);                       // erase 1 char at index 2

    // find
    size_t pos = s.find("gor");           // index or string::npos
    if (pos != string::npos) { /* found */ }

    // comparison, reverse
    string t = s;
    reverse(t.begin(), t.end());
    bool pal = (s == t);

    // convert case
    for (char& c : s) c = tolower(c);

    // split by whitespace using stringstream
    stringstream ss("hello world foo");
    string w;
    while (ss >> w) cout << w << "\n";
    return 0;
}
```

Important: `s.size()` returns `size_t` (unsigned). Comparing with `int` directly can cause subtle bugs — cast to `int` when needed, e.g. `for (int i = 0; i < (int)s.size(); i++)`.

3. Key string patterns

Pattern A: Palindrome checking

```
bool isPalindrome(const string& s) {
    int i = 0, j = (int)s.size() - 1;
    while (i < j) {
        if (tolower(s[i]) != tolower(s[j])) return false;
        i++; j--;
    }
    return true;
}
```

Time: $O(n)$ | **Space:** $O(1)$

Pattern B: Frequency counting (anagram checking)

```
bool isAnagram(const string& a, const string& b) {
    if (a.size() != b.size()) return false;
    int cnt[26] = {0};
    for (char c : a) cnt[c - 'a']++;
    for (char c : b) cnt[c - 'a']--;
    for (int x : cnt) if (x != 0) return false;
    return true;
}
```

Time: $O(n)$ | **Space:** $O(1)$ (fixed 26 slots)

Pattern C: Substring search — the naive $O(n \cdot m)$ and its smarter cousins

Given text `t` (length n) and pattern `p` (length m), find all occurrences.

```
// Naive
for (int i = 0; i + m <= n; i++) {
    if (t.substr(i, m) == p) cout << "match at " << i << "\n";
}
//  $O(n \cdot m)$  worst case
```

The **KMP algorithm** and **Rabin-Karp** do this in $O(n+m)$. You'll rarely need to implement them in interviews, but understand the idea of Rabin-Karp (rolling hash):

- Compute hash of pattern, and rolling hash of each window of text.
- Compare hashes first; only check chars on hash collision.

```
// Rabin-Karp skeleton (base 131, mod 1e9+7)
long long mod = 1e9+7, base = 131;
long long pHash = 0, tHash = 0, powB = 1;
for (int i = 0; i < m; i++) {
    pHash = (pHash * base + p[i]) % mod;
    tHash = (tHash * base + t[i]) % mod;
    if (i < m - 1) powB = powB * base % mod;
}
for (int i = 0; i + m <= n; i++) {
    if (pHash == tHash && t.substr(i, m) == p) { /* match at i */ }
    if (i + m < n) {
        tHash = ((tHash - t[i] * powB % mod + mod) % mod * base + t[i + m]) % mod;
    }
}
```

4. The string DP/pointer patterns

Longest Palindromic Substring and **Longest Common Subsequence** are DP problems — we'll fully solve them in Chapter 19. For now, internalize that: - **Substring** = contiguous slice → sliding window/two pointers. - **Subsequence** = keep order, may skip → DP.

5. Worked examples

Example 1 — Longest substring without repeating characters (LC 3)

```
int lengthOfLongestSubstring(const string& s) {
    int n = s.size(), ans = 0;
    int last[256];
    fill(last, last + 256, -1);
    int left = 0;
    for (int right = 0; right < n; right++) {
        if (last[(unsigned char)s[right]] >= left)
            left = last[(unsigned char)s[right]] + 1; // shrink window
        last[(unsigned char)s[right]] = right;
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

Time: $O(n)$ | **Space:** $O(1)$

How it works: `last[c]` stores the most recent index where character `c` appeared. When we see `s[right]` already inside the current window, we jump `left` just past its previous occurrence. This is the sliding window pattern from Chapter 09.

Example 2 — Valid parentheses among the characters (parenthesis matching)

```
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char top = st.top(); st.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false;
        }
    }
    return st.empty();
}
```

Time: $O(n)$ | **Space:** $O(n)$

6. When to use & common mistakes

When to use strings well: pattern matching, palindromes, anagrams, substrings, parsing. Recognize when the *string nature* matters vs treating it as a char array.

Common mistakes: 1. Not checking `string::npos` with `find`. 2. `size_t` vs `int` comparisons (e.g., `i < s.size() - 1` underflows when empty). 3. Comparing with `==` on the whole string when only a substring matters. 4. Forgetting lower/upper case normalization. 5. String concatenation in a loop (`s += c` in a loop is fine; `s = s + c` is $O(n^2)$ because it copies!). Prefer `push_back` / `+=` / `ostringstream`.

7. Practice

P1 [Easy] — Check if a string is a valid palindrome ignoring non-alphanumeric chars and case. (LC 125)

P2 [Easy] — Given two strings, decide if they are anagrams. (LC 242)

P3 [Medium] — Reverse the order of words in a string. (LC 151)

P4 [Medium] — Longest common prefix of a list of strings. (LC 14)

P5 [Challenging] — Find the index of the first occurrence of pattern `p` in text `t` using $O(n+m)$. Implement either KMP or Rabin-Karp. (LC 28)

P6 [Challenging] — Given a string of digits, return all possible IP addresses by inserting dots. (*backtracking* — combine with Ch. 13)

Solutions in `27-Practice-Solutions.md`.

Chapter 05 — Searching

1. What is searching and why it matters

Searching asks: *is element x in this collection, and where?* Every data structure exists to make some search faster. Two fundamental approaches:

- **Linear search** — check every element. $O(n)$.
- **Binary search** — repeatedly halve a **sorted** range. $O(\log n)$.

Binary search is one of the highest-leverage skills in DSA: it's not just for arrays, it's for any **monotonic predicate** (see "binary search on answer" below).

2. Linear search

```
int linearSearch(const vector<int>& a, int x) {
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] == x) return i;
    return -1;
}
```

Time: $O(n)$ | **Space:** $O(1)$

When to use: unsorted data, small arrays, one-time search. STL: `find`.

3. Binary search — the classic

Prerequisite: the range is **sorted** (or has a monotonic property).

Idea: compare the middle element with the target. - Equal $\rightarrow$ found. - Middle $<$ target $\rightarrow$ target must be in the right half. - Middle $>$ target $\rightarrow$ target must be in the left half. Each step discards half the range.

```
Sorted: [1, 3, 5, 7, 9, 11, 13]   searching for 9

Step 1: lo=0 hi=6 mid=3  a[3]=7 < 9  $\rightarrow$  go right: lo=4
Step 2: lo=4 hi=6 mid=5  a[5]=11 > 9  $\rightarrow$  go left:  hi=4
Step 3: lo=4 hi=4 mid=4  a[4]=9 == 9  $\rightarrow$  found at index 4
```

The standard implementation (returns index or -1)

```
int binarySearch(const vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // avoids overflow of (lo+hi)
        if (a[mid] == x) return mid;
        else if (a[mid] < x) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;    // not found
}
```

Time: $O(\log n)$ | **Space:** $O(1)$

STL equivalents

- `binary_search(begin, end, x)` $\rightarrow$ bool.
- `lower_bound(begin, end, x)` $\rightarrow$ iterator to first element $\geq x$.
- `upper_bound(begin, end, x)` $\rightarrow$ iterator to first element $> x$.

Worked example — count occurrences of x:

```
int lo = lower_bound(a.begin(), a.end(), x) - a.begin();
int hi = upper_bound(a.begin(), a.end(), x) - a.begin();
int count = hi - lo;
```

4. Why the mid formula matters

`(lo + hi) / 2` can overflow when lo and hi are both near `INT_MAX`. `lo + (hi - lo) / 2` cannot. Use it always.

5. Binary search on the answer (the pattern that unlocks hard problems)

Many optimization problems ask: *find the minimum x such that $\text{condition}(x)$ is true*. If `condition` is **monotonic** (false, false, ..., false, true, true, ...), you can binary search **over the answer** instead of over array indices.

Classic example: given sorted array and value x , find the smallest element $\geq x$ (this is exactly `lower_bound`). But the pattern generalizes:

Problem: a pipe delivers `target` liters/hour; each worker processes `unit` per hour. Find the minimal integer time such that k workers can finish an array of jobs, where a worker's time is the max chunk they take... (simplified) → **Koko Eating Bananas / find minimal speed:**

```
bool canEat(int speed, const vector<int>& piles, int hours) {
    long long h = 0;
    for (int p : piles) h += (p + speed - 1) / speed; // ceil(p/speed)
    return h <= hours;
}

int minSpeed(const vector<int>& piles, int hours) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canEat(mid, piles, hours)) hi = mid; // feasible → try smaller
        else lo = mid + 1; // infeasible → go bigger
    }
    return lo;
}
```

Time: $O(n \log M)$ where $M = \text{max value}$ | **Space:** $O(1)$

The template you'll reuse dozens of times:

```
while (lo < hi) {
    mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid; // or lo = mid + 1, depending on direction
    else lo = mid + 1; // or hi = mid - 1
}
```

Why it works: for low speeds, `canEat` is false; for high speeds, true. The predicate flips exactly once — that's monotonicity.

6. Finding the first/last occurrence — lower_bound/upper_bound logic

```
// first index with a[i] >= x (lower_bound)
int lo = 0, hi = n;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] >= x) hi = mid;
    else lo = mid + 1;
}
// answer is lo (== n if none)
```

7. When to use & common mistakes

When to use binary search: sorted arrays; any monotonic predicate over integers or ranges; "minimize X such that...", "maximize X such that...".

Common mistakes: 1. **Forgetting to sort** before binary search. 2. **Off-by-one in the loop bound:** use `lo <= hi` for the classic version, `lo < hi` for the answer template. Mixing them breaks. 3. `mid = (lo + hi)/2` overflow (use `lo + (hi-lo)/2`). 4. Infinite loop: always update `lo/hi` to `mid + 1 / mid - 1` (classic) — never leave them equal to `mid` while the loop can repeat. 5. Applying binary search to a non-monotonic function.

8. Practice

P1 [Easy] — Search for a target in a sorted array and return its index. (LC 704)

P2 [Easy] — Given a sorted array and target, return the first and last position of target using `lower_bound`/`upper_bound` ideas. (LC 34)

P3 [Medium] — Search in a sorted rotated array. (LC 33)

P4 [Medium] — Given a sorted array of *distinct* integers `nums`, find the smallest index `i` where `nums[i] == i`. (classic *binary-search-on-answer* variant)

P5 [Medium] — Find the square root of `x` (integer floor) using binary search without `sqrt()`. (LC 69)

P6 [Challenging] — Koko Eating Bananas: given piles and `h` hours, find the minimum eating speed. (LC 875)

Solutions in `27-Practice-Solutions.md`.

Chapter 06 — Sorting

1. What is sorting and why it matters

Sorting arranges elements in order. It's useful because: - Ordered data enables **binary search** ($O(\log n)$ lookup). - Many problems are trivial on sorted data (medians, k -th smallest, duplicates, merging). - It's a building block for greedy and two-pointer solutions.

Default choice in C++: `std::sort` (introsort: quicksort + heap + insertion). **Time:** $O(n \log n)$ average & worst, **Space:** $O(\log n)$ (stack).

2. Using `std::sort` correctly

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> a = {5, 2, 8, 1, 9};
    sort(a.begin(), a.end());           // ascending: 1 2 5 8 9
    sort(a.begin(), a.end(), greater<int>()); // descending

    // custom comparison (sort pairs by first then second automatically)
    vector<pair<int,int>> p = {{1,5},{3,2},{1,3}};
    sort(p.begin(), p.end());           // {1,3},{1,5},{3,2}

    // custom comparator: sort strings by length, then alphabetically
    vector<string> words = {"banana", "apple", "kiwi", "fig"};
    sort(words.begin(), words.end(), [](const string& x, const string& y){
        if (x.size() != y.size()) return x.size() < y.size();
        return x < y;
    });

    // sorting indices (keeps original data untouched)
    vector<int> idx(a.size());
    iota(idx.begin(), idx.end(), 0);
    sort(idx.begin(), idx.end(), [&](int i, int j){ return a[i] < a[j]; });
    return 0;
}
```

Comparator contract: the comparator must be a **strict weak ordering** — it must return `true` only when `x` should come before `y`, and `x < y` must not equal `y < x`. Returning `true` for equal elements (using `<=`) is undefined behavior. Use `x < y`, never `x <= y`.

3. The classic comparison-based sorts

Selection sort — $O(n^2)$, $O(1)$ space

Repeatedly find the minimum of the remaining part and swap it to the front.

```
void selectionSort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        int mi = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[mi]) mi = j;
        swap(a[i], a[mi]);
    }
}
```

Time: $O(n^2)$ always | **Space:** $O(1)$

Insertion sort — $O(n^2)$, adaptive ($O(n)$ on nearly-sorted)

Maintain a sorted prefix; insert each new element into its correct position.

```
void insertionSort(vector<int>& a) {
    int n = a.size();
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}
```

Time: $O(n^2)$ worst, $O(n)$ best (already sorted) | **Space:** $O(1)$

Bubble sort — $O(n^2)$

Repeatedly swap adjacent out-of-order pairs; the largest "bubbles" to the end.

```
void bubbleSort(vector<int>& a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) { swap(a[j], a[j + 1]); swapped = true; }
        if (!swapped) break; // already sorted
    }
}
```

Time: $O(n^2)$, $O(n)$ best | **Space:** $O(1)$

Merge sort — $O(n \log n)$, stable, divide & conquer

```
[38, 27, 43, 3, 9, 82]
split: [38,27,43] | [3,9,82]
split: [38] [27,43] | [3] [9,82]
sort & merge bottom-up ...
```

```
void merge(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin() + l, a.begin() + m + 1);
    vector<int> R(a.begin() + m + 1, a.begin() + r + 1);
    int i = 0, j = 0, k = l;
    while (i < (int)L.size() && j < (int)R.size())
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < (int)L.size()) a[k++] = L[i++];
    while (j < (int)R.size()) a[k++] = R[j++];
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

Time: $O(n \log n)$ always | **Space:** $O(n)$ (temp arrays) | **Stable:** yes

Why stable matters: sorting by one key while preserving the relative order of equal elements (e.g., sort by score but keep original order among ties). `std::stable_sort` is merge sort.

Quick sort — $O(n \log n)$ avg, $O(n^2)$ worst, in-place

Pick a pivot, partition into " $\leq$ pivot" and " $>$ pivot", recurse on both parts.

```
int partition(vector<int>& a, int lo, int hi) {
    int pivot = a[hi];           // Lomuto partition, pivot = last element
    int i = lo;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    return i;
}

void quickSort(vector<int>& a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}
```

Time: $O(n \log n)$ average, $O(n^2)$ worst (sorted input with bad pivot) | **Space:** $O(\log n)$ expected (stack) | **Stable:** no

Worst case fix: use a random pivot or median-of-three.

4. Comparison table

Sort	Best	Average	Worst	Space	Stable
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes
Radix	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes

5. Linear-time sorts (not comparison-based)

Counting sort — when the value range is small

Count frequencies, then rebuild. $O(n + k)$ where k is the value range.

```
void countingSort(vector<int>& a, int maxVal) {
    vector<int> cnt(maxVal + 1, 0);
    for (int x : a) cnt[x]++;
    int k = 0;
    for (int v = 0; v <= maxVal; v++)
        for (int t = 0; t < cnt[v]; t++) a[k++] = v;
}
```

Time: $O(n + k)$ | **Space:** $O(k)$

Radix sort (digit by digit counting sort) — for integers up to large ranges

Sort by least significant digit, then next, etc. $O(d \cdot (n+k))$ for d digits.

6. Sorting-related problems

- **K-th smallest** → partial sort (`nth_element` in STL, $O(n)$ average).
- **Duplicates** → `unique` after sort.
- **Merge two sorted arrays** → two pointers (Ch. 08).
- **Inversion count** → merge sort with a counter ($O(n \log n)$).

7. When to use & common mistakes

When to use: preprocessing for binary search, greedy order, removing duplicates, finding k-th / median, interval problems (sort by start/end).

Common mistakes: 1. **Comparator using** `<=` instead of `<` — UB / wrong behavior. Equal elements must be indistinguishable. 2. **Mutating while iterating** after sort. 3. Expecting `std::sort` to be stable — use `stable_sort` when order matters. 4. Off-by-one in merge ($m+1$, r inclusive). 5. $O(n^2)$ insertion in a loop (`vector::insert` in the middle is $O(n)$).

8. Practice

P1 [Easy] — Given an array, sort it and return the k-th smallest element. (LC 215)

P2 [Easy] — Merge two sorted arrays. (LC 88)

P3 [Medium] — Given intervals, merge overlapping ones. (LC 56)

P4 [Medium] — Count inversions in an array using merge sort. (LC 315 is harder; use the standard inversion count problem)

P5 [Challenging] — Sort an array of 0s, 1s, and 2s (Dutch national flag) in one pass in place. (LC 75)

P6 [Challenging] — Sort a nearly sorted array where each element is at most k positions from its sorted position, in $O(n \log k)$. (heap pattern — combine with Ch. 16)

Solutions in `27-Practice-Solutions.md`.

Chapter 07 — Hashing

1. What is hashing and why it matters

A **hash table** (a.k.a. hash map / dictionary) maps **keys to values** with **$O(1)$** average lookup, insert, and delete. The secret: a **hash function** converts a key into an array index, so we can jump straight to the bucket.

```
keys:  "apple" --> hash("apple") = 42 --> bucket 42
       "cat"   --> hash("cat")   = 7  --> bucket 7
```

Why it matters: any problem that says "did I see this before?", "count these", "find the pair/duplicate/anagram" becomes $O(n)$ with hashing instead of $O(n^2)$ with nested loops.

2. How a hash table works (conceptually)

1. **Hash function:** `index = hash(key) % tableSize`. A good hash spreads keys evenly.
2. **Collisions:** two keys land in the same bucket. Handled by:
 - **Chaining:** each bucket is a linked list of entries.
 - **Open addressing:** probe to the next free slot.
3. Load factor = entries / buckets. When too high, resize (rehash).

In C++, `std::unordered_map` and `std::unordered_set` implement this. You almost never write your own — but you must understand the **$O(1)$ average / $O(n)$ worst** caveat (many collisions, e.g., adversarial input).

3. The core C++ API

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    unordered_map<string, int> m;    // key: string, value: int
    m["a"] = 1;
    m.insert({"b", 2});
    m.emplace("c", 3);

    cout << m["a"] << "\n";        // 1 (inserts default 0 if missing!)

    if (m.count("b")) cout << "b exists\n"; // presence check
    auto it = m.find("x");
    if (it != m.end()) cout << it->second;   // safe access

    m.erase("a");

    // iterate
    for (auto& [k, v] : m) cout << k << ":" << v << " ";

    unordered_set<int> s;
    s.insert(7); s.insert(7);           // only one 7 (set is unique)
    if (s.count(7)) { /* present */ }
    s.erase(7);
    return 0;
}
```

Trap: `m[key]` on a missing key **inserts** a default value. Use `count` / `find` first if you only want to read.

4. The frequency counter pattern

This is the workhorse. Count occurrences, then answer questions about counts.

Problem: find the most frequent element.

```
int mostFrequent(const vector<int>& a) {
    unordered_map<int,int> cnt;
    for (int x : a) cnt[x]++;
    int bestVal = -1, bestCnt = 0;
    for (auto& [v, c] : cnt)
        if (c > bestCnt) { bestCnt = c; bestVal = v; }
    return bestVal;
}
```

Time: $O(n)$ | **Space:** $O(n)$

5. The "two sum with hashmap" pattern

When you need to find a pair/complement, store **seen values with their indices** so a single pass suffices.

```
vector<int> twoSum(const vector<int>& nums, int target) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)nums.size(); i++) {
        int need = target - nums[i];
        if (pos.count(need))
            return {pos[need], i};
        pos[nums[i]] = i;    // store AFTER checking (avoid reusing same index)
    }
    return {};
}
```

Time: $O(n)$ | **Space:** $O(n)$

6. Prefix sum + hash = O(n) subarray-sum counting

Problem (LC 560): count subarrays whose sum equals k.

Idea: $\text{subarray}(l..r) = \text{pref}[r] - \text{pref}[l-1]$. For each r, we want pref values equal to $\text{pref}[r] - k$ seen earlier. A hashmap of prefix sums does that.

```
int subarraySum(const vector<int>& a, int k) {
    unordered_map<int,int> seen; // prefix sum -> count
    seen[0] = 1; // empty prefix has sum 0
    int cur = 0, ans = 0;
    for (int x : a) {
        cur += x;
        ans += seen[cur - k]; // how many previous prefixes make sum k?
        seen[cur]++;
    }
    return ans;
}
```

Time: O(n) | **Space:** O(n)

Worked example: a = [1,2,3], k = 3 - cur=1, seen={0:1} → ans += seen[-2]=0 → seen={0:1,1:1} - cur=3, seen[-?]=seen[0]=1 → ans=1 → seen={0:1,1:1,3:1} - cur=6, seen[3]=1 → ans=2 → answer 2 ([1,2] and [3]) ✓

7. Anagram grouping pattern

Problem (LC 49): group strings that are anagrams.

Key insight: anagrams share the same sorted form, so use the sorted string as the hash key.

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;
    for (auto& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        groups[key].push_back(s);
    }
    vector<vector<string>> res;
    for (auto& [k, v] : groups) res.push_back(v);
    return res;
}
```

Time: O(n · m log m) where m is string length | **Space:** O(n·m)

8. Custom hash for non-trivial keys

`unordered_map` needs a hash for the key type. Pairs/tuples/vectors don't have a built-in hash. Simplest fix: encode into a `long long` or use a `map` (ordered, O(log n) but no hash needed).

```
long long key = (long long)i * 1000000 + j; // pack 2D coordinate into 1 key
```

9. When to use & common mistakes

When to use hashing: - Membership tests: "have I seen this?" - Frequency / counting. - Complement problems (two sum, pair sums, differences). - Deduplication. - Caching / memoization (Ch. 19).

Common mistakes: 1. `m[key]` inserts when key missing — read via `find` if unsure. 2. Assuming O(1) always — worst case O(n); bounded alphabet sometimes allows a plain array `cnt[26]` which is truly O(1). 3. Two-sum: forgetting to store after check (self-match bug). 4. Using an ordered `map` where an `unordered_map` suffices (paying O(log n) for nothing), or vice versa (needing order but using unordered). 5. Integer overflow in prefix sums / packed keys — use `long long`.

10. Practice

P1 [Easy] — Return true if any value appears at least twice. (LC 217)

P2 [Easy] — Given two strings, decide if they are anagrams. (LC 242)

P3 [Medium] — Given an array of integers, return indices of the two numbers that sum to target. (LC 1)

P4 [Medium] — Count the number of subarrays with sum equal to k. (LC 560)

P5 [Challenging] — Group anagrams. (LC 49)

P6 [Challenging] — Given an array nums, find the longest subarray whose sum is divisible by k. (*prefix-sum mod k stored in hash — think about when two prefixes have the same remainder*)

Solutions in `27-Practice-Solutions.md`.

Chapter 08 — Two Pointers

1. What is it and why it matters

The **two pointers** technique uses two indices (pointers) that sweep the array in a coordinated way, turning an $O(n^2)$ nested-loop solution into $O(n)$.

Why it matters: it's one of the most common interview patterns, it's elegant, and it requires no extra space. It applies to sorted arrays, palindrome checks, merging, trapping water, and "find a pair that satisfies a condition".

2. The two main flavors

Flavor A: Opposite ends (squeeze) — requires sorted array (usually)

Both pointers start at the ends and move **toward each other**.

```
Sorted: [2, 7, 11, 15], target 9
L=0 R=3 sum=2+15=17 > 9 → move R left
L=0 R=2 sum=2+11=13 > 9 → move R left
L=0 R=1 sum=2+7=9 == 9 → found (0,1)
```

```
vector<int> twoSumSorted(vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return {l, r};
        else if (s < target) l++; // sum too small → need bigger: move left up
        else r--; // sum too big → need smaller: move right down
    }
    return {};
}
```

Time: $O(n)$ | **Space:** $O(1)$

Why this works: because the array is sorted. When the sum is too small, the only way to increase it with the current pair is to move `l` right; when too big, move `r` left. We never miss the answer because at every step we discard a region that cannot contain it.

Flavor B: Same direction (fast/slow or runner) — works on any array

Both pointers start at the left and move right, possibly at different speeds. Used for in-place deduplication, moving zeros, removing elements, linked-list cycle detection (Ch. 10).

```
// Remove duplicates in place, return new length (LC 26)
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int w = 0; // write pointer (slow)
    for (int r = 1; r < (int)a.size(); r++) {
```

```

        if (a[r] != a[w]) a[++w] = a[r];
    }
    return w + 1;
}

```

Time: $O(n)$ | **Space:** $O(1)$

3. Classic problem — Container With Most Water (LC 11)

Given heights, find the max water area between any two lines.

Idea: start with the widest container ($L=0$, $R=n-1$). The area depends on the **shorter** line. Since the shorter line limits the water, moving the longer line inward can never improve the area — only moving the *shorter* line could help. So always move the shorter pointer.

```

int maxArea(const vector<int>& h) {
    int l = 0, r = (int)h.size() - 1, best = 0;
    while (l < r) {
        int area = min(h[l], h[r]) * (r - l);
        best = max(best, area);
        if (h[l] < h[r]) l++;
        else r--;
    }
    return best;
}

```

Time: $O(n)$ | **Space:** $O(1)$

4. Trapping Rain Water (LC 42) — the trickiest two-pointer

Each column can hold $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$ water. Two pointers track the running max from both sides.

```

int trap(const vector<int>& h) {
    int l = 0, r = (int)h.size() - 1;
    int leftMax = 0, rightMax = 0, ans = 0;
    while (l <= r) {
        if (leftMax <= rightMax) {
            leftMax = max(leftMax, h[l]);
            ans += leftMax - h[l];
            l++;
        } else {
            rightMax = max(rightMax, h[r]);
            ans += rightMax - h[r];
            r--;
        }
    }
    return ans;
}

```

Time: $O(n)$ | **Space:** $O(1)$

Intuition: at each column, water depends on the *smaller* of the two maxes. So we process the side whose max is currently smaller — the safe side — and add the difference.

5. Merging two sorted arrays (the "merge" step of merge sort)

```

vector<int> mergeSorted(const vector<int>& a, const vector<int>& b) {
    int i = 0, j = 0;
    vector<int> res;
    while (i < (int)a.size() && j < (int)b.size()) {
        if (a[i] <= b[j]) res.push_back(a[i++]);
        else res.push_back(b[j++]);
    }
    while (i < (int)a.size()) res.push_back(a[i++]);
    while (j < (int)b.size()) res.push_back(b[j++]);
}

```

```
    return res;
}
```

Time: $O(n + m)$ | **Space:** $O(n + m)$

6. When to use & common mistakes

When to use: - Sorted array + pair/triplet sums or "compare with target". - Palindromes (from both ends). - In-place dedup / partition (move zeros, remove element). - Merging two sorted sequences. - Minimize `|a[i] - b[j]|` type problems (one pointer per array).

Common mistakes: 1. Using two pointers on an **unsorted** array when sortedness is required. 2. Off-by-one: `while (l <= r)` vs `while (l < r)` — depends on whether the middle element matters. 3. Moving the wrong pointer (e.g., moving the longer line in "most water"). 4. Not handling the case where both pointers point at the same element. 5. Forgetting that `l` and `r` must move **every** iteration to avoid infinite loops.

7. Practice

P1 [Easy] — Two Sum on a sorted array, return indices. (LC 167)

P2 [Easy] — Check if a string is a palindrome. (LC 125)

P3 [Medium] — Given a sorted array, remove duplicates in place and return the new length. (LC 26)

P4 [Medium] — Move all zeroes to the end while preserving the relative order of non-zero elements. (LC 283)

P5 [Challenging] — Three Sum: find all unique triplets that sum to zero. (LC 15)

P6 [Challenging] — Trap Rain Water. (LC 42)

Solutions in `27-Practice-Solutions.md`.

Chapter 09 — Sliding Window

1. What is it and why it matters

The **sliding window** pattern maintains a **contiguous window** (a slice) of an array or string and slides it along, updating the answer incrementally. It converts problems about "subarrays / substrings" from $O(n^2)$ (try every start & end) into $O(n)$.

Why it matters: "longest/shortest substring with property P" and "subarray with sum condition" are everywhere in interviews. The window slides, and we only do $O(1)$ work per step.

2. Fixed-size window

Problem: find the max sum of any subarray of size k .

Idea: compute the sum of the first k elements, then for each step add the new right element and remove the element leaving the window.

```
a = [2, 1, 5, 1, 3, 2], k = 3
```

```
Window: [2,1,5] sum=8
slide:  [1,5,1] sum=7   (add a[3], remove a[0])
slide:  [5,1,3] sum=9   ← max
slide:  [1,3,2] sum=6
```

```
int maxSumFixedK(const vector<int>& a, int k) {
    int n = a.size();
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];    // initial window
    int best = sum;
    for (int i = k; i < n; i++) {
        sum += a[i] - a[i - k];                // slide: add right, remove left
    }
```

```

        best = max(best, sum);
    }
    return best;
}

```

Time: $O(n)$ | **Space:** $O(1)$

3. Variable-size window (the "shrink when invalid" pattern)

Problem: find the length of the longest subarray with $\text{sum} \leq k$ (all positive).

Idea: extend `right` as far as possible. When the sum exceeds k , shrink from the `left` until the window is valid again.

```

int longestSubarraySumLEQK(const vector<int>& a, int k) {
    int left = 0, sum = 0, best = 0;
    for (int right = 0; right < (int)a.size(); right++) {
        sum += a[right]; // extend
        while (sum > k && left <= right) { // shrink until valid
            sum -= a[left++];
        }
        best = max(best, right - left + 1); // window is valid now
    }
    return best;
}

```

Time: $O(n)$ | **Space:** $O(1)$

Why $O(n)$ and not $O(n^2)$: `left` and `right` each only move forward, total n moves each. Amortized $O(n)$.

4. Variable window with a condition map (strings)

Problem (LC 3): longest substring without repeating characters.

Idea: keep the last-seen index of each char; when `s[right]` appeared inside the window, jump `left` past that occurrence.

```

int lengthOfLongestSubstring(const string& s) {
    int last[128];
    fill(last, last + 128, -1);
    int left = 0, best = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        if (last[(unsigned char)c] >= left)
            left = last[(unsigned char)c] + 1; // drop the duplicate
        last[(unsigned char)c] = right;
        best = max(best, right - left + 1);
    }
    return best;
}

```

Time: $O(n)$ | **Space:** $O(1)$ (fixed 128)

The general "condition with frequency map" template (used for "at most k distinct", "minimum window substring", etc.):

```

int slidingWindow(const string& s, int k) {
    int left = 0, valid = 0, best = 0;
    unordered_map<char, int> cnt;
    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        if (cnt[c]++ == 0) valid++; // a new distinct char entered
        while (valid > k) { // condition violated → shrink
            char d = s[left++];
            if (--cnt[d] == 0) valid--;
        }
        best = max(best, right - left + 1);
    }
}

```

```
    return best;
}
```

5. Worked example — minimum window substring (LC 76)

Find the shortest substring of `s` containing all characters of `t`.

Idea: window over `s`; track how many characters of `t` are satisfied. When all are present, shrink to find the shortest.

```
string minWindow(const string& s, const string& t) {
    int need[128] = {0}, have[128] = {0};
    int needCount = 0, haveCount = 0;
    for (char c : t) if (need[(unsigned char)c]++ == 0) needCount++;

    int left = 0, bestLen = INT_MAX, bestStart = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        char c = s[right];
        have[(unsigned char)c]++;
        if (need[(unsigned char)c] && have[(unsigned char)c] == need[(unsigned char)c])
            haveCount++;
        while (haveCount == needCount) {           // valid → try to shrink
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestStart = left;
            }
            char d = s[left++];
            if (need[(unsigned char)d] && have[(unsigned char)d] == need[(unsigned char)d])
                haveCount--;
            have[(unsigned char)d]--;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(bestStart, bestLen);
}
```

Time: $O(n)$ | **Space:** $O(1)$

6. When to use & common mistakes

When to use: questions mentioning **subarray/substring** + a constraint (sum $\leq k$, at most k distinct, contains all chars, no repeats). Ask: "can the answer be a contiguous slice?" If yes, sliding window may apply.

Common mistakes: 1. Using a **fixed** window template for a variable-size problem (or vice versa). 2. Forgetting that the data must allow the window to shrink correctly — negative numbers break the "sum $> k$ then shrink" logic (the window is no longer monotonic). For negative numbers, use prefix sums or other techniques. 3. Infinite loop: make sure `left` always advances in the shrink loop. 4. Updating `best` at the wrong place (after shrinking vs before). 5. Not handling empty string / $k=0$ edge cases.

7. Practice

P1 [Easy] — Given a binary array, find the max number of consecutive 1s. (LC 485)

P2 [Medium] — Longest substring without repeating characters. (LC 3)

P3 [Medium] — Max sum of a subarray of size k . (LC 643 is average; adapt)

P4 [Medium] — Longest substring with at most k distinct characters. (adapted from LC 340)

P5 [Challenging] — Minimum window substring. (LC 76)

P6 [Challenging] — Given an array of positive ints and target k , return the minimal length of a contiguous subarray with sum $\geq k$. (LC 209)

Solutions in `27-Practice-Solutions.md`.

Chapter 10 — Linked Lists

1. What is a linked list and why it matters

A **linked list** is a chain of **nodes**, each holding a value and a pointer to the next node. Unlike an array, the nodes are **not contiguous** in memory — each one is allocated separately and linked via pointers.

```
head → [5|·] → [12|·] → [-3|·] → [8|∅]
      value next
```

Why it matters: $O(1)$ insertion/deletion at a known position (no shifting), dynamic size without reallocation, and it teaches pointer manipulation — the foundation for trees and graphs. Also: interviews *love* testing whether you can rewire pointers (reverse, detect cycle, merge, remove).

2. Basic structure and operations

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {}
};
```

Traversal

```
void printList(ListNode* head) {
    while (head != nullptr) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << "\n";
}
```

Time: $O(n)$ | **Space:** $O(1)$

Insert at head ($O(1)$)

```
ListNode* insertAtHead(ListNode* head, int v) {
    ListNode* n = new ListNode(v);
    n->next = head;      // new node points to old head
    return n;           // new node is the new head
}
```

Delete a node given only a pointer to it (LC 237 trick)

You don't have access to the previous node — so **copy** the next node's value into the current one, then unlink the next node.

```
void deleteNode(ListNode* node) {
    node->val = node->next->val;    // copy successor's value
    ListNode* tmp = node->next;
    node->next = node->next->next;  // skip successor
    delete tmp;
}
```

3. Building a list from a vector

```
ListNode* build(const vector<int>& v) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    for (int x : v) {
        tail->next = new ListNode(x);
        tail = tail->next;
    }
}
```

```
    return dummy.next; // dummy node avoids special-casing the empty list
}
```

4. Core algorithms

Reverse a linked list (LC 206) — the #1 linked-list question

Original: 1 → 2 → 3 → 4 → ∅
 Reversed: ∅ ← 1 ← 2 ← 3 ← 4

Idea: walk with three pointers. `prev` holds what we've reversed so far; `cur` is the node being processed; `nxt` saves the rest before we break the link.

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur) {
        ListNode* nxt = cur->next; // save the rest
        cur->next = prev;          // reverse this link
        prev = cur;
        cur = nxt;
    }
    return prev; // prev is the new head
}
```

Time: O(n) | **Space:** O(1)

Find the middle (slow & fast pointers)

```
ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) { // fast moves 2x, slow moves 1x
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow; // when fast hits the end, slow is the middle
}
```

Time: O(n) | **Space:** O(1)

Detect a cycle — Floyd's tortoise & hare (LC 141)

If a cycle exists, the fast pointer eventually catches the slow one from behind.

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true; // collision ⇒ cycle
    }
    return false;
}
```

Time: O(n) | **Space:** O(1)

Find the node where the cycle begins (LC 142)

After the pointers meet, reset one to head and advance **both one step at a time** — they meet at the cycle start. (Proof: the distance from head to cycle start equals the distance from the meeting point around the cycle to the cycle start.)

```

ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { // cycle found
            slow = head;
            while (slow != fast) { slow = slow->next; fast = fast->next; }
            return slow;
        }
    }
    return nullptr;
}

```

Merge two sorted lists (LC 21)

```

ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* t = &dummy;
    while (a && b) {
        if (a->val <= b->val) { t->next = a; a = a->next; }
        else { t->next = b; b = b->next; }
        t = t->next;
    }
    t->next = a ? a : b;
    return dummy.next;
}

```

Time: $O(n + m)$ | **Space:** $O(1)$

Remove the n-th node from the end (LC 19)

Advance `fast` by `n` first, then move both until `fast` reaches the end — `slow` now points to the node just before the target.

```

ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* fast = &dummy;
    ListNode* slow = &dummy;
    for (int i = 0; i <= n; i++) fast = fast->next; // fast is n+1 ahead
    while (fast) { slow = slow->next; fast = fast->next; }
    ListNode* del = slow->next;
    slow->next = slow->next->next; // unlink
    delete del;
    return dummy.next;
}

```

Time: $O(n)$ | **Space:** $O(1)$

5. Doubly linked lists

Each node also has a `prev` pointer. $O(1)$ deletion when you have the node directly (no need to find predecessor). Used to implement LRU caches (LC 146).

```

struct DNode {
    int key, val;
    DNode* prev;
    DNode* next;
    DNode(int k, int v) : key(k), val(v), prev(nullptr), next(nullptr) {}
};

```

6. The dummy node trick (learn this — it saves hours)

Always use a `dummy` head node when the real head might change (deletion, insertion at front). Then `return dummy.next` handles all edge cases uniformly.

7. When to use & common mistakes

When to use: the STL `std::list` exists, but interview problems want you to implement node manipulation yourself. Real-world: undo/redo, browser history, LRU cache.

Common mistakes: 1. **Dereferencing `nullptr`** — always null-check before `->next`. 2. Forgetting to save `nxt` before rewiring (loses the rest of the list). 3. Infinite loops in cycle detection code that never advances. 4. Confusing `slow / fast` roles or off-by-one in the "remove n-th from end" offset. 5. Returning the wrong head after reversal/deletion — the head **changes**. 6. Memory leaks — `delete` nodes you remove (in interviews, usually fine to skip, but good practice).

8. Practice

P1 [Easy] — Reverse a singly linked list. (LC 206)

P2 [Easy] — Merge two sorted linked lists. (LC 21)

P3 [Medium] — Detect whether a linked list has a cycle. (LC 141)

P4 [Medium] — Remove the n-th node from the end of a list. (LC 19)

P5 [Challenging] — Return the node where the cycle begins. (LC 142)

P6 [Challenging] — Reorder list: $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ (LC 143 — use *middle + reverse + merge*)

P7 [Challenging] — Implement an LRU cache. (LC 146 — *doubly linked list + hashmap*)

Solutions in `27-Practice-Solutions.md`.

Chapter 11 — Stacks & Queues / Deques

1. What are they and why they matter

Stack — LIFO (Last In, First Out). Like a stack of plates: you can only take the top one.

Queue — FIFO (First In, First Out). Like a checkout line: first to arrive, first served.

Deque — double-ended queue: push/pop from both ends.

Why they matter: stacks naturally model nesting (parentheses, function calls, DFS, backtracking, undo), and **monotonic stacks** solve the classic "next greater / nearest smaller element" problems in $O(n)$. Queues model scheduling, BFS, and level-order traversal.

2. The three operations that matter

Operation	Stack	Queue	Deque
Push	<code>push (top)</code>	<code>push (back)</code>	<code>push_back / push_front</code>
Pop	<code>pop (top)</code>	<code>pop (front)</code>	<code>pop_back / pop_front</code>
Peek	<code>top</code>	<code>front / back</code>	<code>front / back</code>
Complexity	$O(1)$	$O(1)$	$O(1)$

```
stack<int> st; st.push(1); st.push(2); st.push(3);
cout << st.top();           // 3
st.pop();                   // removes 3 (void!)
cout << st.top();           // 2

queue<int> q; q.push(1); q.push(2); q.push(3);
cout << q.front();          // 1
```

```
q.pop();           // removes 1
cout << q.front(); // 2

deque<int> dq; dq.push_back(1); dq.push_front(0);
dq.pop_back(); dq.pop_front();
```

3. Classic stack problem — matching parentheses (LC 20)

Problem: is "{[()]}" valid? Every opener must be closed by the matching closer, in the correct nesting order.

Idea: push openers onto the stack. When you see a closer, it must match the top of the stack (the most recent unmatched opener).

```
s = "{[()]}"
( → push
{ → push
[ → push
] → matches top [ → pop
) → matches top { → pop
} → matches top ( → pop
stack empty ⇒ valid
```

```
bool isValid(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false; // no matching opener
            char top = st.top(); st.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) return false; // mismatched type
        }
    }
    return st.empty(); // all openers matched
}
```

Time: $O(n)$ | **Space:** $O(n)$

4. The monotonic stack pattern (very important)

A **monotonic stack** keeps elements in increasing or decreasing order as we scan. It answers "next greater", "next smaller", "nearest greater", etc. in $O(n)$.

Problem (LC 496/739): for each element, find the index of the **next greater element** to its right (or -1).

```
a = [73, 74, 75, 71, 69, 72, 76, 73]
next greater (index): [1, 2, 6, 5, 5, 6, -1, -1]
```

Idea: scan right to left, keeping a stack of *candidates* that is strictly increasing in value from bottom to top. For $a[i]$, pop everything smaller or equal — those can't be a "next greater" for anyone further left. The stack top is the answer.

```
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices, values strictly increasing
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && a[st.top()] <= a[i]) st.pop(); // pop smaller/equal
        if (!st.empty()) res[i] = a[st.top()];
        st.push(i);
    }
    return res;
}
```

Time: $O(n)$ — each element pushed & popped at most once | **Space:** $O(n)$

Why it's correct: when we process `i` from right to left, the stack contains only elements to the right of `i` with strictly increasing values. Popping smaller/equal elements is safe because those elements can never be the "next greater" for any element further left (the current `a[i]` is bigger and closer). This "popping the dominated candidates" idea is the heart of every monotonic stack problem.

Nearest smaller to the left — the mirror image

```
vector<int> prevSmaller(const vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st; // increasing from bottom (values)
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] >= a[i]) st.pop();
        if (!st.empty()) res[i] = st.top();
        st.push(i);
    }
    return res;
}
```

5. Largest rectangle in histogram (LC 84) — the classic hard problem

For each bar, it can stretch left and right until hitting a **shorter** bar. The previous-smaller and next-smaller indices bound its rectangle. Using two monotonic stack passes:

```
int largestRectangleArea(const vector<int>& h) {
    int n = h.size();
    vector<int> left(n), right(n);
    stack<int> st;

    for (int i = 0; i < n; i++) { // nearest smaller to the left (index)
        while (!st.empty() && h[st.top()] >= h[i]) st.pop();
        left[i] = st.empty() ? 0 : st.top() + 1;
        st.push(i);
    }
    while (!st.empty()) st.pop();
    for (int i = n - 1; i >= 0; i--) { // nearest smaller to the right (index)
        while (!st.empty() && h[st.top()] >= h[i]) st.pop();
        right[i] = st.empty() ? n - 1 : st.top() - 1;
        st.push(i);
    }
    int best = 0;
    for (int i = 0; i < n; i++)
        best = max(best, h[i] * (right[i] - left[i] + 1));
    return best;
}
```

Time: $O(n)$ | **Space:** $O(n)$

6. Queue / deque classic — sliding window maximum (LC 239)

Problem: for each window of size `k`, find its maximum.

Idea: keep a **deque of candidate indices** with decreasing values. The front is the window maximum. Before each step, evict indices that leave the window and indices whose value is dominated by the newly added one.

```
vector<int> maxSlidingWindow(const vector<int>& a, int k) {
    deque<int> dq; // indices, values decreasing front-back
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // out of window
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // dominated
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

Time: $O(n)$ | **Space:** $O(k)$

7. When to use & common mistakes

When to use: - Stack: nesting/parsing, "undo", DFS, backtracking, **next/previous greater or smaller** (monotonic), evaluating expressions. - Queue: BFS, level-order, scheduling, streaming. - Deque: sliding window min/max, palindrome-checking from both ends.

Common mistakes: 1. `pop()` returns void — read `top()` / `front()` first. 2. Forgetting to check `empty()` before `top()` / `pop()`. 3. In monotonic stack, choosing `<=` vs `<` in the pop condition changes whether equal elements are counted as blockers — think about duplicates. 4. Off-by-one in window eviction (`<= i - k`). 5. Using `stack` where BFS needs `queue` (or vice versa).

8. Practice

P1 [Easy] — Valid parentheses. (LC 20)

P2 [Easy] — Implement a queue using two stacks. (LC 232)

P3 [Medium] — Daily temperatures: for each day, days until a warmer day. (LC 739 — *next greater*)

P4 [Medium] — Evaluate Reverse Polish Notation. (LC 150)

P5 [Challenging] — Largest rectangle in histogram. (LC 84)

P6 [Challenging] — Sliding window maximum. (LC 239)

Solutions in `27-Practice-Solutions.md`.

Chapter 12 — Recursion

1. What is recursion and why it matters

Recursion is when a function calls **itself** to solve a smaller instance of the same problem. Every recursive function needs:

1. **Base case** — when to stop (returns directly, no recursion).
2. **Recursive case** — reduce the problem toward the base case.

```
f(5) = 5 * f(4)
      = 5 * 4 * f(3)
      = 5 * 4 * 3 * f(2)
      = 5 * 4 * 3 * 2 * f(1)
      = 5 * 4 * 3 * 2 * 1      ← base case reached
```

Why it matters: recursion is the natural way to express many problems — trees, graphs, divide & conquer, backtracking, DP. If you can't write recursion, you can't write most of the rest of this book.

2. The anatomy of a recursive function

```
int factorial(int n) {
    if (n <= 1) return 1;      // base case
    return n * factorial(n - 1); // recursive case
}
```

Time: $O(n)$ | **Space:** $O(n)$ (call stack)

Important: the call stack matters. Each call allocates a frame. Deep recursion (100k+) can **overflow the stack** in C++. That's why some solutions use an explicit stack/iteration.

3. How the call stack works

```
main()
  factorial(5)
    factorial(4)
      factorial(3)
        ...
        factorial(1) → returns 1
    unwinds: 2*1=2, 3*2=6, 4*6=24, 5*24=120
```

4. Core recursion examples

Sum of digits

```
int sumDigits(int n) {
  if (n == 0) return 0;
  return n % 10 + sumDigits(n / 10);
}
```

Check if a string is a palindrome recursively

```
bool isPal(const string& s, int l, int r) {
  if (l >= r) return true;           // base: nothing to compare
  if (s[l] != s[r]) return false;
  return isPal(s, l + 1, r - 1);    // shrink both ends
}
```

Power (fast exponentiation) — $O(\log n)$

```
313 = 36 · 36 · 3      (odd → one extra multiply)
36  = 33 · 33
33  = 31 · 31 · 3
```

```
long long power(long long a, long long n) {
  if (n == 0) return 1;
  long long half = power(a, n / 2);
  half = half * half;
  return (n % 2 == 0) ? half : half * a;
}
```

Time: $O(\log n)$ | **Space:** $O(\log n)$

Fibonacci — the cautionary tale

```
int fib(int n) {                      // NAIVE
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}
//  $O(2^n)$  — same subproblem computed thousands of times
```

```
      fib(5)
     /  \
    fib(4) fib(3)
   /  \  /  \
  fib(3) fib(2) fib(2) fib(1)
 /  \
fib(2) fib(1) ← fib(2) recomputed 3 times!
```

The fix is memoization (Chapter 19). Whenever recursion recomputes the same state, cache results.

5. The recursion checklist (how to approach any recursive problem)

1. **Define the function signature** — what does it take, what does it return?
2. **State the recursive definition in words** — " $f(n) = \dots$ depending on $f(n-1)$ ".

3. **Find the base case(s)** — the smallest inputs, returned directly.
4. **Trust the recursion** — assume `f(n-1)` is correct, build `f(n)` from it.
5. **Verify termination** — every recursive call moves toward a base case.

6. When to use & common mistakes

When to use: problems with a natural recursive structure (trees, nested structures, divide & conquer), anything that can be defined "in terms of a smaller version of itself".

Common mistakes: 1. **Missing or wrong base case** → infinite recursion / stack overflow. 2. **No progress** — recursive call with the same argument → infinite loop. 3. **Returning before combining** — the result of the recursive call must be used. 4. **Stack overflow** on deep recursion — iterative or tail-call rewrite for very deep recursion; or increase stack. 5. **Side effects** — modifying shared state in a way that breaks other branches (see backtracking, Ch. 13).

7. Practice

P1 [Easy] — Reverse a string in place using recursion. (LC 344)

P2 [Easy] — Compute n-th Fibonacci with recursion. (LC 509)

P3 [Medium] — Given a string, print all subsets (combinations) of its characters. (power set — LC 78 is the array version)

P4 [Medium] — Tower of Hanoi: move n disks from rod A to rod C using rod B. (classic)

P5 [Challenging] — Given n pairs of parentheses, generate all well-formed combinations. (LC 22 — combine recursion with a running count of open/close)

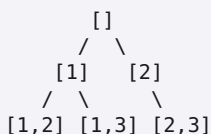
P6 [Challenging] — Implement fast exponentiation, then use it to compute `a^n mod m`. (modular exponentiation)

Solutions in `27-Practice-Solutions.md`.

Chapter 13 — Backtracking

1. What is backtracking and why it matters

Backtracking is a systematic way to search through all possibilities: build a solution step by step, and **when a step leads to a dead end, undo it ("backtrack") and try the next option**.



Why it matters: it's the general technique for "generate all combinations / permutations / subsets", Sudoku, N-Queens, path finding in mazes, and constraint problems. It's essentially DFS over a *decision space* (Ch. 12 + Ch. 18).

2. The backtracking skeleton

```
void backtrack(state, choices) {
    if (isSolution(state)) {
        record(state);
        return;
    }
    for (choice : choices) {
        if (isValid(state, choice)) {
            makeChoice(state, choice); // commit
            backtrack(state, choices); // recurse deeper
            undoChoice(state, choice); // ← THE "backtrack" step
        }
    }
}
```

```

    }
}

```

The **undo step** is what makes it backtracking. Without undo, choices leak into other branches.

3. Example 1 — generate all subsets (power set, LC 78)

Problem: given distinct ints, return all subsets.

Decision: for each element, either *include* it or *exclude* it.

```

vector<vector<int>> subsets(const vector<int>& nums) {
    vector<vector<int>> res;
    vector<int> cur;
    function<void(int)> dfs = [&](int i) {
        if (i == (int)nums.size()) { res.push_back(cur); return; }

        cur.push_back(nums[i]); // choose to include
        dfs(i + 1);
        cur.pop_back(); // undo (backtrack)

        dfs(i + 1); // choose to exclude
    };
    dfs(0);
    return res;
}

```

Time: $O(2^n)$ subsets, each $O(1)$ to record | **Space:** $O(n)$ recursion depth

Worked example: nums = [1,2] - dfs(0): include 1 → dfs(1): include 2 → dfs(2) record [1,2]; undo 2; exclude 2 → record [1] - undo 1; exclude 1 → dfs(1): include 2 → record [2]; exclude 2 → record [] - Result: [1,2], [1], [2], [] ✓

4. Example 2 — generate all permutations (LC 46)

Decision: pick which unused element goes next. Track used indices.

```

vector<vector<int>> permute(const vector<int>& nums) {
    vector<vector<int>> res;
    vector<int> cur;
    vector<bool> used(nums.size(), false);
    function<void()> dfs = [&]() {
        if (cur.size() == nums.size()) { res.push_back(cur); return; }
        for (int i = 0; i < (int)nums.size(); i++) {
            if (used[i]) continue;
            used[i] = true;
            cur.push_back(nums[i]);
            dfs();
            cur.pop_back(); // undo
            used[i] = false; // undo
        }
    };
    dfs();
    return res;
}

```

Time: $O(n!)$ | **Space:** $O(n)$

5. Example 3 — N-Queens (LC 51)

Place n queens so none attack each other. **Decision:** which column in each row. **Pruning:** a column or diagonal already used cannot be chosen.

```

vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> res;
    vector<int> col(n, -1); // col[row] = column used
    vector<bool> usedCol(n, false), d1(2*n, false), d2(2*n, false);
}

```

```

function<void(int)> dfs = [&](int row) {
    if (row == n) {
        vector<string> board(n, string(n, '.'));
        for (int r = 0; r < n; r++) board[r][col[r]] = 'Q';
        res.push_back(board);
        return;
    }
    for (int c = 0; c < n; c++) {
        if (usedCol[c] || d1[row + c] || d2[row - c + n]) continue;
        col[row] = c; usedCol[c] = true;
        d1[row + c] = true; d2[row - c + n] = true;
        dfs(row + 1);
        usedCol[c] = false;
        d1[row + c] = false; d2[row - c + n] = false;
    }
};
dfs(0);
return res;
}

```

Time: $O(n!)$ | **Space:** $O(n)$

Diagonal trick: all squares on the same $\backslash$ diagonal share $row + col$; all squares on the same $/$ diagonal share $row - col$. Add $+n$ to keep the index non-negative.

6. Pruning — why backtracking can beat brute force

Backtracking's power comes from **pruning**: detecting early that a partial solution can never become valid, and stopping that branch immediately. In N-Queens, a wrong column is rejected before placing later queens. This turns naive $O(n^n)$ into practical $O(n!)$.

7. When to use & common mistakes

When to use: "generate all", "find all ways", "is there a way to fill/place/split" with a **small** input ($n \leq \sim 15-20$, because it's exponential). If input is large, think greedy or DP instead.

Common mistakes: 1. **Forgetting to undo** — the most common bug; state leaks across branches. 2. **Not copying the result** before modifying the shared `cur` vector. 3. **No pruning** — exploring branches that can never succeed. 4. **Mutating global state** that other branches rely on. 5. **Infinite recursion** — missing base case or choices that don't shrink state.

8. Practice

P1 [Easy] — Subsets. (LC 78)

P2 [Medium] — Generate all well-formed parentheses for n pairs. (LC 22)

P3 [Medium] — Permutations. (LC 46)

P4 [Medium] — Combination Sum: find all combinations that sum to target (numbers can repeat). (LC 39)

P5 [Challenging] — N-Queens. (LC 51)

P6 [Challenging] — Sudoku Solver. (LC 37)

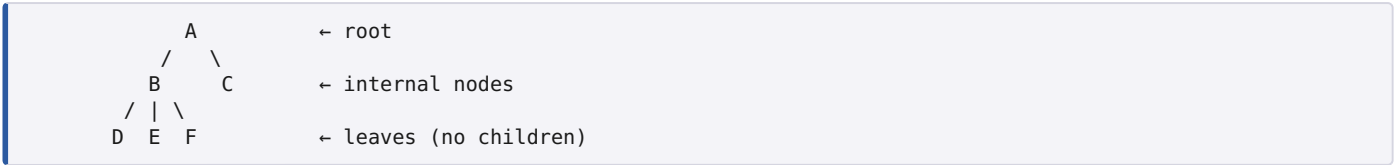
P7 [Challenging] — Word Search: does a word exist in a grid by moving up/down/ left/right, visiting each cell once? (LC 79)

Solutions in `27-Practice-Solutions.md`.

Chapter 14 — Trees

1. What is a tree and why it matters

A **tree** is a **hierarchical** data structure: a root node with children, each child being the root of a **subtree**. Every node has exactly one parent (except the root).



Why it matters: trees model any hierarchy — file systems, HTML/DOM, JSON, family trees, expression trees, decision trees. They're also the stepping stone to BSTs, heaps, tries, segment trees, and to graph search (BFS/DFS are tree ideas applied to graphs).

2. Vocabulary

Term	Meaning
Root	the top node (no parent)
Child	node directly below another
Parent	node directly above another
Sibling	nodes sharing a parent
Leaf	node with no children
Height	longest path from node to a leaf (root height = tree height)
Depth	number of edges from root to the node
Level	nodes at the same depth
Subtree	a node plus all its descendants
Binary tree	each node has at most 2 children

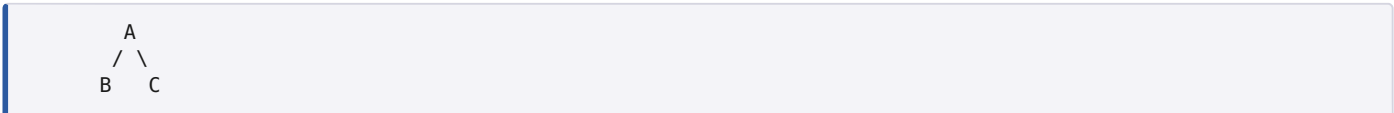
3. Representing a binary tree in C++

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

Bonus property: in a perfect binary tree of height h there are 2^h leaves and $2^{(h+1)} - 1$ total nodes. So a tree with n nodes has height $O(\log n)$ if *balanced*, but up to $O(n)$ if it degenerates into a chain.

4. The four traversals

Traversal	Order	Used for
Preorder	root → left → right	copying, serialization
Inorder	left → root → right	sorted output (on BST)
Postorder	left → right → root	deletion, free memory
Level-order (BFS)	row by row	shortest path in unweighted trees



```

    / \ \
   D  E F

```

```

Preorder:  A B D E C F
Inorder:   D B E A C F
Postorder: D E B F C A
Levelorder: A B C D E F

```

Recursive traversals

```

void preorder(TreeNode* r) {
    if (!r) return;
    cout << r->val << " ";    // visit root first
    preorder(r->left);
    preorder(r->right);
}

void inorder(TreeNode* r) {
    if (!r) return;
    inorder(r->left);
    cout << r->val << " ";    // visit root in the middle
    inorder(r->right);
}

void postorder(TreeNode* r) {
    if (!r) return;
    postorder(r->left);
    postorder(r->right);
    cout << r->val << " ";    // visit root last
}

```

Time: $O(n)$ | **Space:** $O(h)$ for the stack ($O(n)$ worst if skewed)

Level-order traversal (BFS with a queue)

```

void levelOrder(TreeNode* root) {
    if (!root) return;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* cur = q.front(); q.pop();
        cout << cur->val << " ";
        if (cur->left) q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
}

```

Time: $O(n)$ | **Space:** $O(n)$ (queue holds a level at a time)

Level-by-level variant (LC 102 — returns vector of levels)

```

vector<vector<int>> levelOrderByLevel(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();    // size of current level
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* cur = q.front(); q.pop();
            level.push_back(cur->val);
            if (cur->left) q.push(cur->left);
            if (cur->right) q.push(cur->right);
        }
        res.push_back(level);
    }
    return res;
}

```

5. Key tree problems & their patterns

Max depth of a tree (LC 104)

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(maxDepth(root->left), maxDepth(root->right));  
}
```

Time: $O(n)$ | **Space:** $O(h)$

Symmetric tree (LC 101) — compare left & right mirrored

```
bool isMirror(TreeNode* a, TreeNode* b) {  
    if (!a && !b) return true;  
    if (!a || !b) return false;  
    return a->val == b->val  
        && isMirror(a->left, b->right)  
        && isMirror(a->right, b->left);  
}  
bool isSymmetric(TreeNode* root) {  
    return root ? isMirror(root->left, root->right) : true;  
}
```

Diameter of a binary tree (LC 543)

Longest path between any two nodes. At each node, the longest path *through* it is `leftDepth + rightDepth`.

```
int diameterOfBinaryTree(TreeNode* root) {  
    int best = 0;  
    function<int(TreeNode*)> depth = [&](TreeNode* r) {  
        if (!r) return 0;  
        int l = depth(r->left), rr = depth(r->right);  
        best = max(best, l + rr); // path through r  
        return 1 + max(l, rr); // depth of r  
    };  
    depth(root);  
    return best;  
}
```

Time: $O(n)$ | **Space:** $O(h)$

Lowest Common Ancestor (LC 236)

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {  
    if (!root || root == p || root == q) return root;  
    TreeNode* l = lowestCommonAncestor(root->left, p, q);  
    TreeNode* r = lowestCommonAncestor(root->right, p, q);  
    if (l && r) return root; // p and q are in different subtrees → root is LCA  
    return l ? l : r;  
}
```

Intuition: if both *p* and *q* are found in different children, the current node is the LCA. If only one child found a match, that child's result carries up.

6. When to use & common mistakes

When to use trees: any hierarchy, anything where you process children before/after the parent (postorder for "compute from children"), anything needing a sorted or priority structure (BST/heap).

Common mistakes: 1. **Dereferencing null child** — always check `if (!node)` first. 2. **Forgetting base case** `if (!root)` → stack overflow. 3. In recursion, mixing up return values (e.g., returning depth where a bool is needed). 4. Level-order: forgetting to capture `sz` before popping (the queue changes size). 5. Thinking a tree must be binary — generalize for *n*-ary. 6. Not knowing when to use recursion vs. iterative (deep trees → prefer iterative or explicit stack).

7. Practice

P1 [Easy] — Maximum depth of a binary tree. (LC 104)

P2 [Easy] — Invert a binary tree. (LC 226)

P3 [Medium] — Binary tree level order traversal. (LC 102)

P4 [Medium] — Check if a binary tree is symmetric. (LC 101)

P5 [Challenging] — Validate a binary search tree. (LC 98 — *hint: use inorder or a min/max bound*)

P6 [Challenging] — Construct a binary tree from preorder and inorder traversals. (LC 105)

P7 [Challenging] — Lowest common ancestor. (LC 236)

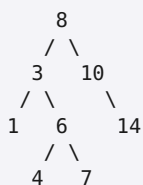
Solutions in `27-Practice-Solutions.md`.

Chapter 15 — Binary Search Trees

1. What is a BST and why it matters

A **Binary Search Tree (BST)** is a binary tree with the **ordering property**:

For every node: all values in the **left subtree** < **node value** < **all values in the right subtree**.



Why it matters: the ordering makes search, insert, and delete **$O(\log n)$** on a *balanced* tree — we discard half the tree at every step, exactly like binary search but on dynamic data. BSTs power ordered maps/sets in many languages (in C++, `std::map` uses a red-black tree, a self-balancing BST).

2. Core operations

Search — $O(\log n)$ balanced, $O(n)$ skewed

Search for 6:
8 > 6 → go left
3 < 6 → go right
6 == 6 → found

```
bool search(TreeNode* root, int x) {  
    if (!root) return false;  
    if (root->val == x) return true;  
    if (x < root->val) return search(root->left, x);  
    return search(root->right, x);  
}
```

Time: $O(h)$ where h = height = $O(\log n)$ balanced | **Space:** $O(h)$

Insert — $O(\log n)$

Insert follows the search path and attaches the new node where the search would fail.

```
TreeNode* insert(TreeNode* root, int x) {  
    if (!root) return new TreeNode(x);  
    if (x < root->val) root->left = insert(root->left, x);  
    else if (x > root->val) root->right = insert(root->right, x);  
}
```

```

    return root; // equal → do nothing (duplicates not stored)
}

```

Delete — the trickiest (3 cases)

```

TreeNode* minNode(TreeNode* root) {
    while (root && root->left) root = root->left;
    return root;
}

TreeNode* removeNode(TreeNode* root, int x) {
    if (!root) return nullptr;
    if (x < root->val) root->left = removeNode(root->left, x);
    else if (x > root->val) root->right = removeNode(root->right, x);
    else { // found the node to delete
        if (!root->left) return root->right; // case 1: no left child
        if (!root->right) return root->left; // case 2: no right child
        // case 3: two children → replace with inorder successor
        TreeNode* succ = minNode(root->right);
        root->val = succ->val;
        root->right = removeNode(root->right, succ->val);
    }
    return root;
}

```

Case 3 explanation: with two children, replace the node's value with its **inorder successor** (smallest value in the right subtree), then delete that successor from the right subtree. The successor has at most one child, so deletion is easy.

3. Inorder traversal of a BST = sorted output

This is the most important BST fact:

```

void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}
// prints 1 3 4 6 7 8 10 14 for the tree above

```

Time: $O(n)$

Consequences: - **Validate BST (LC 98):** inorder must be strictly increasing. - **k-th smallest element:** k-th element in inorder (LC 230). - **BST to sorted array** and back: trivial.

4. Validating a BST (LC 98) — the classic trap

Trap: checking only `left < node < right` is **not enough**:

```

      5
     /\
    3  7
   /\  /\
  6  8

← valid? left subtree of 7: 6 < 7 ✓ but 6 > 5!

```

You must carry the **allowed range (min, max)** down the recursion.

```

bool valid(TreeNode* root, long long lo, long long hi) {
    if (!root) return true;
    if (root->val <= lo || root->val >= hi) return false;
    return valid(root->left, lo, root->val) &&
           valid(root->right, root->val, hi);
}

bool isValidBST(TreeNode* root) {
    return valid(root, LLONG_MIN, LLONG_MAX);
}

```

Time: $O(n)$ | **Space:** $O(h)$

Alternative: do inorder and check it's strictly increasing.

5. Finding the k-th smallest element (LC 230)

```
int kthSmallest(TreeNode* root, int k) {
    int count = 0, ans = -1;
    function<void(TreeNode*)> inorder = [&](TreeNode* r) {
        if (!r || count >= k) return;
        inorder(r->left);
        if (++count == k) { ans = r->val; return; }
        inorder(r->right);
    };
    inorder(root);
    return ans;
}
```

Time: $O(h + k)$ | **Space:** $O(h)$

6. Balanced BSTs — why order matters

The tree above is **balanced** (each side similar height) $\rightarrow O(\log n)$ operations. But insert 1,2,3,4,5,6,7 in order and you get a **skewed chain** $\rightarrow$ operations degrade to $O(n)$, same as a linked list.

Self-balancing BSTs (AVL, Red-Black) rebalance after every insert/delete to keep height $O(\log n)$. In C++, `std::map` / `std::set` do this for you. You rarely implement them by hand — but you must *know why* `std::map` is $O(\log n)$.

7. When to use & common mistakes

When to use: you need ordered, dynamic data with fast search/insert/delete; you need "nearest value", "predecessor/successor", "k-th smallest". If you don't need *order*, prefer a hash map ($O(1)$).

Common mistakes: 1. **Insert/delete not updating the parent pointer** — must `root->left = insert(...)`. 2. **The validation trap** — forgetting to pass (min, max) bounds. 3. Duplicates: decide policy (ignore, allow left or right) and be consistent. 4. Deleting a node with two children without handling the successor. 5. Assuming a BST is always balanced — worst case is a chain.

8. Practice

P1 [Easy] — Search in a BST. (LC 700)

P2 [Easy] — Insert into a BST. (LC 701)

P3 [Medium] — Validate a BST. (LC 98)

P4 [Medium] — Find the k-th smallest element in a BST. (LC 230)

P5 [Challenging] — Delete a node in a BST. (LC 450)

P6 [Challenging] — Given a BST, convert it to a "greater tree" where each node's value becomes its original value plus the sum of all values greater than it. (LC 538 — *reverse inorder*)

Solutions in `27-Practice-Solutions.md`.

Chapter 16 — Heaps / Priority Queues

1. What is a heap and why it matters

A **heap** is a complete binary tree stored in an array, with the **heap property**:

- **Max-heap**: every node $\geq$ its children $\rightarrow$ the **maximum is at the root**.
- **Min-heap**: every node $\leq$ its children $\rightarrow$ the **minimum is at the root**.

Max-heap (as tree):	Stored as array:
<pre> 90 / \ 80 70 / \ / \ 40 50 60 20 / 30</pre>	<pre>[90, 80, 70, 40, 50, 60, 20, 30]</pre>
	children of index i:
	left = $2*i+1$
	right = $2*i+2$
	parent of i = $(i-1)/2$

Why it matters: you get **$O(\log n)$** push and pop of the extreme element, and **$O(1)$** access to it. That's the perfect tool for "repeatedly get the min/max" — the backbone of Dijkstra, Prim, Huffman coding, Top-K problems, and job scheduling.

2. Priority queue — the STL heap

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    priority_queue<int> pq;           // MAX-heap by default
    pq.push(3); pq.push(10); pq.push(1);
    cout << pq.top();                // 10
    pq.pop();                         // removes 10
    cout << pq.top();                // 3

    // MIN-heap
    priority_queue<int, vector<int>, greater<int>> minpq;

    // heap of pairs (max by first, then second)
    priority_queue<pair<int,int>> pairpq;

    // min-heap on second of pair (for Dijkstra-style)
    using P = pair<long long,int>;
    priority_queue<P, vector<P>, greater<P>> dhpq;

    return 0;
}
```

Time: push/pop $O(\log n)$, top $O(1)$, construction from n elements $O(n)$

Custom comparator for a max-heap of "smallest comes first" by a custom rule:

```
struct Cmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        return a.second > b.second;    // min-heap by second value
    }
};
priority_queue<pair<int,int>, vector<pair<int,int>>, Cmp> pq;
```

3. Building a heap by hand (if asked)

The array index math (children at $2i+1$, $2i+2$) is all you need:

```
void heapifyUp(vector<int>& a, int i) {
    while (i > 0 && a[i] > a[(i - 1) / 2]) {
        swap(a[i], a[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}
```

```

    }
}

void heapifyDown(vector<int>& a, int n, int i) {
    while (true) {
        int l = 2 * i + 1, r = 2 * i + 2, m = i;
        if (l < n && a[l] > a[m]) m = l;
        if (r < n && a[r] > a[m]) m = r;
        if (m == i) break;
        swap(a[i], a[m]);
        i = m;
    }
}

```

4. Core heap algorithms

Top-K smallest / largest (LC 215 — k-th largest)

Trick: keep a min-heap of size k for the k *largest* elements.

```

int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minpq; // min-heap
    for (int x : nums) {
        minpq.push(x);
        if ((int)minpq.size() > k) minpq.pop(); // evict the smallest of the top-k
    }
    return minpq.top(); // smallest of the top-k = k-th largest
}

```

Time: $O(n \log k)$ | **Space:** $O(k)$

Merge k sorted lists (LC 23) — heap of current heads

```

// Using ListNode from Ch. 10
ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; }; // min-heap
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
    for (ListNode* h : lists) if (h) pq.push(h);
    ListNode dummy(0);
    ListNode* t = &dummy;
    while (!pq.empty()) {
        ListNode* cur = pq.top(); pq.pop();
        t->next = cur;
        t = t->next;
        if (cur->next) pq.push(cur->next);
    }
    return dummy.next;
}

```

Time: $O(n \log k)$ | **Space:** $O(k)$

5. Worked example — running median (find median of a stream)

Idea: a **max-heap for the lower half** and a **min-heap for the upper half**. Keep them balanced (sizes differ by at most 1). The median is `maxHeap.top()` (odd size) or the average of both tops (even).

```

class MedianFinder {
    priority_queue<int> lo; // max-heap (lower half)
    priority_queue<int, vector<int>, greater<int>> hi; // min-heap (upper half)
public:
    void addNum(int x) {
        lo.push(x);
        hi.push(lo.top()); lo.pop(); // rebalance: largest lower → upper
        if (lo.size() < hi.size()) { // keep lo ≥ hi in size
            lo.push(hi.top()); hi.pop();
        }
    }
    double findMedian() {

```

```

    if (lo.size() > hi.size()) return lo.top();
    return (lo.top() + hi.top()) / 2.0;
}
};

```

addNum: $O(\log n)$ | **findMedian:** $O(1)$ | **Space:** $O(n)$

6. When to use & common mistakes

When to use: - "k-th largest/smallest", "top k". - Repeatedly extracting min/max (Dijkstra, Prim, Huffman). - Streaming medians. - Scheduling problems (earliest deadline, smallest job first). - Sorting a nearly sorted array in $O(n \log k)$ (Ch. 06 P6).

Common mistakes: 1. Forgetting `greater<int>` for a min-heap. 2. Off-by-one in child/parent index math ($2i+1$, $2i+2$, $(i-1)/2$). 3. Pushing duplicate entries and getting inconsistent results (rare, but be aware). 4. Using `priority_queue` where a simple `set` or `sort` suffices. 5. When writing a custom comparator, getting the direction wrong — the comparator defines a **min-heap** if `a` should come before `b` when `a > b` ... carefully test.

7. Practice

P1 [Easy] — Return the k-th largest element in an unsorted array. (LC 215)

P2 [Easy] — Merge k sorted linked lists into one sorted list. (LC 23)

P3 [Medium] — K Closest Points to Origin: return the k closest. (LC 973 — *heap or nth_element*)

P4 [Medium] — Find Median from Data Stream. (LC 295 — *two heaps*)

P5 [Challenging] — Given a list of tasks `[A,B,A,C,...]` and a cooldown `n`, find the least number of units to schedule all tasks. (LC 621 — *greedy + heap/counting*)

P6 [Challenging] — Sort a nearly sorted array where each element is within `k` of its sorted position, in $O(n \log k)$. (*min-heap window*)

Solutions in `27-Practice-Solutions.md`.

Chapter 17 — Greedy Algorithms

1. What is a greedy algorithm and why it matters

A **greedy** algorithm makes the **locally optimal choice at each step**, hoping it leads to the globally optimal solution. It never reconsiders earlier decisions.

Why it matters: when a greedy choice is provably correct, you get an algorithm that is fast (often $O(n \log n)$) and simple — no DP table, no exponential search. It solves interval scheduling, coin change (some variants), Huffman coding, MST, activity selection, and many more.

The catch: greedy **isn't always correct**. The classic counterexample is coin change: coins `{1, 5, 11}`, amount 15. Greedy picks `11 + 1 + 1 + 1 + 1` (5 coins), but the optimal is `5 + 5 + 5` (3 coins). Always *prove* the greedy choice works before using it.

2. The recipe

1. Sort the input by some key (most greedy problems need sorted input).
2. Take the "best" item first (greedy choice).
3. Show that an optimal solution can always include this best item (exchange argument), and that the rest is a smaller subproblem (optimal substructure).

3. Classic problems

Activity selection (maximize number of non-overlapping activities)

Problem: given start/finish times, pick the max number that don't overlap. **Greedy choice:** pick the activity that **finishes earliest** (then recurse).

```
int activitySelection(vector<pair<int,int>> act) {
    sort(act.begin(), act.end(), [](auto& a, auto& b){ return a.second < b.second; });
    int count = 0, lastEnd = INT_MIN;
    for (auto& [s, e] : act) {
        if (s >= lastEnd) { count++; lastEnd = e; } // compatible
    }
    return count;
}
```

Time: $O(n \log n)$ | **Space:** $O(1)$

Why it works: the earliest-finishing activity can always be part of an optimal solution (swapping any other first activity for it keeps the schedule feasible and no worse). This is the **exchange argument**.

Coin change (only when denominations allow) — the "does greedy work?" question

For coin systems where each coin $\geq$ twice the previous, greedy is optimal. Otherwise use DP (Ch. 19).

```
int minCoinsGreedy(const vector<int>& coins, int amount) {
    int cnt = 0;
    for (int c : coins) { // assume coins sorted descending
        while (amount >= c) { amount -= c; cnt++; }
    }
    return (amount == 0) ? cnt : -1;
}
```

Jump Game (LC 55) — the greedy "furthest reachable" pattern

Problem: can you reach the last index given max jumps at each index? **Greedy:** maintain the furthest index reachable so far; if any $i > \text{reach}$, fail.

```
bool canJump(const vector<int>& nums) {
    int reach = 0;
    for (int i = 0; i < (int)nums.size(); i++) {
        if (i > reach) return false; // stuck
        reach = max(reach, i + nums[i]);
    }
    return true;
}
```

Time: $O(n)$ | **Space:** $O(1)$

Meeting rooms / min platforms (LC 253 / classic railway platforms)

Problem: min number of rooms to hold all meetings. **Greedy idea:** sweep the timeline. Sort all start and end times together; each start needs a room (+1), each end frees one (−1). Track the peak.

```
int minMeetingRooms(vector<pair<int,int>>& times) {
    vector<int> starts, ends;
    for (auto& [s, e] : times) { starts.push_back(s); ends.push_back(e); }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());
    int rooms = 0, peak = 0, j = 0;
    for (int i = 0; i < (int)starts.size(); i++) {
        if (starts[i] < ends[j]) rooms++; // need a new room
        else j++; // reuse a freed room
        peak = max(peak, rooms);
    }
}
```

```
}  
    return peak;  
}
```

Time: $O(n \log n)$ | **Space:** $O(n)$

Huffman coding (optimal prefix-free codes)

Repeatedly merge the two smallest frequencies into one node (a heap does this in $O(n \log n)$). The most frequent characters end up with the shortest codes.

4. When greedy works vs. when you need DP

Signal	Greedy likely	DP needed
Maximize count of items	✓	
Locally optimal choice provably optimal (exchange argument)	✓	
Decision affects future costs		✓ (overlapping subproblems)
Need <i>all</i> solutions / any of them	✓ (count/any)	
Count of ways / min cost over all orders		✓

Golden rule: greedy is a *guess* until you **prove** it. When in doubt, write DP.

5. When to use & common mistakes

When to use: scheduling/interval problems, "minimum number of X to cover Y", partitioning, spanning, and any problem with a clear "take the best first" step that you can justify.

Common mistakes: 1. **Using greedy when it's not optimal** — the #1 mistake. Coin change, fractional vs 0/1 knapsack (0/1 knapsack needs DP!), etc. 2. **Forgetting to sort** before the greedy pass. 3. **Wrong sort key** (sort by start instead of finish in activity selection). 4. Not proving optimality — in interviews, walk through the exchange argument. 5. Edge cases: negative values, empty input, overflow in sums.

6. Practice

P1 [Easy] — Can you reach the last index? (LC 55)

P2 [Easy] — Assign cookies: maximize the number of satisfied children (each child needs a cookie $\geq$ its greed factor). (LC 455)

P3 [Medium] — Given coins of a system where greedy works, return the min coins for amount. (*variation*)

P4 [Medium] — Min number of platforms/rooms. (LC 253)

P5 [Challenging] — Interval Partitioning / Minimum Number of Arrows to Burst Balloons. (LC 452)

P6 [Challenging] — Jump Game II: minimum number of jumps to reach the end. (LC 45 — *greedy "furthest reach in current jump" or BFS*)

P7 [Challenging] — Gas Station: can you complete a circular route? (LC 134)

Solutions in `27-Practice-Solutions.md`.

Chapter 18 — Graphs

1. What is a graph and why it matters

A **graph** is a set of **nodes (vertices)** connected by **edges**. It's the most general way to model relationships: social networks, roads/maps, the web, dependency orders, game states, circuits, molecules.

```

1 -- 2
|    |
4 -- 3 -- 5

```

Vocabulary: - **Undirected edge** $u-v$: can travel both ways. - **Directed edge** $u \rightarrow v$: only one way. - **Weighted edge**: has a cost/length. - **Path**: sequence of edges from node to node. **Cycle**: path that returns to start. - **Connected**: every pair of nodes reachable. **Tree**: connected & acyclic. - **DAG**: Directed Acyclic Graph — the bread and butter of topological sorting & DP.

2. Representing a graph in C++

Adjacency list (the standard — always prefer this)

```

int n = 5;
vector<vector<int>> g(n);    // g[u] = list of neighbors of u

g[0].push_back(1);
g[1].push_back(2);
// ... for directed edge u→v: g[u].push_back(v) only
// ... for weighted: vector<vector<pair<int,int>>> g; g[u].push_back({v, w});

```

Space: $O(V + E)$. Iterating neighbors of a node is $O(\text{degree})$. Fast.

Adjacency matrix

```

vector<vector<int>> mat(n, vector<int>(n, 0)); // mat[u][v] = edge?
mat[1][2] = 1;

```

Space: $O(V^2)$. $O(1)$ edge query. Only use for dense small graphs ($V \leq \sim 1000$).

Rule of thumb: use the adjacency list. It's what everyone uses and what the problems expect.

3. Depth-First Search (DFS)

Recursive; explores as deep as possible, then backtracks. Used for: reachability, counting components, cycle detection, topological sort, SCC.

```

vector<vector<int>> g;
vector<bool> vis;

void dfs(int u) {
    vis[u] = true;
    for (int v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}

```

Time: $O(V + E)$ | **Space:** $O(V)$ (vis + recursion stack)

Count connected components — call dfs for every unvisited node:

```

int components = 0;
for (int i = 0; i < n; i++) {
    if (!vis[i]) { components++; dfs(i); }
}

```

4. Breadth-First Search (BFS)

Explores level by level with a queue. Used for: **shortest path in unweighted graphs**, level-order traversal, "minimum number of moves".

Start at 1. Levels: 1 | 2 4 | 3 | 5

Queue: [1] → pop 1, push neighbors 2,4 → [2,4] → pop 2, push 3 → [4,3] → pop 4 → [3] → pop 3, push 5 → [5]

```
vector<int> bfs(int start, const vector<vector<int>>& g) {
    int n = g.size();
    vector<int> dist(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] == -1) {           // first time we see v = shortest
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
```

Time: $O(V + E)$ | **Space:** $O(V)$

Why BFS gives shortest paths in unweighted graphs: the queue processes nodes in non-decreasing distance order. The first time a node is discovered, no shorter path exists.

5. BFS/DFS on a grid (the classic "islands" problem, LC 200)

Grids are graphs where each cell is a node and neighbors are up/down/left/right.

```
int numIslands(vector<vector<char>>& grid) {
    int n = grid.size(), m = grid[0].size();
    int di[4] = {-1, 1, 0, 0}, dj[4] = {0, 0, -1, 1};
    int ans = 0;

    function<void(int,int)> dfs = [&](int i, int j) {
        grid[i][j] = '0';           // mark visited in place (sink the island)
        for (int k = 0; k < 4; k++) {
            int ni = i + di[k], nj = j + dj[k];
            if (ni < 0 || ni >= n || nj < 0 || nj >= m) continue;
            if (grid[ni][nj] == '1') dfs(ni, nj);
        }
    };

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (grid[i][j] == '1') { ans++; dfs(i, j); }
    return ans;
}
```

Time: $O(n \cdot m)$ | **Space:** $O(n \cdot m)$ worst (stack)

6. Topological sort (ordering tasks with dependencies)

Problem: given a DAG (dependencies like A must happen before B), produce a linear order where every edge goes forward.

Method (Kahn's algorithm): repeatedly remove a node with **in-degree 0** and decrement its neighbors' in-degrees.

```
Courses: A(deps: none), B(deps: A), C(deps: A), D(deps: B, C)
in-degrees: A=0, B=1, C=1, D=2
Queue of in-degree-0: [A]
Pop A → B:1→0, C:1→0 → queue [B, C]
Pop B → D:2→1 ... Pop C → D:1→0 → queue [D]
Pop D. Order: A, B, C, D
```

```

vector<int> topoSort(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : g[u]) indeg[v]++;

    queue<int> q;
    for (int u = 0; u < n; u++)
        if (indeg[u] == 0) q.push(u);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : g[u])
            if (--indeg[v] == 0) q.push(v);
    }
    // if order.size() != n → there is a cycle (LC 207 uses exactly this)
    return order;
}

```

Time: $O(V + E)$ | **Space:** $O(V)$

Alternative: DFS-based topological sort — append node after visiting all descendants, then reverse.

7. Shortest paths

Unweighted → BFS (done above), $O(V+E)$

Weighted, non-negative → Dijkstra (Ch. 25)

Weighted, may have negative edges → Bellman-Ford (Ch. 25)

All pairs → Floyd-Warshall (Ch. 25)

8. Minimum Spanning Tree (MST)

An MST connects all nodes with the **minimum total edge weight** and no cycles.

Kruskal's algorithm (Ch. 22 DSU + sorting)

Sort edges by weight; add an edge if it connects two different components (cycle check via DSU).

```

int mstCost(int n, vector<array<int,3>>& edges) {    // {w, u, v}
    sort(edges.begin(), edges.end());
    // ... DSU (union-find) from Ch. 22
    DSU dsu(n);
    int total = 0, used = 0;
    for (auto& [w, u, v] : edges) {
        if (dsu.unite(u, v)) { total += w; used++; }
    }
    return used == n - 1 ? total : -1;    // -1 if not connected
}

```

Time: $O(E \log E)$ (sorting) | **Space:** $O(V)$

Prim's algorithm — greedy with a min-heap

Grow one tree: always add the cheapest edge from the tree to a node outside it. (Implementation in Ch. 25.)

9. When to use & common mistakes

When to use: anything with entities + relationships ("reachable", "shortest path", "connected", "dependencies", "islands/regions in a grid"). **Hint:** if the input is a grid, think BFS/DFS first. If it's a list of `[u, v]` edges, build the adjacency list.

Common mistakes: 1. **Missing visited array** → infinite recursion/loops on cycles. 2. Marking visited **after** pushing (BFS) instead of when pushing → duplicate work. 3. Using DFS for shortest path (DFS doesn't guarantee shortest in general graphs — use BFS for unweighted, Dijkstra for weighted). 4. Off-by-one / bounds check in grids. 5. Undirected graph → forgetting to add both `g[u].push_back(v)` and `g[v].push_back(u)`. 6. Forgetting that topological sort works **only on DAGs**. 7. `int` overflow in path costs — use `long long` with `INF = 1e18`.

10. Practice

P1 [Easy] — Find if there's a path between two nodes in an undirected graph. (LC 1971)

P2 [Medium] — Number of Islands. (LC 200)

P3 [Medium] — Clone a graph. (LC 133 — DFS + hashmap)

P4 [Medium] — Rotting Oranges: minutes until all oranges rot (multi-source BFS). (LC 994)

P5 [Medium] — Course Schedule: can you finish? (cycle detection in a DAG) (LC 207)

P6 [Medium] — Course Schedule II: return the order. (LC 210 — topological sort)

P7 [Challenging] — Number of provinces (connected components). (LC 547)

P8 [Challenging] — Minimum spanning tree cost. (Kruskal on sample data)

Solutions in `27-Practice-Solutions.md`.

Chapter 19 — Dynamic Programming

1. What is DP and why it matters

Dynamic Programming (DP) is the art of solving a problem by breaking it into **overlapping subproblems**, solving each **once**, and **reusing** the results.

Two ingredients are required: 1. **Optimal substructure** — the optimal solution contains optimal solutions to subproblems. 2. **Overlapping subproblems** — the same subproblem appears many times.

Recall the naive Fibonacci (`fib(2)` computed over and over). DP fixes exactly that: `fib(5) = fib(4) + fib(3)` — memoize, and the tree collapses to a chain.

Naive <code>fib(5)</code>: <pre> f(5) / \ f(4) f(3) / \ / \ f(3) f(2) f(2) f(1) ... </pre>	Memoized <code>fib(5)</code>: <pre> f(5) / \ f(4) f(3) ← computed once, reused / f(3) / f(2) ... only n+1 states </pre>
--	--

Why it matters: DP turns exponential ($O(2^n)$) into polynomial ($O(n \cdot k)$) — the single biggest win in all of DSA. It's essential for interviews and is the hardest topic to grasp; the examples below build intuition step by step.

2. The two DP styles

Style	How	When
Memoization (top-down)	Recursive + a cache (<code>dp</code> array / hashmap); compute on demand	natural recursive formulation
Tabulation (bottom-up)	Fill a <code>dp</code> table in a fixed order; no recursion	state order is obvious

Memoization is easier to write (just add a cache to recursion). **Tabulation avoids stack overflow** and often allows memory optimization. Learn both.

3. Step-by-step: Fibonacci (the hello-world of DP)

Step 1 — define the state

`dp[i]` = the i -th Fibonacci number.

Step 2 — recurrence

`dp[i] = dp[i-1] + dp[i-2]`.

Step 3 — base cases

`dp[0] = 0`, `dp[1] = 1`.

Memoized:

```
int fib(int n) {
    static vector<int> memo(31, -1);
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
```

Time: $O(n)$ | **Space:** $O(n)$

Tabulated + space optimized:

```
int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

Time: $O(n)$ | **Space:** $O(1)$ — because only the last two matter

4. The DP recipe (use for every problem)

1. **State:** what does `dp[i]` (or `dp[i][j]`) mean? (Define precisely!)
2. **Recurrence:** how to compute `dp[i]` from smaller states? (The transition.)
3. **Base cases:** smallest states, set directly.
4. **Order:** compute in an order where dependencies are already computed.
5. **Answer:** which cell(s) hold the answer?

5. Classic examples

Climbing stairs (LC 70)

Problem: climb n stairs, taking 1 or 2 steps. How many distinct ways?

State: `dp[i]` = ways to reach step i . **Recurrence:** `dp[i] = dp[i-1] + dp[i-2]` (last step was a 1-step or a 2-step). Same as Fibonacci.

```
int climbStairs(int n) {
    if (n <= 2) return n;
    vector<int> dp(n + 1);
    dp[1] = 1; dp[2] = 2;
    for (int i = 3; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Min cost climbing stairs (LC 746)

Problem: each step costs `cost[i]`; you can step 1 or 2. Min total cost to reach the top.

```
int minCostClimbingStairs(const vector<int>& cost) {
    int n = cost.size();
    vector<int> dp(n + 1, 0);
    for (int i = 2; i <= n; i++)
        dp[i] = min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);
    return dp[n];
}
```

Time: $O(n)$ | **Space:** $O(n) \rightarrow$ can be $O(1)$

Longest increasing subsequence (LC 300)

Problem: longest subsequence (not necessarily contiguous) that is strictly increasing.

State: `dp[i]` = length of LIS ending at index `i`. **Recurrence:** `dp[i] = 1 + max(dp[j])` for all `j < i` with `a[j] < a[i]` (else 1).

```
int lengthOfLIS(const vector<int>& a) {
    int n = a.size(), ans = 1;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

Time: $O(n^2)$ | **Space:** $O(n)$

$O(n \log n)$ upgrade (binary search + patience sorting): keep `tails`, the smallest possible ending value for an LIS of each length. For each `x`, find the first `tails` $\geq x$ and replace it with `x` (`lower_bound`). See Ch. 26.

0/1 Knapsack

Problem: `n` items with weights & values, capacity `W`. Choose items (each once) to maximize value.

State: `dp[i][w]` = max value using the first `i` items with capacity `w`. **Recurrence:** `dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight[i]] + value[i])` — either skip item `i`, or take it (if it fits).

```
int knapsack(const vector<int>& wt, const vector<int>& val, int W) {
    int n = wt.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int w = W; w >= wt[i]; w--) // descend! so each item used once
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Time: $O(n \cdot W)$ | **Space:** $O(W)$ (rolled up from $O(n \cdot W)$)

Why descend in the 1D version: if we went left→right, `dp[w - wt[i]]` might already include item `i` (used twice). Descending means we always read the previous row's values.

Longest common subsequence (LC 1143)

Problem: longest subsequence common to two strings.

State: `dp[i][j]` = LCS of `a[0..i-1]` and `b[0..j-1]`. **Recurrence:** - if `a[i-1] == b[j-1]`: `dp[i][j] = dp[i-1][j-1] + 1` - else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

```
int longestCommonSubsequence(const string& a, const string& b) {
    int n = a.size(), m = b.size();
    // ...
}
```

```
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
        else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
return dp[n][m];
}
```

Time: $O(n \cdot m)$ | **Space:** $O(n \cdot m) \rightarrow O(\min(n, m))$

Longest palindromic substring (LC 5)

Problem: longest contiguous palindrome in a string.

State: $dp[i][j]$ = is $s[i..j]$ a palindrome? **Recurrence:** $dp[i][j] = (s[i] == s[j]) \ \&\& \ (j - i \leq 2 \ || \ dp[i+1][j-1])$.

```
string longestPalindrome(const string& s) {
    int n = s.size();
    if (n < 2) return s;
    vector<vector<bool>> dp(n, vector<bool>(n, false));
    int bestL = 0, bestR = 0;
    for (int len = 1; len <= n; len++) { // fill by length!
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (s[i] == s[j] && (len <= 2 || dp[i + 1][j - 1])) {
                dp[i][j] = true;
                if (len > bestR - bestL + 1) { bestL = i; bestR = j; }
            }
        }
    }
    return s.substr(bestL, bestR - bestL + 1);
}
```

Time: $O(n^2)$ | **Space:** $O(n^2)$

Why fill by length: $dp[i+1][j-1]$ is a *shorter* substring — it must be computed before $dp[i][j]$. Filling outer loops over length guarantees that.

6. Common DP types cheat sheet

Type	State	Classic problem
1D array	$dp[i]$ over prefix	house robber, climb stairs, LIS
2D grid	$dp[i][j]$ over cells	min path sum, unique paths
String DP	$dp[i][j]$ over indices	LCS, edit distance, palindrome
Knapsack	$dp[i][w]$ weight dimension	0/1 knapsack, subset sum
Interval DP	$dp[l][r]$ over subarrays	matrix chain, burst balloons
DP on tree	$dp[node][state]$	max independent set on tree
Bitmask DP	$dp[mask]$ over subsets	TSP, assignments

7. When to use & common mistakes

When to use DP: "count the number of ways", "minimum/maximum cost", "is it possible", and the input is moderate ($n \leq \sim 5000$ for $O(n^2)$, $W \leq \sim 1e5$ for knapsack). If n is tiny (≤ 20), backtracking/bitmask might be simpler.

Common mistakes: 1. **No base case** $\rightarrow$ index out of bounds. 2. **Wrong state definition** — vague states cause wrong transitions. Define it exactly. 3. **Wrong order** — using $dp[i+1]$ before computing it. 4. **Off-by-one** with 1-indexed dp tables (add a dummy row/col). 5. Forgetting 0/1 vs unbounded knapsack (descend vs ascend

in 1D). 6. Not optimizing space when only the last row is needed. 7. Starting `dp` with `-1` sentinel in memoization and confusing "uncomputed" with a valid `0`.

8. Practice

P1 [Easy] — Climbing stairs. (LC 70)

P2 [Easy] — House robber: max money without robbing two adjacent houses. (LC 198)

P3 [Medium] — Longest increasing subsequence. (LC 300)

P4 [Medium] — Coin change: fewest coins to make amount. (LC 322 — *unbounded knapsack*)

P5 [Medium] — Longest common subsequence. (LC 1143)

P6 [Challenging] — Unique paths in a grid. (LC 62)

P7 [Challenging] — Longest palindromic substring. (LC 5)

P8 [Challenging] — Edit distance: min ops to convert word1 to word2. (LC 72)

P9 [Challenging] — 0/1 knapsack: max value within capacity. (*classic*)

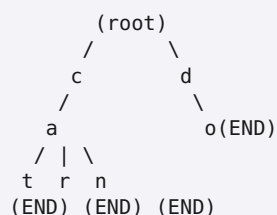
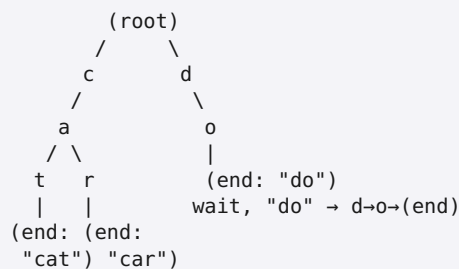
Solutions in `27-Practice-Solutions.md`.

Chapter 20 — Tries

1. What is a trie and why it matters

A **trie** (prefix tree) is a tree of characters. Each **node** is one character, and a **path from root to a node spells a prefix**. A flag marks where a word ends.

Insert: "cat", "car", "can", "do"



Why it matters: finding whether a word is in a set is $O(L)$ where L = word length, **independent of how many words exist**. It's the standard tool for **prefix queries** ("all words starting with 'ca'"), **autocomplete**, spell check, and the classic interview problem "Implement a Trie" and "Word Search II".

2. C++ implementation

```
struct TrieNode {
    TrieNode* child[26];
    bool isEnd;
    TrieNode() : isEnd(false) {
        for (int i = 0; i < 26; i++) child[i] = nullptr;
    }
};
```

```

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }

    void insert(const string& w) {
        TrieNode* cur = root;
        for (char c : w) {
            int i = c - 'a';
            if (!cur->child[i]) cur->child[i] = new TrieNode();
            cur = cur->child[i];
        }
        cur->isEnd = true;
    }

    bool search(const string& w) {           // exact word?
        TrieNode* cur = root;
        for (char c : w) {
            int i = c - 'a';
            if (!cur->child[i]) return false;
            cur = cur->child[i];
        }
        return cur->isEnd;
    }

    bool startsWith(const string& p) {       // any word with this prefix?
        TrieNode* cur = root;
        for (char c : p) {
            int i = c - 'a';
            if (!cur->child[i]) return false;
            cur = cur->child[i];
        }
        return true;
    }
};

```

Time: insert/search/startsWith $O(L)$ where L = length of the word | **Space:** $O(\text{total number of characters stored})$

3. Worked example — word search II (LC 212, the crown jewel)

Problem: given a grid of letters and a list of words, find all words that can be formed by tracing adjacent cells.

Naive: for each word, DFS the grid $\rightarrow O(\text{words} \times \text{grid} \times \text{path})$. **Better:** build a trie of *all* words once, then do **one** DFS over the grid, walking the trie as you go. This shares the expensive exploration across words that share prefixes.

```

// skeleton
struct TrieNode { /* same as above, plus children */ };

vector<string> findWords(vector<vector<char>>& board, vector<string>& words) {
    TrieNode* root = new TrieNode();
    for (auto& w : words) { /* insert w into trie */ }

    vector<string> res;
    int n = board.size(), m = board[0].size();
    int di[4] = {-1, 1, 0, 0}, dj[4] = {0, 0, -1, 1};

    function<void(int,int,TrieNode*,string)> dfs = [&](int i, int j, TrieNode* node, string cur) {
        if (node->isEnd) { res.push_back(cur); node->isEnd = false; } // dedupe
        for (int k = 0; k < 4; k++) {
            int ni = i + di[k], nj = j + dj[k];
            if (ni < 0 || ni >= n || nj < 0 || nj >= m) continue;
            char c = board[ni][nj];
            if (c == '#' || !node->child[c - 'a']) continue; // visited or no path
            board[ni][nj] = '#';
            dfs(ni, nj, node->child[c - 'a'], cur + c);
            board[ni][nj] = c;
        }
    };

    for (int i = 0; i < n; i++)

```

```

    for (int j = 0; j < m; j++)
        if (root->child[board[i][j] - 'a']) {
            char c = board[i][j];
            board[i][j] = '#';
            dfs(i, j, root->child[c - 'a'], string(1, c));
            board[i][j] = c;
        }
    return res;
}

```

Time: $O(4^L)$ worst per start cell, but the trie prunes dead branches aggressively.

4. Trie variants & extensions

- **Counting words / prefixes:** each node stores `count`.
- **Sorting strings:** DFS the trie and you get strings in sorted order (lexicographic).
- **Autocomplete:** DFS from the prefix node collecting words (BFS for shortest, or collect all).

5. When to use & common mistakes

When to use: - Prefix queries / autocomplete / dictionary search. - Many pattern searches over the same text (Aho-Corasick extends the idea). - Problems that say "check if a string matches a set of patterns".

Common mistakes: 1. **Forgetting `isEnd`** — `startsWith` and `search` are different operations. 2. Not initializing child pointers to `nullptr`. 3. Memory leaks / huge memory for large alphabets — use a map child or a vector sized to the alphabet; 26 is fine for lowercase. 4. In Word Search II, forgetting to unmark visited cells (backtracking!). 5. Adding duplicate results — clear `isEnd` once found.

6. Practice

P1 [Easy] — Implement a Trie (insert, search, startsWith). (LC 208)

P2 [Medium] — Design a data structure that supports adding words and searching for a word where `.` can match any letter. (LC 211 — trie + DFS for the wildcard)

P3 [Challenging] — Word Search II. (LC 212)

P4 [Challenging] — Longest word in a dictionary that can be built one character at a time by other words in the dictionary. (LC 720 — insert all, then DFS)

Solutions in `27-Practice-Solutions.md`.

Chapter 21 — Bit Manipulation

1. What is it and why it matters

Bit manipulation works directly on the individual bits of integers. Each integer is a tiny array of bits (0/1), and we can read/modify them with operators.

Why it matters: - **Speed & memory:** represent a set of up to ~60 elements in a single 64-bit integer (a **bitset**). Check membership, add, remove, union, intersect in $O(1)$. - **Bitmask DP:** states like "which cities have I visited" = one integer. Enables TSP-style problems (Ch. 19 bitmask DP). - **Cute one-liners:** check power of two, count bits, find odd occurrence — common interview tricks.

2. The operators

Operator	Meaning	Example
<code>&</code>	AND (both 1)	<code>5 & 3 = 1</code>
<code> </code>	OR (either 1)	<code>5 3 = 7</code>

Operator	Meaning	Example
<code>^</code>	XOR (different)	<code>5 ^ 3 = 6</code>
<code>~</code>	NOT (flip all)	<code>~5 = -6</code> (two's complement!)
<code><<</code>	left shift ($\times 2$ per shift)	<code>1 << 3 = 8</code>
<code>>></code>	right shift ($\div 2$ per shift)	<code>8 >> 2 = 2</code>

Precedence gotcha: `&`, `|`, `^` bind *looser* than `==` and `!=`. Always parenthesize: `if ((x & mask) == 0)` — not `if (x & mask == 0)`.

3. The ten most useful idioms

```
int n = 0b1010;           // binary literal = 10

// 1. check if bit i is set
bool isset = (n >> i) & 1;

// 2. set bit i
n |= (1 << i);

// 3. clear bit i
n &= ~(1 << i);

// 4. toggle bit i
n ^= (1 << i);

// 5. count set bits
int cnt = __builtin_popcount(n);           // builtin!
long long x = 0xF0F0;
int cnt2 = __builtin_popcountll(x);        // for long long

// 6. lowest set bit
int low = n & (-n);                       // isolate lowest 1-bit

// 7. clear the lowest set bit
n = n & (n - 1);                          // classic

// 8. check power of two
bool isPow2 = (n > 0) && (n & (n - 1)) == 0;

// 9. find the single non-repeated number in an array (XOR trick)
//    a ^ a = 0, a ^ 0 = a → XOR all: duplicates cancel
int xorAll = 0; for (int v : arr) xorAll ^= v; // result = the odd one

// 10. index of the lowest set bit
int idx = __builtin_ctz(n);               // count trailing zeros
```

4. Worked examples

Example A — find the one element appearing once (LC 136)

```
int singleNumber(const vector<int>& a) {
    int res = 0;
    for (int x : a) res ^= x;
    return res;
}
```

Time: $O(n)$ | **Space:** $O(1)$

Why: XOR is commutative and $x \oplus x = 0$. Pairs vanish; only the single survives.

Example B — count how many bits differ between two numbers (Hamming distance, LC 461)

```
int hammingDistance(int a, int b) {
    int diff = a ^ b;           // 1s where bits differ
    return __builtin_popcount(diff);
}
```

Example C — subsets via bitmask (enumerate all subsets of n elements)

```
int n = 3; // elements a[0..2]
for (int mask = 0; mask < (1 << n); mask++) { // 0..7, all 2^n subsets
    for (int i = 0; i < n; i++)
        if (mask & (1 << i)) cout << i << " "; // element i in subset
    cout << "\n";
}
```

Time: $O(2^n \cdot n)$

Example D — the "only two singles" problem (LC 260)

Problem: every number appears twice except two. Find them.

Idea: XOR all $\rightarrow$ xorAll = $a \oplus b$ (the two singles). Any set bit in xorAll distinguishes them. Partition by that bit; XOR each partition separately.

```
vector<int> singleNumberIII(const vector<int>& a) {
    long long x = 0;
    for (int v : a) x ^= v;           // = single1 ^ single2
    int bit = x & (-x);               // lowest set bit (they differ here)
    int s1 = 0, s2 = 0;
    for (int v : a) {
        if (v & bit) s1 ^= v;
        else s2 ^= v;
    }
    return {s1, s2};
}
```

5. Bitmask DP example — assignment / subset optimization

Problem: assign n tasks to n people, each person-task has cost[i][j]; each person gets exactly one task. Min total cost.

State: $dp[mask]$ = min cost to assign the tasks in mask (bit i set = task i done). We assign person $popcount(mask)$ next.

```
int assignment(const vector<vector<int>>& cost, int n) {
    int dp[1 << n];
    fill(dp, dp + (1 << n), INT_MAX);
    dp[0] = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int person = __builtin_popcount(mask); // which person we're assigning
        if (person == n) continue;
        for (int task = 0; task < n; task++) {
            if (mask & (1 << task)) continue; // task already assigned
            int nmask = mask | (1 << task);
            dp[nmask] = min(dp[nmask], dp[mask] + cost[person][task]);
        }
    }
    return dp[(1 << n) - 1];
}
```

Time: $O(2^n \cdot n)$ — vs $O(n!)$ brute force. Enormous win for $n \leq 20$.

6. When to use & common mistakes

When to use: - Sets/states that fit in ≤ 64 bits (bitset, bitmask DP). - XOR tricks ("find the odd one out"). - Power-of-two / parity / range-of-bits checks. - Performance-critical code (bit ops are single instructions).

Common mistakes: 1. **Shift overflow:** `1 << 31` overflows `int`. Use `1LL << i` or `1u << i`. 2. **Operator precedence:** `x & mask == 0` parses as `x & (mask == 0)`. Parenthesize! 3. **Sign issues:** `~n` on signed ints flips the sign bit. Use unsigned or mask. 4. Forgetting that right shift of a negative signed int is implementation-defined. 5. `__builtin_*` on the wrong width (`popcount` vs `popcountll`).

7. Practice

P1 [Easy] — Single Number (the odd one out). (LC 136)

P2 [Easy] — Power of Two. (LC 231)

P3 [Easy] — Hamming distance between two integers. (LC 461)

P4 [Medium] — Count the number of set bits from 0 to n (each number's bits). (LC 338)

P5 [Challenging] — Single Number III (two singles). (LC 260)

P6 [Challenging] — Given an array where every element appears 3 times except one, find it using bitwise counting. (LC 137)

P7 [Challenging] — Sum of two integers without using `+` or `-`. (LC 371)

Solutions in `27-Practice-Solutions.md`.

Chapter 22 — Disjoint Set Union (DSU / Union-Find)

1. What is it and why it matters

Disjoint Set Union (DSU), a.k.a. **Union-Find**, manages a collection of **disjoint (non-overlapping) sets** with two operations: - `find(x)` — which set does x belong to? (returns a representative / root) - `unite(x, y)` — merge the sets containing x and y.

Why it matters: with two optimizations it runs in **nearly $O(1)$** per operation (inverse-Ackermann, $\sim O(1)$ in practice). It's the tool for "are these two connected?", "how many components?", and the backbone of **Kruskal's MST**.

Real uses: social networks ("are Alice and Bob connected?"), image segmentation, Kruskal's algorithm, "accounts merge", dynamic connectivity.

2. The idea

Each set is a tree; each node points to its **parent**, and the **root** is the set representative.

After `unite(1, 2)`:

```
1
|
2
```

After `unite(3, 4)`:

```
1
|
2
| \
3  4
```

After `unite(2, 3)`:

```
1
| \
2  4? (merge via root)
|
```

Actually the forest evolves:

sets `{1,2}`, `{3,4}`, `{5}`

```
1      3      5
|      |
2      4
```

`find(4)`: `4 → 3 → 3 (root)`. `unite(2, 4)`: attach root of one set to the other.

3. Implementation with both optimizations

Path compression

During `find`, point every node on the path directly to the root — flattens the tree.

Union by size (or rank)

Attach the **smaller** tree under the **larger** one — keeps trees shallow.

```
struct DSU {
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n);
        sz.assign(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]); // path compression
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false; // already in the same set
        if (sz[a] < sz[b]) swap(a, b); // union by size
        parent[b] = a;
        sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) { return find(a) == find(b); }
    int sizeOf(int x) { return sz[find(x)]; }
};
```

Time: find/unite $\sim O(\alpha(n)) \approx O(1)$ with both optimizations | **Space:** $O(n)$

4. Worked example — number of connected components (LC 323 variant)

```
int countComponents(int n, vector<vector<int>>& edges) {
    DSU dsu(n);
    int comps = n;
    for (auto& e : edges)
        if (dsu.unite(e[0], e[1])) comps--; // each successful merge reduces count
    return comps;
}
```

5. Worked example — Kruskal's MST (uses Ch. 18 idea + DSU)

```
int mstCost(int n, vector<array<int,3>>& edges) { // {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    int total = 0, used = 0;
    for (auto& [w, u, v] : edges) {
        if (dsu.unite(u, v)) { // edge connects two different sets?
            total += w;
            if (++used == n - 1) break; // a spanning tree needs exactly n-1 edges
        }
    }
    return used == n - 1 ? total : -1;
}
```

Time: $O(E \log E)$ for sorting + $O(E \alpha(V))$ for DSU | **Space:** $O(V)$

Why it's correct (Kruskal): process edges by weight ascending; an edge is added only if it connects two currently-disconnected components — this can never create a cycle, and greedily choosing the cheapest safe edge is provably optimal.

6. When to use & common mistakes

When to use: - "Count components", "are they connected", "merge groups". - Kruskal's MST, cycle detection in undirected graphs (`unite` returns false $\Rightarrow$ cycle). - Problems phrased as "union by condition" (e.g., accounts merge, redundant connection). - Offline query problems (process queries in reverse).

Common mistakes: 1. **Forgetting path compression or union by size** $\rightarrow O(n)$ find, TLE on large inputs. 2. Calling `find` inconsistently (reading `parent[x]` directly instead of `find(x)`). 3. Off-by-one if nodes are 1-indexed — size your DSU `n+1`. 4. `unite` must return whether a merge happened — many problems rely on that bool. 5. Modifying `sz` wrong on failed merges (only update on success).

7. Practice

P1 [Easy] — Find the number of connected components in an undirected graph. (LC 547 / 323)

P2 [Medium] — Accounts Merge: merge emails sharing an account. (LC 721)

P3 [Medium] — Redundant Connection: find the edge that creates a cycle. (LC 684)

P4 [Challenging] — Longest Consecutive Sequence: find the length of the longest run of consecutive numbers. (LC 128 — can use DSU or hashmap approach)

P5 [Challenging] — Number of Islands II: add islands one by one, return the count after each addition. (LC 305)

Solutions in `27-Practice-Solutions.md`.

Chapter 23 — Fenwick Trees (Binary Indexed Tree)

1. What is it and why it matters

A **Fenwick tree / Binary Indexed Tree (BIT)** supports two operations on an array: - **Point update:** add a value to `a[i]` — $O(\log n)$. - **Prefix query:** sum of `a[0..i]` — $O(\log n)$.

The naive approach is $O(1)$ for one operation but $O(n)$ for the other. A prefix-sum array makes queries $O(1)$ but updates $O(n)$. The BIT balances both at $O(\log n)$, and uses only $O(n)$ memory.

Why it matters: whenever you have an array that changes *and* you need range/prefix queries (running scores, frequency counts, inversions, dynamic prefix sums), the BIT is the classic answer. It's smaller and faster than a segment tree for prefix-style queries.

2. The core insight: binary representation

The BIT stores partial sums so that each index `i` (1-based) covers a range of length `lowbit(i) = i & (-i)`:

`BIT[i]` stores sum of `a[i - lowbit(i) + 1 .. i]`

Array `a` (1-based): `[3, 1, 4, 1, 5, 9, 2, 6]`
`lowbit(6) = 2  $\rightarrow$  BIT[6] = sum of a[5..6] = 5 + 9 = 14`

Update: to add `v` to `a[i]`, update `BIT[i]`, then `i += lowbit(i)`, repeat. **Query:** to get prefix sum up to `i`, add `BIT[i]`, then `i -= lowbit(i)`, repeat.

Update at 3 (+v): touches 3, 4, 8
Query prefix up to 6: reads 6, 4

Binary walks: 3 = 011 $\rightarrow$ 4 = 100 $\rightarrow$ 8 = 1000
6 = 110 $\rightarrow$ 4 = 100 $\rightarrow$ stop

3. Implementation (1-indexed; size n+1)

```
struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void add(int idx, int delta) {           // point update: a[idx] += delta
        for (; idx <= n; idx += idx & (-idx))
            bit[idx] += delta;
    }

    int sum(int idx) {                       // prefix sum a[1..idx]
        int res = 0;
        for (; idx > 0; idx -= idx & (-idx))
            res += bit[idx];
        return res;
    }

    int rangeSum(int l, int r) {             // sum a[l..r]
        return sum(r) - sum(l - 1);
    }
};
```

Time: add/sum $O(\log n)$ | **Space:** $O(n)$

Why `idx & (-idx)` works: `-idx` in two's complement flips all bits then adds 1; `idx & (-idx)` isolates the lowest set bit — the amount to jump.

4. Build in $O(n)$

```
for (int i = 1; i <= n; i++) {
    bit[i] += a[i];           // 1-indexed a
    int j = i + (i & (-i));
    if (j <= n) bit[j] += bit[i];
}
```

5. Classic application — counting inversions

Problem: count pairs (i, j) with $i < j$ and $a[i] > a[j]$.

Idea: sweep left→right. Use the BIT as a **frequency table** over values. When processing $a[i]$, the number of already-seen values **greater** than $a[i]$ is `total_seen - count( $\leq a[i]$ )`.

```
long long countInversions(const vector<int>& a) {
    // coordinate compression (values may be large)
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    Fenwick bit(vals.size());
    long long inv = 0, seen = 0;
    for (int x : a) {
        int rank = lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1;
        int leq = bit.sum(rank);           // how many  $\leq x$  seen so far
        inv += seen - leq;                 // those greater than x
        bit.add(rank, 1);
        seen++;
    }
    return inv;
}
```

Time: $O(n \log n)$ | **Space:** $O(n)$

6. When to use & common mistakes

When to use: - Dynamic prefix sums with point updates. - Frequency counting with updates (inversions, "count of values $\leq x$ so far"). - Problems that need "add then query prefix" many times.

Common mistakes: 1. **Forgetting 1-indexing** — BIT needs indices $1..n$; index 0 is a sentinel and breaks. 2. Off-by-one in `rangeSum(l, r)` (`sum(r) - sum(l-1)`). 3. Using `int` where sums overflow — use `long long`. 4. Forgetting that `sum(idx)` is inclusive of `idx`. 5. Using a BIT when you need *min/max over a range with updates* — for that, a segment tree (Ch. 24) or a "max-Fenwick" pattern is needed.

7. Practice

P1 [Easy] — Implement a Fenwick tree; run point-update & prefix-sum queries on sample data. (*implement first*)

P2 [Medium] — Count inversions in an array using a BIT. (*classic*)

P3 [Medium] — Given a stream of numbers, find the count of numbers less than the current one (Prefix sums with frequency BIT). (*adapted from LC 315 — count of smaller numbers after self*)

P4 [Challenging] — Dynamic ranking: maintain an array with point updates and "k-th smallest among the first i elements" queries. (*BIT + binary search*)

Solutions in `27-Practice-Solutions.md`.

Chapter 24 — Segment Trees

1. What is it and why it matters

A **segment tree** is a binary tree where each node stores an aggregate (sum, min, max, gcd, etc.) over a **range** of an array. It supports: - **Range query** (e.g., "sum of $a[l..r]$ ", "min of $a[l..r]$ ") — $O(\log n)$. - **Point update** (change one element) — $O(\log n)$.

It's the **generalization** of the Fenwick tree: it can answer *any* associative query (sum, min, max, gcd, OR, ...), not just prefix sums. And with **lazy propagation** it can also do **range updates** in $O(\log n)$.

Why it matters: it's the workhorse for "many range queries + updates" problems — the kind that appear in contests and hard interviews. The array of size n is stored as a tree of about $4n$ nodes.

Array: [1, 3, 5, 7] (0-indexed)
Segment tree (sums):

```

      [0..3] = 16
     /      \
  [0..1] = 4  [2..3] = 12
  /  \      /  \
[0]=1 [1]=3 [2]=5 [3]=7
```

2. The array representation

The tree is stored in a flat array. For node i : - left child = $2*i + 1$ - right child = $2*i + 2$ - parent = $(i - 1) / 2$

The root (index 0) covers $[0, n-1]$. Each node covers a range $[l, r]$; its children cover $[l, mid]$ and $[mid+1, r]$.

3. Implementation — sum segment tree

```
struct SegTree {
    int n;
    vector<long long> tree;
    SegTree(const vector<int>& a) {
        n = a.size();
```

```

        tree.assign(4 * n, 0);
        build(a, 0, 0, n - 1);
    }

    void build(const vector<int>& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int mid = (l + r) / 2;
        build(a, 2 * node + 1, l, mid);
        build(a, 2 * node + 2, mid + 1, r);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }

    void update(int idx, int val, int node, int l, int r) { // point update
        if (l == r) { tree[node] = val; return; }
        int mid = (l + r) / 2;
        if (idx <= mid) update(idx, val, 2 * node + 1, l, mid);
        else update(idx, val, 2 * node + 2, mid + 1, r);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }

    long long query(int ql, int qr, int node, int l, int r) { // range query
        if (ql > r || qr < l) return 0; // fully outside
        if (ql <= l && r <= qr) return tree[node]; // fully inside
        int mid = (l + r) / 2;
        return query(ql, qr, 2 * node + 1, l, mid)
            + query(ql, qr, 2 * node + 2, mid + 1, r);
    }

    // convenience wrappers
    void update(int idx, int val) { update(idx, val, 0, 0, n - 1); }
    long long query(int l, int r) { return query(l, r, 0, 0, n - 1); }
};

```

Time: build $O(n)$, update/query $O(\log n)$ | **Space:** $O(n)$

The three query cases: 1. **No overlap** → return the identity (0 for sum, $+\infty$ for min, $-\infty$ for max). 2. **Full overlap** → return this node's stored value (whole range is covered). 3. **Partial overlap** → split and combine the two children.

4. Worked example — range minimum query (min tree)

Only the combine step changes: `tree[node] = min(left, right)`; identity = `INT_MAX`.

// query returns INT_MAX for no-overlap; combine with min

5. Lazy propagation (range updates)

Without lazy propagation, updating a range `[l, r]` would touch $O(n)$ nodes. Lazy propagation defers the work: when an update fully covers a node, store the pending delta in a `lazy` array and stop. Children are updated only when we actually descend into them.

```

struct SegTreeLazy {
    int n;
    vector<long long> tree, lazy; // lazy[node] = pending add for this node's range

    SegTreeLazy(const vector<int>& a) { n = a.size(); tree.assign(4*n,0); lazy.assign(4*n,0); build(a,0,0,n-1)
; }

    void build(const vector<int>& a, int node, int l, int r) {
        if (l == r) { tree[node] = a[l]; return; }
        int mid = (l+r)/2;
        build(a,2*node+1,l,mid); build(a,2*node+2,mid+1,r);
        tree[node] = tree[2*node+1] + tree[2*node+2];
    }

    void push(int node, int l, int r) { // apply pending to children
        if (lazy[node]) {
            tree[node] += lazy[node] * (r - l + 1);
            if (l != r) {
                lazy[2*node+1] += lazy[node];
                lazy[2*node+2] += lazy[node];
            }
        }
    }
};

```

```

        lazy[node] = 0;
    }
}
void rangeAdd(int ql, int qr, long long val, int node, int l, int r) {
    push(node, l, r);
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) {
        tree[node] += val * (r - l + 1);
        if (l != r) { lazy[2*node+1] += val; lazy[2*node+2] += val; }
        return;
    }
    int mid = (l+r)/2;
    rangeAdd(ql, qr, val, 2*node+1, l, mid);
    rangeAdd(ql, qr, val, 2*node+2, mid+1, r);
    tree[node] = tree[2*node+1] + tree[2*node+2];
}
long long query(int ql, int qr, int node, int l, int r) {
    push(node, l, r);
    if (ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return tree[node];
    int mid = (l+r)/2;
    return query(ql, qr, 2*node+1, l, mid) + query(ql, qr, 2*node+2, mid+1, r);
}
};

```

Time: range update/query $O(\log n)$ | **Space:** $O(n)$

The rule: call `push` at the *start* of every visit so pending values are applied before reading or descending.

6. When to use & common mistakes

When to use: - Many **range queries + point updates** → segment tree (or Fenwick if prefix-only). - Many **range updates + range queries** → segment tree with lazy. - Non-prefix aggregates (min/max/gcd over ranges) → segment tree (Fenwick can't). - Range **k-th smallest** queries (merge sort tree / wavelet tree ideas).

Need	Fenwick (Ch. 23)	Segment tree
Prefix sums + point update	✓ (simpler, faster)	✓
Range min/max + point update	✗	✓
Range add + range query	✗ (with difficulty)	✓ lazy
Complexity	$O(\log n)$	$O(\log n)$, ~4x memory

Common mistakes: 1. **Array size:** allocate $4*n$, not $2*n$ (a common overflow bug). 2. Forgetting identity values per query type (sum→0, min→+inf, max→-inf, gcd→0). 3. In lazy trees, forgetting `push` before reads/descents. 4. Off-by-one in recursion bounds ($l == r$ base case). 5. Integer overflow in `tree[node] += val * (r - l + 1)` — use `long long`.

7. Practice

P1 [Easy] — Implement a sum segment tree; run range-sum + point-update queries. (*implement first*)

P2 [Medium] — Range sum query with point updates (the segment-tree version). (LC 307)

P3 [Medium] — Count of Smaller Numbers After Self. (LC 315 — *can use a segment tree over ranks or a Fenwick*)

P4 [Challenging] — Range minimum queries with range-add updates (lazy). (*classic HDU/RMQ problem*)

P5 [Challenging] — Range XOR queries with point updates. (*adapt the combine step*)

Solutions in `27-Practice-Solutions.md`.

Chapter 25 — Advanced Graph Algorithms

This chapter upgrades the graph toolkit from Ch. 18 with the algorithms that solve weighted shortest paths, MST (Prim), strongly connected components, bridges/articulation points, and touches on max flow.

1. Dijkstra — shortest path, non-negative weights

What: single-source shortest paths when all edge weights are ≥ 0 . **Why:** the standard "fastest route" algorithm; $O((V+E) \log V)$ with a heap. **How:** BFS-like, but the "frontier" is ordered by **current best distance**. Repeatedly extract the closest unsettled node and relax its edges.

Worked example — graph with edges: 1→2 (4), 1→3 (1), 3→2 (2), 2→4 (1), 3→4 (5).

```
dist from 1: d[1]=0
pop 1 (d=0): relax 2→4, 3→1
pop 3 (d=1): relax 2→3, 4→6
pop 2 (d=3): relax 4→4 (better than 6!)
pop 4 (d=4): done
Answer: dist[1]=0, dist[3]=1, dist[2]=3, dist[4]=4
```

```
typedef pair<long long, int> P;          // {distance, node}
const long long INF = 1e18;

vector<long long> dijkstra(int src, const vector<vector<P>>& g) {
    int n = g.size();
    vector<long long> dist(n, INF);
    priority_queue<P, vector<P>, greater<P>> pq; // min-heap on distance
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;           // stale entry → skip
        for (auto& [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Time: $O((V+E) \log V)$ | **Space:** $O(V+E)$

Why the heap + continue trick: old heap entries become stale when a node is improved. Skipping $d > \text{dist}[u]$ makes them harmless.

Doesn't work with negative edges (a shorter path found later could have been missed). That's Bellman-Ford's job.

2. Bellman-Ford — shortest path, negative weights allowed

What: single-source shortest paths, works with **negative edges**, detects **negative cycles**. **How:** relax **all edges V-1 times**. After k rounds, $\text{dist}[u]$ is the best path with $\leq k$ edges. A V-th round that still improves something $\Rightarrow$ a negative cycle exists.

```
vector<long long> bellmanFord(int n, int src, const vector<array<int,3>>& edges) {
    vector<long long> dist(n, INF);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {           // relax n-1 times
        bool changed = false;
        for (auto& [u, v, w] : edges)
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                changed = true;
            }
        if (!changed) break;
    }
}
```

```

    }
    for (auto& [u, v, w] : edges) // detect negative cycle
        if (dist[u] != INF && dist[u] + w < dist[v])
            /* negative cycle present */;
    return dist;
}

```

Time: $O(V \cdot E)$ | **Space:** $O(V)$

3. Floyd-Warshall — all-pairs shortest paths

What: shortest path between **every pair** of nodes (works with negative edges, no negative cycles). **How:** DP over intermediate nodes: $dp[k][i][j] = \min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])$. In-place, 3 nested loops.

```

void floydWarshall(vector<vector<long long>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; k++) // intermediate node
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}

```

Time: $O(V^3)$ | **Space:** $O(V^2)$ — only for small graphs ($V \leq \sim 400$)

4. Prim's MST — greedy with a heap

What: minimum spanning tree (like Kruskal, but grows one tree). **How:** start from any node; repeatedly add the cheapest edge that connects a visited node to an unvisited node.

```

long long prim(int n, const vector<vector<P>>& g) {
    vector<bool> inMST(n, false);
    vector<long long> key(n, INF);
    priority_queue<P, vector<P>, greater<P>> pq;
    key[0] = 0;
    pq.push({0, 0});
    long long total = 0;
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true;
        total += d;
        for (auto& [v, w] : g[u])
            if (!inMST[v] && w < key[v]) {
                key[v] = w;
                pq.push({w, v});
            }
    }
    return total;
}

```

Time: $O((V+E) \log V)$ | **Space:** $O(V+E)$

Kruskal vs Prim: Kruskal = sort edges + DSU (sparse graphs). Prim = heap (dense graphs, or when adjacency list is given).

5. Strongly Connected Components (Kosaraju / Tarjan)

What: in a **directed** graph, an SCC is a maximal set of nodes where every node can reach every other. Condensing SCCs into single nodes makes a **DAG** — the gateway to DAG problems.

Kosaraju's algorithm: 1. DFS, record **finish order**. 2. Reverse all edges. 3. DFS in reverse finish order — each pass discovers one SCC.

```

vector<int> sccKosaraju(int n, const vector<vector<int>>& g) {
    vector<vector<int>> gr(n);
    for (int u = 0; u < n; u++)
        for (int v : g[u]) gr[v].push_back(u);

    vector<bool> vis(n, false);
    vector<int> order;
    function<void(int)> dfs1 = [&](int u) {
        vis[u] = true;
        for (int v : g[u]) if (!vis[v]) dfs1(v);
        order.push_back(u);
    };
    for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);

    vector<int> comp(n, -1);
    int id = 0;
    function<void(int)> dfs2 = [&](int u) {
        comp[u] = id;
        for (int v : gr[u]) if (comp[v] == -1) dfs2(v);
    };
    for (int i = n - 1; i >= 0; i--)
        if (comp[order[i]] == -1) { dfs2(order[i]); id++; }
    return comp; // comp[u] = component id of u
}

```

Time: $O(V+E)$ | **Space:** $O(V+E)$

Useful fact: if we contract SCCs, the result is a DAG — so "number of SCCs with out-degree 0" answers "nodes that can reach everything" (LC 2115 / popular problems).

6. Bridges & articulation points (Tarjan's algorithm)

What: - **Bridge:** an edge whose removal disconnects the graph. - **Articulation point:** a node whose removal disconnects the graph.

How (DFS, discover times): $tin[u]$ = discovery time; $low[u]$ = lowest discovery time reachable from u 's subtree via back edges. - Edge $u-v$ (v is child) is a **bridge** iff $low[v] > tin[u]$. - Node u is an **articulation point** iff some child v has $low[v] \geq tin[u]$ (or u is root with ≥ 2 children).

```

vector<vector<int>> findBridges(int n, const vector<vector<int>>& g) {
    vector<int> tin(n, -1), low(n);
    int timer = 0;
    vector<vector<int>> bridges;

    function<void(int,int)> dfs = [&](int u, int parent) {
        tin[u] = low[u] = timer++;
        for (int v : g[u]) {
            if (v == parent) continue;
            if (tin[v] != -1) low[u] = min(low[u], tin[v]); // back edge
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.push_back({u, v}); // bridge!
            }
        }
    };
    for (int i = 0; i < n; i++) if (tin[i] == -1) dfs(i, -1);
    return bridges;
}

```

Time: $O(V+E)$ | **Space:** $O(V+E)$

Common bug: the `if (v == parent) continue;` skips the direct parent — but with multi-edges, `v == parent` could be a second edge. Use an edge id instead if parallel edges exist.

7. Max flow (brief introduction)

Problem: given a directed graph with capacities, find the max flow from source s to sink t .

Ford-Fulkerson / Edmonds-Karp: repeatedly find an augmenting path (BFS → Edmonds-Karp, $O(VE^2)$) in the **residual graph**, pushing the bottleneck flow. Add reverse edges so flow can be "undone".

```
// skeleton (adjacency list of {neighbor, capacity, reverse_edge_index})
// while (bfs finds path s→t with capacity > 0)
//     push bottleneck along path, decrement forward, increment reverse
```

Why it matters: max flow = min cut (max-flow min-cut theorem) — it solves bipartite matching, vertex-disjoint paths, project selection, and many assignment problems.

8. Algorithm selection table

Problem	Algorithm	Complexity
Shortest path, unweighted	BFS	$O(V+E)$
Shortest path, non-negative weights	Dijkstra	$O((V+E) \log V)$
Shortest path, negative weights (no neg cycle)	Bellman-Ford	$O(VE)$
All-pairs shortest paths	Floyd-Warshall	$O(V^3)$
Minimum spanning tree	Kruskal / Prim	$O(E \log E) / O((V+E) \log V)$
Cycle detection, directed	DFS (colors) / topo order	$O(V+E)$
Topological order	Kahn (in-degree)	$O(V+E)$
Strongly connected components	Kosaraju / Tarjan	$O(V+E)$
Bridges / articulation points	Tarjan	$O(V+E)$
Max flow	Ford-Fulkerson / Dinic	$O(V E^2) / O(V^2 E)$

9. When to use & common mistakes

When to use: - "Shortest / cheapest path" → BFS/Dijkstra/Bellman-Ford by weight sign. - "Every pair of cities distance" → Floyd-Warshall. - "Min cost to connect everything" → Kruskal/Prim. - "Order/dependency/leader election in directed graphs" → SCC. - "Which roads/nodes are critical" → bridges/articulation points. - "Assign / match / max throughput" → max flow.

Common mistakes: 1. Using Dijkstra with negative weights (silently wrong results). 2. Forgetting the stale-entry `continue` in Dijkstra (infinite loop risk). 3. Bellman-Ford off-by-one on the number of relaxations ($n-1$, not n). 4. Floyd-Warshall loop order — **k must be the outermost loop**. 5. In Tarjan, not using edge ids for multi-edges. 6. `int` overflow in distances — use `long long`, `INF = 1e18`.

10. Practice

P1 [Easy] — Shortest path with BFS in an unweighted graph. (LC 127 is the word-ladder version; try the plain version first)

P2 [Medium] — Dijkstra: network delay time (min time for signal to reach all nodes). (LC 743)

P3 [Medium] — Cheapest Flights Within K Stops. (LC 787 — BFS/bellman-style, careful)

P4 [Medium] — Find the City With the Smallest Number of Neighbors at a Threshold Distance. (LC 1334 — Floyd-Warshall)

P5 [Challenging] — Number of provinces / SCC applications. (LC 547 or a Kosaraju example)

P6 [Challenging] — Critical Connections in a Network (find all bridges). (LC 1192 — Tarjan)

Solutions in `27-Practice-Solutions.md`.

Chapter 26 — Problem-Solving Patterns

This chapter is the meta-skill: how to **look at a new problem** and pick the right tool. It collects the patterns scattered through the book into a decision framework, plus the "catalog" of classic patterns you should recognize on sight.

1. The problem-solving framework

When you get any problem, walk this ladder:

1. UNDERSTAND
 - Restate the problem in your own words.
 - What is the input size? (→ which complexity is feasible, see Ch. 02)
 - What exactly is the output? (one answer / all answers / a bool / count?)
2. CLASSIFY
 - Array/String/Sequence → think: sorted? subarray/substring? prefix sums?
 - Grid → BFS/DFS.
 - "Entities + relationships" → graph.
 - "Min / max / count of ways / possible?" → DP, greedy, or backtracking.
 - "Has been seen / duplicate / pair" → hashing.
 - "k-th / top-k / nearest" → heap.
 - "Order preserved / monotone / next greater" → monotonic stack.
 - "Queries + updates on ranges" → segment tree / Fenwick / prefix sums.
3. BRUTE FORCE FIRST
 - Write the $O(n^2)$ / exponential version. It clarifies the structure.
 - Then ask: which step is wasteful? Which pattern removes it?
4. IMPROVE
 - $O(n^2)$ nested loop → hashing, two pointers, sliding window, or sorting.
 - Repeated recomputation → memoization / DP / prefix sums.
 - Try every subarray → sliding window or prefix sum.
 - "Maximum/minimum X such that condition" → binary search on answer.
5. CHECK EDGE CASES
 - Empty input, single element, all equal, all negative, overflow (use long long).

2. Pattern catalog (recognize → apply)

#	Pattern	Signature	Complexity	Home
1	Prefix sums	range sums, subarray sums	build $O(n)$, query $O(1)$	Ch. 03
2	Binary search on answer	min/max X such that $P(X)$ monotonic	$O(\log M \cdot \text{check})$	Ch. 05
3	Two pointers (opposite)	sorted array pair/triplet	$O(n)$	Ch. 08
4	Two pointers (same dir)	in-place dedup / partition	$O(n)$	Ch. 08
5	Sliding window (fixed)	subarray of size k	$O(n)$	Ch. 09
6	Sliding window (variable)	longest/shortest substring with property	$O(n)$	Ch. 09
7	HashMap complement	"find the pair that satisfies"	$O(n)$	Ch. 07
8	Frequency counter	anagrams, duplicates	$O(n)$	Ch. 07
9	Monotonic stack	next/previous greater/smaller	$O(n)$	Ch. 11
10	Slow/fast pointers	cycle detection, middle	$O(n)$	Ch. 10
11	Dummy node	linked list head mutations	$O(n)$	Ch. 10
12	Backtracking	generate all / fill constraints	exponential, pruned	Ch. 13
13	Greedy (exchange arg)	interval scheduling, jumps	$O(n \log n)$	Ch. 17
14	BFS	shortest path unweighted, levels	$O(V+E)$	Ch. 18
15	DFS	reachability, components, topo	$O(V+E)$	Ch. 18
16	Dijkstra	weighted shortest path ≥ 0	$O((V+E) \log V)$	Ch. 25

#	Pattern	Signature	Complexity	Home
17	Topological sort	dependencies, order	$O(V+E)$	Ch. 18
18	MST (Kruskal/Prim)	min cost to connect	$O(E \log E)$	Ch. 22/25
19	Memoization / tabulation	count ways, min cost, possible	polynomial	Ch. 19
20	0/1 vs unbounded knapsack	capacity constraint	$O(n \cdot W)$	Ch. 19
21	Bitmask DP	subset states, $n \leq 20$	$O(2^n \cdot n)$	Ch. 21
22	DSU	connectivity, components, MST	$\sim O(1)$	Ch. 22
23	Fenwick / Segment tree	dynamic range queries	$O(\log n)$	Ch. 23/24
24	Trie	prefix matching, autocomplete	$O(L)$	Ch. 20
25	SCC condensation	directed reachability groups	$O(V+E)$	Ch. 25

3. Pattern spotlights (the ones people miss)

Binary search on answer (Ch. 05)

Look for: "**minimum/maximum X**" where feasibility is monotonic. Examples: min speed to eat bananas in h hours; min capacity ship to ship all in d days; maximize the minimum distance between cows; smallest split such that $\text{sum} \leq k$.

Template:

```
bool feasible(int x);    // monotonic: false...false, true...true

int lo = 1, hi = 1e9;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;    // minimize
    else lo = mid + 1;
}
```

Monotonic stack (Ch. 11)

Look for: "next **greater/smaller**", "nearest greater/smaller", "largest rectangle", "stock span". Key idea: **pop dominated candidates** to keep the stack monotonic.

The "two-transition" DP reading (Ch. 19)

For "min/max cost" sequence problems, most solutions have exactly **two or three transitions**. Draw $\text{dp}[i]$, list the last-step choices, write the recurrence.

Multi-source BFS (Ch. 18)

"Rotting oranges", "zero-one matrix distance to nearest 0", "walls and gates" — push **all** starting points into the queue at once; distances still come out correct because BFS processes by level.

Reverse-thinking / process backwards

Some problems are easier backwards: - "Remove stones" → add stones back (DSU offline). - "Can reach target" → work from target toward start (LC 45 / jump game II). - "Delete edges" → build up with DSU (number of islands II).

Coordinate compression

When values are large but only their **order** matters (inversions, k-th rank, segment tree over ranks), map each value to its 1-indexed sorted position:

```
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
int rank = lower_bound(v.begin(), v.end(), x) - v.begin() + 1;
```

4. Complexity → algorithm cheat (input size decoder)

n (input size)	Target complexity	Reach for
$n \leq 10$	$O(n!)$, $O(2^n)$	backtracking, bitmask DP
$n \leq 20$	$O(2^n)$	bitmask DP
$n \leq 500$	$O(n^3)$	Floyd-Warshall, interval DP
$n \leq 5000$	$O(n^2)$	LIS $O(n^2)$, 2D DP
$n \leq 1e5$	$O(n \log n)$	sort, Dijkstra, Fenwick, segment tree, greedy
$n \leq 1e7$	$O(n)$	sliding window, two pointers, hashing
any	$O(\log n)$	binary search

5. Interview tactics (how to actually solve in 45 minutes)

1. **Ask clarifying questions** — constraints, duplicates, negative values, memory.
2. **Say the brute force out loud** first. It shows engagement and gives structure.
3. **State your improved approach** with complexity before coding.
4. **Write clean, self-documenting code** — helpers, no magic numbers.
5. **Walk through a small example** by hand with your code.
6. **Test edge cases** — empty, one element, large values.
7. If stuck, **go back up the ladder** — re-read constraints, reconsider pattern.

6. Practice

These mix every pattern — solve them from scratch with the framework above.

P1 [Easy] — Contains duplicate (hash). (LC 217)

P2 [Easy] — Valid palindrome (two pointers). (LC 125)

P3 [Medium] — Two sum (hash). (LC 1)

P4 [Medium] — Longest substring without repeating chars (sliding window). (LC 3)

P5 [Medium] — K-th largest element (heap). (LC 215)

P6 [Medium] — Number of islands (BFS/DFS). (LC 200)

P7 [Medium] — House robber (DP). (LC 198)

P8 [Challenging] — Meeting rooms II (greedy + two pointers on sorted). (LC 253)

P9 [Challenging] — Sliding window maximum (monotonic deque). (LC 239)

P10 [Challenging] — Word ladder (BFS over strings). (LC 127)

P11 [Challenging] — Minimum cost to connect sticks (greedy + heap). (LC 1167)

Solutions in `27-Practice-Solutions.md`.

Chapter 27 — Practice Solutions & Hints

How to use this file: try each problem on your own first. If you're stuck, read only the **hint** for that problem. Only after genuinely attempting (or to verify) should you read the **solution**. The hints are written to push you in the right direction without giving away the whole answer.

Every problem is organized by its chapter. Problems are referenced by their number in each chapter's practice section (e.g., "03-P3" = Chapter 03, Problem 3).

Part A — Hints

Chapter 01 (STL)

- **P1 (217):** Use an `unordered_set`; return true the moment an insert collides.
- **P2 (88):** Two pointers from the *end* of both arrays writing into the back of the first array avoids extra space.
- **P3 (3):** Sliding window with the "last seen index" map — covered fully in Ch. 09.
- **P4:** Count divisors for each number; group by count in an `unordered_map<int,int>`; track max group size. Mind n up to $\sim 4 \cdot 10^4$.

Chapter 02 (Complexity)

- **P1:** (a) outer loop $O(\log n)$, inner $O(n) \rightarrow O(n \log n)$. (b) sum of $i^2 \rightarrow O(n^3)$. (c) recursion halves n with $O(1)$ work $\rightarrow O(\log n)$ time, $O(\log n)$ space.
- **P2:** $a=3, b=3, d=1 \rightarrow$ compare `a` and `b^d = 3`.
- **P3:** $O(n^2)$ means 10x input $\rightarrow 100x$ time.
- **P4:** An outer loop that runs n times and an inner loop that starts at 1 and doubles up to $\sim i$: the inner does $O(\log i)$ work. Sum $\log i$ over $i = \log(n!) \approx n \log n$.

Chapter 03 (Arrays)

- **P1 (1480):** `pref[i] = pref[i-1] + a[i]`.
- **P2 (189):** Reverse the whole array, then reverse the two halves. Remember `k %= n`.
- **P3 (560):** Prefix sums in a hashmap counting how many times each prefix sum has occurred. Full solution in Ch. 07.
- **P4 (121):** Track the min price seen so far; `answer = max(price - minSoFar)`.
- **P5 (238):** Two passes — left products then right products.
- **P6:** Sliding window with a frequency map, `max distinct = k`.

Chapter 04 (Strings)

- **P1 (125):** Two pointers; skip non-alphanumerics; compare lowercased chars.
- **P2 (242):** Frequency array of 26; ++ for s, -- for t; all must be 0.
- **P3 (151):** Reverse the whole string, then reverse each word. Watch spaces.
- **P4 (14):** Sort or scan — compare first string's chars with the rest.
- **P5 (28):** Rabin-Karp rolling hash, or implement KMP's prefix function.
- **P6 (restore IP):** Backtracking; a segment is 1-3 digits, ≤ 255 , no leading zeros.

Chapter 05 (Searching)

- **P1 (704):** Classic binary search. `lo <= hi`, `mid = lo + (hi-lo)/2`.
- **P2 (34):** `lower_bound` then `upper_bound`; or two binary searches.
- **P3 (33):** One side of mid is always sorted — decide which and search there.
- **P4:** `a[i] == i` is monotonic-ish once you consider `a[i] - i` — binary search on `a[i] - i >= 0`.
- **P5 (69):** Binary search x in $[0, n]$; `mid*mid <= n` is the monotonic predicate. Watch overflow: use `long long`.
- **P6 (875):** Binary search on speed; `canEat(mid) = sum of ceil(pile/mid) <= h`.

Chapter 06 (Sorting)

- **P1 (215):** `nth_element` ($O(n)$) or a size- k min-heap.
- **P2 (88):** Standard merge, from the back.
- **P3 (56):** Sort by start; merge when `next.start <= cur.end`.

- **P4:** Merge sort counting inversions during the merge (when a left element > right element, all remaining left elements are inversions).
- **P5 (75):** Dutch National Flag — three pointers (lo, mid, hi).
- **P6:** A window of size k+1; put the window in a min-heap, pop the smallest each step.

Chapter 07 (Hashing)

- **P1 (217):** `unordered_set`.
- **P2 (242):** Frequency array.
- **P3 (1):** Store `value → index`; check complement.
- **P4 (560):** `seen[prefix]`, add `seen[cur-k]` each step.
- **P5 (49):** Sorted string as the key.
- **P6:** Store first occurrence of each `prefixSum % k`; when a remainder repeats, the subarray between them is divisible by k. Watch negatives in C++ `%`.

Chapter 08 (Two Pointers)

- **P1 (167):** Squeeze from both ends; move `l` if `sum < target` else `r`.
- **P2 (125):** Two pointers with char filtering.
- **P3 (26):** Fast/slow write pointer.
- **P4 (283):** Same idea — write non-zeros forward, then zero-fill the tail.
- **P5 (15):** Sort, fix one element, two-pointer the rest; skip duplicates.
- **P6 (42):** The leftMax/rightMax approach from the chapter.

Chapter 09 (Sliding Window)

- **P1 (485):** Single pass tracking current run; reset on 0.
- **P2 (3):** Last-seen map + shrink window on repeat.
- **P3:** Fixed window sum, slide with `+a[i] - a[i-k]`.
- **P4 (340):** Frequency map + `while (distinct > k)` shrink.
- **P5 (76):** The need/have pattern from the chapter.
- **P6 (209):** Variable window; while `sum ≥ k`, shrink left and update answer.

Chapter 10 (Linked Lists)

- **P1 (206):** Three pointers `prev/cur/nxt`.
- **P2 (21):** Dummy node + merge.
- **P3 (141):** Slow/fast.
- **P4 (19):** Fast n+1 ahead, then move both.
- **P5 (142):** After collision, reset one to head; both move 1 step.
- **P6 (143):** Find middle → reverse second half → interleave.
- **P7 (146):** Doubly linked list + hashmap; move-to-front on access; evict tail on overflow.

Chapter 11 (Stacks & Queues)

- **P1 (20):** Push openers, match closers.
- **P2 (232):** Use two stacks; when `out` empty, pour `in` into `out`.
- **P3 (739):** Monotonic decreasing stack of indices; `answer = i - top`.
- **P4 (150):** Push numbers; on operator, pop two, apply, push result.
- **P5 (84):** `prev-smaller` + `next-smaller` with monotonic stacks (chapter code).
- **P6 (239):** Monotonic deque of indices (chapter code).

Chapter 12 (Recursion)

- **P1 (344):** Swap ends, recurse inward, base case `l >= r`.
- **P2 (509):** `fib(n)=fib(n-1)+fib(n-2)`, base `n≤1`.
- **P3 (78):** Include/exclude recursion (chapter code).
- **P4 (Hanoi):** Move `n-1` to aux, move `1` to dest, move `n-1` from aux to dest.
- **P5 (22):** Recursion with `open / close` counts; only add `)` if `close < open`.
- **P6:** Fast exponentiation, but `ans = (ans * a) % m` at every multiply.

Chapter 13 (Backtracking)

- **P1 (78):** Include/exclude.
- **P2 (22):** open/close counts (see 12-P5).
- **P3 (46):** `used[]` + pick unused each step.
- **P4 (39):** Choose a candidate, stay at the *same index* (can reuse), backtrack on `sum > target`; sort + skip duplicates for uniqueness.
- **P5 (51):** Column + diagonal pruning (chapter code).
- **P6 (37):** Find an empty cell, try '1'..'9', validate row/col/box, recurse.
- **P7 (79):** DFS from each matching start; mark visited in place, unmark on backtrack.

Chapter 14 (Trees)

- **P1 (104):** `1 + max(left, right)`.
- **P2 (226):** Swap children, recurse.
- **P3 (102):** BFS with `sz` capture.
- **P4 (101):** `isMirror(left, right)` (chapter code).
- **P5 (98):** (min, max) bounds or inorder-is-increasing.
- **P6 (105):** Preorder gives root; inorder splits left/right; recurse with index maps.
- **P7 (236):** The "both sides found" logic (chapter code).

Chapter 15 (BST)

- **P1 (700):** Standard BST search.
- **P2 (701):** Attach at the null spot.
- **P3 (98):** Bounds.
- **P4 (230):** Inorder with a counter.
- **P5 (450):** Three delete cases (chapter code).
- **P6 (538):** Reverse inorder, accumulate running sum.

Chapter 16 (Heaps)

- **P1 (215):** size-`k` min-heap or `nth_element`.
- **P2 (23):** Heap of current heads (chapter code).
- **P3 (973):** Max-heap of size `k` by `distance2`; or `nth_element` with comparator.
- **P4 (295):** Two heaps (chapter code).
- **P5 (621):** Count frequencies; max heap; schedule with cooldown; or math formula `max(maxFreq-1)*(n+1) + countOfMax`.
- **P6:** Sliding window min-heap of size `k+1`.

Chapter 17 (Greedy)

- **P1 (55):** Furthest reachable index (chapter code).
- **P2 (455):** Sort both; greedy smallest cookie that satisfies smallest greed.
- **P3:** For systems where greedy works (each coin $\geq 2x$ previous), take as many largest as possible.

- **P4 (253):** Sort starts/ends sweep (chapter code).
- **P5 (452):** Sort by end; shoot when a balloon's start > last arrow point.
- **P6 (45):** BFS over "reach windows" — track current reach and next reach.
- **P7 (134):** If total gas $\geq$ total cost, a solution exists; find the index where the running deficit is minimized (reset start when balance < 0).

Chapter 18 (Graphs)

- **P1 (1971):** BFS/DFS with visited; or DSU.
- **P2 (200):** Chapter code.
- **P3 (133):** DFS/BFS + `unordered_map<Node*, Node*>` for clones.
- **P4 (994):** Multi-source BFS (all rotten cells seeded).
- **P5 (207):** Kahn's algorithm — if `order.size() != n`, there's a cycle.
- **P6 (210):** Return the Kahn order.
- **P7 (547):** Count components (DFS or DSU).
- **P8:** Kruskal (Ch. 22 code).

Chapter 19 (DP)

- **P1 (70):** `dp[i]=dp[i-1]+dp[i-2]`.
- **P2 (198):** `dp[i]=max(dp[i-1], dp[i-2]+a[i])`.
- **P3 (300):** Chapter code ($O(n^2)$); bonus: $O(n \log n)$ patience sorting.
- **P4 (322):** Unbounded knapsack, min coins: `dp[w]=min(dp[w], dp[w-coin]+1)`; loop coin outer, w ascending.
- **P5 (1143):** Chapter code.
- **P6 (62):** `dp[i][j]=dp[i-1][j]+dp[i][j-1]`, base top row & left col = 1.
- **P7 (5):** Chapter code (fill by length).
- **P8 (72):** `dp[i][j]` = edit distance of prefixes; if equal copy diagonal, else $1 + \min(\text{insert/delete/replace})$.
- **P9:** Chapter code (descend in 1D).

Chapter 20 (Tries)

- **P1 (208):** Chapter code.
- **P2 (211):** DFS from trie node; '.' means try all 26 children.
- **P3 (212):** Chapter skeleton.
- **P4 (720):** Insert all; DFS longest word where every prefix exists in the trie.

Chapter 21 (Bit Manipulation)

- **P1 (136):** XOR all.
- **P2 (231):** `n > 0 && (n & (n-1)) == 0`.
- **P3 (461):** `popcount(a ^ b)`.
- **P4 (338):** `dp[i] = dp[i >> 1] + (i & 1)` — very elegant.
- **P5 (260):** XOR all, isolate lowest set bit, partition & XOR each group.
- **P6 (137):** Count each bit across all numbers mod 3.
- **P7 (371):** `sum = a ^ b`, `carry = (a & b) << 1`, repeat until carry = 0.

Chapter 22 (DSU)

- **P1 (547/323):** Chapter code.
- **P2 (721):** Map email $\rightarrow$ index; union emails of each account; rebuild from representatives.
- **P3 (684):** Union edges; first edge that returns false from unite is the answer.
- **P4 (128):** Hash set of values; only start runs from `x` when `x-1` absent; extend while `x+len` present. (Can also use DSU.)

- **P5 (305):** Process additions; union with already-land neighbors; decrement count per successful union.

Chapter 23 (Fenwick)

- **P1:** Implement the chapter code; test add/sum on a sample.
- **P2:** Chapter code (compression + counts).
- **P3 (315):** Process right→left; `rank = rank of nums[i]`; `answer = sum(rank-1)`; then `add(rank, 1)`.
- **P4:** BIT + binary search on the prefix sum to find the k-th smallest.

Chapter 24 (Segment Trees)

- **P1:** Implement and test build/query/update.
- **P2 (307):** Standard segment tree (chapter code) — or a Fenwick tree.
- **P3 (315):** Segment tree over ranks (or Fenwick) — same as 23-P3.
- **P4:** Lazy propagation version (chapter code).
- **P5:** Combine step = `tree[node] = tree[left] ^ tree[right]`, identity 0.

Chapter 25 (Advanced Graphs)

- **P1:** BFS.
- **P2 (743):** Dijkstra; `answer = max dist`; -1 if any unreachable.
- **P3 (787):** Bellman-Ford relaxed K times (each iteration = one stop more), or BFS over (city, stops) with a dist map.
- **P4 (1334):** Floyd-Warshall, then count reachable $\leq$ threshold.
- **P5:** Kosaraju (chapter code); count components.
- **P6 (1192):** Tarjan bridge-finding (chapter code).

Chapter 26 (Patterns)

- **P1 (217):** set. **P2 (125):** two pointers. **P3 (1):** hashmap. **P4 (3):** sliding window. **P5 (215):** heap. **P6 (200):** DFS. **P7 (198):** DP.
- **P8 (253):** sort starts/ends. **P9 (239):** monotonic deque. **P10 (127):** BFS, transform each char, avoid regenerating words.
- **P11 (1167):** always combine the two smallest sticks (min-heap).

Part B — Full Solutions

Chapter 01 — STL

01-P1 — Contains duplicate (LC 217)

```
bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> s;
    for (int x : nums) {
        if (s.count(x)) return true;
        s.insert(x);
    }
    return false;
}
```

Time $O(n)$, space $O(n)$.

01-P2 — Merge sorted arrays in place (LC 88)

```
void merge(vector<int>& a, int m, vector<int>& b, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
```

```

while (i >= 0 && j >= 0)
    a[k--] = (a[i] > b[j]) ? a[i--] : b[j--];
while (j >= 0) a[k--] = b[j--];
}

```

Time $O(m+n)$, space $O(1)$.

01-P3 — Longest substring without repeating characters (LC 3)

See full solution in Ch. 09 (09-P2).

01-P4 — Largest group by number of divisors

```

int largestGroupSize(int n) {
    unordered_map<int,int> cnt;
    int best = 0;
    for (int x = 1; x <= n; x++) {
        int d = 0;
        for (int i = 1; i * i <= x; i++) {
            if (x % i == 0) {
                d++;
                if (i != x / i) d++;
            }
        }
        best = max(best, ++cnt[d]);
    }
    return best;
}

```

Time $O(n\sqrt{n})$, space $O(n)$.

Chapter 02 — Complexity

02-P1 — Analyze the snippets

```

// (a)  $O(n \log n)$ : outer halving ( $\log n$ )  $\times$  inner linear ( $n$ )
// (b)  $O(n^3)$ :  $\sum_{i=0}^{n-1} i^2 = n(n-1)(2n-1)/6 \rightarrow O(n^3)$ 
// (c)  $O(\log n)$  time,  $O(\log n)$  space: halves  $n$  each call, constant work per call

```

02-P2 — Master method

$T(n) = 3T(n/3) + O(n)$: $a=3, b=3, d=1 \rightarrow a == b^d$ ($3 == 3$) $\rightarrow O(n \log n)$.

```

// The described function:
int f(int n) {
    if (n <= 1) return 1;
    int sum = 0;
    for (int i = 0; i < n; i++) sum++; //  $O(n)$ 
    return f(n / 3) + f(n / 3) + f(n / 3) + sum;
}

```

02-P3 — $O(n^2)$ scaling

10x input $\rightarrow$ 100x time $\rightarrow 2 \text{ s} \times 100 = \mathbf{200 \text{ seconds}}$ ($\approx 3.3 \text{ min}$).

02-P4 — An $O(n \log n)$ loop structure

```

for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j *= 2) x++; //  $\log_2 i$  iterations
//  $\sum \log_2 i = \log_2(n!) \approx n \log_2 n$ 

```

Justification: the inner loop runs $\lceil \log_2 i \rceil$ times; the sum telescopes to $O(n \log n)$.

Chapter 03 — Arrays

03-P1 — Running sum (LC 1480)

```
vector<int> runningSum(vector<int>& a) {  
    for (int i = 1; i < (int)a.size(); i++) a[i] += a[i - 1];  
    return a;  
}
```

03-P2 — Rotate right by k (LC 189)

```
void rotate(vector<int>& a, int k) {  
    int n = a.size();  
    k %= n;  
    reverse(a.begin(), a.end());  
    reverse(a.begin(), a.begin() + k);  
    reverse(a.begin() + k, a.end());  
}
```

Time $O(n)$, space $O(1)$.

03-P3 — Subarray sum equals k (LC 560)

```
int subarraySum(vector<int>& a, int k) {  
    unordered_map<int, int> seen;  
    seen[0] = 1;  
    int cur = 0, ans = 0;  
    for (int x : a) {  
        cur += x;  
        ans += seen[cur - k];  
        seen[cur]++;  
    }  
    return ans;  
}
```

Time $O(n)$, space $O(n)$.

03-P4 — Best time to buy and sell (LC 121)

```
int maxProfit(vector<int>& p) {  
    int best = 0, mn = INT_MAX;  
    for (int x : p) {  
        mn = min(mn, x);  
        best = max(best, x - mn);  
    }  
    return best;  
}
```

03-P5 — Product of array except self (LC 238)

```
vector<int> productExceptSelf(vector<int>& a) {  
    int n = a.size();  
    vector<int> res(n, 1);  
    for (int i = 1; i < n; i++) res[i] = res[i - 1] * a[i - 1]; // left products  
    int r = 1;  
    for (int i = n - 1; i >= 0; i--) { res[i] *= r; r *= a[i]; } // right products  
    return res;  
}
```

03-P6 — Longest subarray with at most k distinct values

```
int longestAtMostK(const vector<int>& a, int k) {  
    int l = 0, best = 0;  
    unordered_map<int, int> cnt;  
    for (int r = 0; r < (int)a.size(); r++) {  
        cnt[a[r]]++;  
        while ((int)cnt.size() > k) {  
            if (--cnt[a[l]] == 0) cnt.erase(a[l]);  
            l++;  
        }  
        best = max(best, r - l + 1);  
    }
```

```

        l++;
    }
    best = max(best, r - l + 1);
}
return best;
}

```

Time $O(n)$, space $O(k)$.

Chapter 04 — Strings

04-P1 — Valid palindrome (LC 125)

```

bool isPalindrome(string s) {
    int i = 0, j = s.size() - 1;
    while (i < j) {
        while (i < j && !isalnum(s[i])) i++;
        while (i < j && !isalnum(s[j])) j--;
        if (tolower(s[i]) != tolower(s[j])) return false;
        i++; j--;
    }
    return true;
}

```

04-P2 — Valid anagram (LC 242)

```

bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    int cnt[26] = {0};
    for (char c : s) cnt[c - 'a']++;
    for (char c : t) if (--cnt[c - 'a'] < 0) return false;
    return true;
}

```

04-P3 — Reverse words in a string (LC 151)

```

string reverseWords(string s) {
    stringstream ss(s);
    string w, res;
    while (ss >> w) res = w + (res.empty() ? "" : " ") + res;
    return res;
}

```

Or: reverse whole string, then reverse each word.

04-P4 — Longest common prefix (LC 14)

```

string longestCommonPrefix(vector<string>& strs) {
    if (strs.empty()) return "";
    for (int i = 0; i < (int)strs[0].size(); i++)
        for (auto& s : strs)
            if (i >= (int)s.size() || s[i] != strs[0][i])
                return strs[0].substr(0, i);
    return strs[0];
}

```

Time $O(n \cdot L)$.

04-P5 — First occurrence of pattern (LC 28) with Rabin-Karp

```

int strStr(string t, string p) {
    int n = t.size(), m = p.size();
    if (m == 0) return 0;
    if (m > n) return -1;
    long long mod = 1e9 + 7, base = 131;
    long long pH = 0, tH = 0, powB = 1;
    for (int i = 0; i < m; i++) {
        pH = (pH * base + p[i]) % mod;
        tH = (tH * base + t[i]) % mod;
    }
}

```

```

        powB = powB * base % mod;
    }
    for (int i = 0; i + m <= n; i++) {
        if (pH == tH && t.substr(i, m) == p) return i;
        if (i + m < n)
            tH = ((tH - t[i] * powB % mod + mod) % mod * base + t[i + m]) % mod;
    }
    return -1;
}

```

Time $O(n+m)$ average, space $O(1)$.

04-P6 — Restore IP addresses

```

vector<string> restoreIpAddresses(string s) {
    vector<string> res;
    string cur;
    function<void(int,int)> dfs = [&](int pos, int seg) {
        if (seg == 4) {
            if (pos == (int)s.size()) { res.push_back(cur); }
            return;
        }
        for (int len = 1; len <= 3 && pos + len <= (int)s.size(); len++) {
            string part = s.substr(pos, len);
            if (len > 1 && part[0] == '0') continue;           // no leading zeros
            if (stoi(part) > 255) continue;
            string before = cur;
            cur += part + (seg == 3 ? "" : ".");
            dfs(pos + len, seg + 1);
            cur = before;
        }
    };
    dfs(0, 0);
    return res;
}

```

Chapter 05 — Searching

05-P1 — Binary search (LC 704)

```

int search(vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

05-P2 — First and last position (LC 34)

```

vector<int> searchRange(vector<int>& a, int x) {
    int l = lower_bound(a.begin(), a.end(), x) - a.begin();
    if (l == (int)a.size() || a[l] != x) return {-1, -1};
    int r = upper_bound(a.begin(), a.end(), x) - a.begin() - 1;
    return {l, r};
}

```

05-P3 — Search in rotated sorted array (LC 33)

```

int search(vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[lo] <= a[mid]) { // left half is sorted
            if (a[lo] <= x && x < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        }
    }
}

```

```

    } else { // right half is sorted
        if (a[mid] < x && x <= a[hi]) lo = mid + 1;
        else hi = mid - 1;
    }
}
return -1;
}

```

05-P4 — Fixed point index where $a[i] == i$

```

int fixedPoint(vector<int>& a) {
    int lo = 0, hi = (int)a.size() - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] - mid >= 0) { ans = mid; hi = mid - 1; } // try earlier
        else lo = mid + 1;
    }
    return ans;
}

```

(Works for distinct/sorted input; the monotonic predicate is $a[i] - i \geq 0$.)

05-P5 — Square root (LC 69)

```

int mySqrt(int x) {
    long long lo = 0, hi = x;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (mid * mid <= x) lo = mid + 1;
        else hi = mid - 1;
    }
    return (int)hi;
}

```

05-P6 — Koko eating bananas (LC 875)

```

bool canEat(int speed, const vector<int>& piles, int h) {
    long long t = 0;
    for (int p : piles) t += (p + speed - 1) / speed;
    return t <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canEat(mid, piles, h)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}

```

Chapter 06 — Sorting

06-P1 — K-th largest (LC 215)

```

int findKthLargest(vector<int>& a, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int x : a) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    return pq.top();
}

```

Time $O(n \log k)$, space $O(k)$. Or `nth_element(a.begin(), a.begin()+k-1, a.end())`.

06-P2 — Merge sorted arrays (LC 88)

See 01-P2.

06-P3 — Merge intervals (LC 56)

```
vector<vector<int>> merge(vector<vector<int>>& in) {
    sort(in.begin(), in.end());
    vector<vector<int>> res;
    for (auto& itv : in) {
        if (res.empty() || itv[0] > res.back()[1]) res.push_back(itv);
        else res.back()[1] = max(res.back()[1], itv[1]);
    }
    return res;
}
```

Time $O(n \log n)$.

06-P4 — Count inversions with merge sort

```
long long inv = 0;
void mergeCount(vector<int>& a, int l, int m, int r) {
    vector<int> L(a.begin()+l, a.begin()+m+1), R(a.begin()+m+1, a.begin()+r+1);
    int i = 0, j = 0, k = l;
    while (i < (int)L.size() && j < (int)R.size()) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else { a[k++] = R[j++]; inv += (long long)(L.size() - i); }
    }
    while (i < (int)L.size()) a[k++] = L[i++];
    while (j < (int)R.size()) a[k++] = R[j++];
}
void sortCount(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    sortCount(a, l, m); sortCount(a, m + 1, r);
    mergeCount(a, l, m, r);
}
```

Time $O(n \log n)$.

06-P5 — Sort colors / Dutch national flag (LC 75)

```
void sortColors(vector<int>& a) {
    int lo = 0, mid = 0, hi = (int)a.size() - 1;
    while (mid <= hi) {
        if (a[mid] == 0) { swap(a[lo++], a[mid++]); }
        else if (a[mid] == 1) mid++;
        else { swap(a[mid], a[hi--]); }
    }
}
```

06-P6 — Sort a nearly sorted array (each element within k)

```
vector<int> sortNearlySorted(vector<int>& a, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        pq.push(a[i]);
        if ((int)pq.size() > k) { res.push_back(pq.top()); pq.pop(); }
    }
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    return res;
}
```

Time $O(n \log k)$.

Chapter 07 — Hashing

07-P1 (217), 07-P2 (242), 07-P3 (1), 07-P4 (560)

Same as 01-P1, 04-P2, 01-P3's pair version (see Ch. 01 solution for two-sum pattern), and 03-P3 respectively.
For LC 1:

```
vector<int> twoSum(vector<int>& a, int target) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)a.size(); i++) {
        int need = target - a[i];
        if (pos.count(need)) return {pos[need], i};
        pos[a[i]] = i;
    }
    return {};
}
```

07-P5 — Group anagrams (LC 49)

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> g;
    for (auto& s : strs) {
        string k = s; sort(k.begin(), k.end());
        g[k].push_back(s);
    }
    vector<vector<string>> res;
    for (auto& [k, v] : g) res.push_back(v);
    return res;
}
```

07-P6 — Longest subarray divisible by k

```
int longestDivisible(const vector<int>& a, int k) {
    unordered_map<int,int> first; // remainder -> first index
    first[0] = -1;
    int sum = 0, best = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        sum += a[i];
        int r = ((sum % k) + k) % k; // C++ negative-mod fix
        if (first.count(r)) best = max(best, i - first[r]);
        else first[r] = i;
    }
    return best;
}
```

Time $O(n)$, space $O(n)$.

Chapter 08 — Two Pointers

08-P1 — Two sum sorted (LC 167)

```
vector<int> twoSum(vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return {l + 1, r + 1};
        if (s < target) l++; else r--;
    }
    return {};
}
```

08-P2 (125) — see 04-P1.

08-P3 — Remove duplicates (LC 26)

```
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int w = 0;
    for (int r = 1; r < (int)a.size(); r++)
        if (a[r] != a[w]) a[++w] = a[r];
    return w + 1;
}
```

08-P4 — Move zeroes (LC 283)

```
void moveZeroes(vector<int>& a) {
    int w = 0;
    for (int x : a) if (x != 0) a[w++] = x;
    while (w < (int)a.size()) a[w++] = 0;
}
```

08-P5 — 3Sum (LC 15)

```
vector<vector<int>> threeSum(vector<int>& a) {
    sort(a.begin(), a.end());
    vector<vector<int>> res;
    int n = a.size();
    for (int i = 0; i < n; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.push_back({a[i], a[l], a[r]});
                while (l < r && a[l] == a[l + 1]) l++;
                while (l < r && a[r] == a[r - 1]) r--;
                l++; r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
    return res;
}
```

Time $O(n^2)$, space $O(1)$ extra.

08-P6 — Trapping rain water (LC 42)

```
int trap(vector<int>& h) {
    int l = 0, r = (int)h.size() - 1, lm = 0, rm = 0, ans = 0;
    while (l <= r) {
        if (lm <= rm) { lm = max(lm, h[l]); ans += lm - h[l++]; }
        else { rm = max(rm, h[r]); ans += rm - h[r--]; }
    }
    return ans;
}
```

Chapter 09 — Sliding Window

09-P1 — Max consecutive ones (LC 485)

```
int findMaxConsecutiveOnes(vector<int>& a) {
    int cur = 0, best = 0;
    for (int x : a) {
        if (x == 1) cur++;
        else cur = 0;
        best = max(best, cur);
    }
    return best;
}
```

09-P2 — Longest substring without repeating (LC 3)

```
int lengthOfLongestSubstring(string s) {
    int last[128]; fill(last, last + 128, -1);
    int l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (last[(unsigned char)s[r]] >= l) l = last[(unsigned char)s[r]] + 1;
        last[(unsigned char)s[r]] = r;
        best = max(best, r - l + 1);
    }
}
```

```

    return best;
}

```

09-P3 — Max sum of subarray of size k

```

int maxSumFixedK(vector<int>& a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    int best = sum;
    for (int i = k; i < (int)a.size(); i++) {
        sum += a[i] - a[i - k];
        best = max(best, sum);
    }
    return best;
}

```

09-P4 — Longest substring with at most k distinct (LC 340)

```

int lengthOfLongestSubstringKDistinct(string s, int k) {
    int cnt[128] = {0}, distinct = 0, l = 0, best = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        if (cnt[(unsigned char)s[r]]++ == 0) distinct++;
        while (distinct > k) {
            if (--cnt[(unsigned char)s[l++]] == 0) distinct--;
        }
        best = max(best, r - l + 1);
    }
    return best;
}

```

09-P5 — Minimum window substring (LC 76)

```

string minWindow(string s, string t) {
    int need[128] = {0}, have[128] = {0}, needCnt = 0, haveCnt = 0;
    for (char c : t) if (need[(unsigned char)c]++ == 0) needCnt++;
    int l = 0, bestLen = INT_MAX, start = 0;
    for (int r = 0; r < (int)s.size(); r++) {
        char c = s[r];
        have[(unsigned char)c]++;
        if (need[(unsigned char)c] && have[(unsigned char)c] == need[(unsigned char)c]) haveCnt++;
        while (haveCnt == needCnt) {
            if (r - l + 1 < bestLen) { bestLen = r - l + 1; start = l; }
            char d = s[l++];
            if (need[(unsigned char)d] && have[(unsigned char)d] == need[(unsigned char)d]) haveCnt--;
            have[(unsigned char)d]--;
        }
    }
    return bestLen == INT_MAX ? "" : s.substr(start, bestLen);
}

```

09-P6 — Minimum size subarray sum $\geq k$ (LC 209)

```

int minSubArrayLen(int target, vector<int>& a) {
    int l = 0, sum = 0, best = INT_MAX;
    for (int r = 0; r < (int)a.size(); r++) {
        sum += a[r];
        while (sum >= target) {
            best = min(best, r - l + 1);
            sum -= a[l++];
        }
    }
    return best == INT_MAX ? 0 : best;
}

```

Chapter 10 — Linked Lists

```

struct ListNode { int val; ListNode* next; ListNode(int v): val(v), next(nullptr){} };

```

10-P1 — Reverse (LC 206)

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr, *cur = head;
    while (cur) {
        ListNode* nxt = cur->next;
        cur->next = prev;
        prev = cur; cur = nxt;
    }
    return prev;
}
```

10-P2 — Merge two sorted lists (LC 21)

```
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0), *t = &dummy;
    while (a && b) {
        if (a->val <= b->val) { t->next = a; a = a->next; }
        else { t->next = b; b = b->next; }
        t = t->next;
    }
    t->next = a ? a : b;
    return dummy.next;
}
```

10-P3 — Has cycle (LC 141)

```
bool hasCycle(ListNode* head) {
    ListNode *s = head, *f = head;
    while (f && f->next) {
        s = s->next; f = f->next->next;
        if (s == f) return true;
    }
    return false;
}
```

10-P4 — Remove n-th from end (LC 19)

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0); dummy.next = head;
    ListNode *f = &dummy, *s = &dummy;
    for (int i = 0; i <= n; i++) f = f->next;
    while (f) { s = s->next; f = f->next; }
    s->next = s->next->next;
    return dummy.next;
}
```

10-P5 — Cycle start node (LC 142)

```
ListNode* detectCycle(ListNode* head) {
    ListNode *s = head, *f = head;
    while (f && f->next) {
        s = s->next; f = f->next->next;
        if (s == f) {
            s = head;
            while (s != f) { s = s->next; f = f->next; }
            return s;
        }
    }
    return nullptr;
}
```

10-P6 — Reorder list (LC 143)

```
void reorderList(ListNode* head) {
    if (!head || !head->next) return;
    // middle
    ListNode *s = head, *f = head;
    while (f && f->next) { s = s->next; f = f->next->next; }
```

```

// reverse second half
ListNode* prev = nullptr, *cur = s;
while (cur) { ListNode* nxt = cur->next; cur->next = prev; prev = cur; cur = nxt; }
// interleave
ListNode* a = head; ListNode* b = prev;
while (b->next) {
    ListNode* an = a->next, *bn = b->next;
    a->next = b; b->next = an;
    a = an; b = bn;
}
}

```

10-P7 — LRU cache (LC 146)

```

class LRUCache {
    int cap;
    list<pair<int,int>> lru;
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        if (!mp.count(key)) return -1;
        auto it = mp[key];
        int val = it->second;
        lru.erase(it);
        lru.push_front({key, val});
        mp[key] = lru.begin();
        return val;
    }
    void put(int key, int value) {
        if (mp.count(key)) {
            lru.erase(mp[key]);
        } else if ((int)lru.size() == cap) {
            mp.erase(lru.back().first);
            lru.pop_back();
        }
        lru.push_front({key, value});
        mp[key] = lru.begin();
    }
};

```

Chapter 11 — Stacks & Queues

11-P1 — Valid parentheses (LC 20)

```

bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') || (c == ']' && t != '[') || (c == '}' && t != '{'))
                return false;
        }
    }
    return st.empty();
}

```

11-P2 — Queue with two stacks (LC 232)

```

class MyQueue {
    stack<int> in, out;
    void pour() { while (!in.empty()) { out.push(in.top()); in.pop(); } }
public:
    void push(int x) { in.push(x); }
    int pop() { if (out.empty()) pour(); int v = out.top(); out.pop(); return v; }
    int peek() { if (out.empty()) pour(); return out.top(); }
    bool empty() { return in.empty() && out.empty(); }
};

```

11-P3 — Daily temperatures (LC 739)

```
vector<int> dailyTemperatures(vector<int>& t) {
    int n = t.size();
    vector<int> res(n, 0);
    stack<int> st; // decreasing temps
    for (int i = 0; i < n; i++) {
        while (!st.empty() && t[st.top()] < t[i]) {
            res[st.top()] = i - st.top();
            st.pop();
        }
        st.push(i);
    }
    return res;
}
```

11-P4 — Evaluate RPN (LC 150)

```
int evalRPN(vector<string>& tok) {
    stack<int> st;
    for (auto& s : tok) {
        if (s == "+" || s == "-" || s == "*" || s == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (s == "+") st.push(a + b);
            else if (s == "-") st.push(a - b);
            else if (s == "*") st.push(a * b);
            else st.push(a / b);
        } else st.push(stoi(s));
    }
    return st.top();
}
```

11-P5 — Largest rectangle in histogram (LC 84)

See the chapter code (monotonic stacks for prev/next smaller).

11-P6 — Sliding window maximum (LC 239)

See the chapter code (monotonic deque).

Chapter 12 — Recursion

12-P1 — Reverse string (LC 344)

```
void reverseString(vector<char>& s) {
    function<void(int,int)> rev = [&](int l, int r) {
        if (l >= r) return;
        swap(s[l], s[r]);
        rev(l + 1, r - 1);
    };
    rev(0, (int)s.size() - 1);
}
```

12-P2 — Fibonacci (LC 509)

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

(For $n > \sim 30$, add memoization — see Ch. 19.)

12-P3 — Subsets (LC 78)

See Ch. 13 code.

12-P4 — Tower of Hanoi

```
void hanoi(int n, char from, char aux, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, aux);
    cout << "move disk " << n << " from " << from << " to " << to << "\n";
    hanoi(n - 1, aux, from, to);
}
```

12-P5 — Generate parentheses (LC 22)

```
vector<string> generateParenthesis(int n) {
    vector<string> res;
    function<void(int,int,string)> dfs = [&](int open, int close, string cur) {
        if (open == n && close == n) { res.push_back(cur); return; }
        if (open < n) dfs(open + 1, close, cur + '(');
        if (close < open) dfs(open, close + 1, cur + ')');
    };
    dfs(0, 0, "");
    return res;
}
```

12-P6 — Modular exponentiation

```
long long modPow(long long a, long long n, long long m) {
    if (n == 0) return 1 % m;
    long long half = modPow(a, n / 2, m);
    half = half * half % m;
    return (n % 2 == 0) ? half : half * a % m;
}
```

Chapter 13 — Backtracking

13-P1 — Subsets (LC 78)

```
vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> res; vector<int> cur;
    function<void(int)> dfs = [&](int i) {
        if (i == (int)nums.size()) { res.push_back(cur); return; }
        cur.push_back(nums[i]); dfs(i + 1); cur.pop_back();
        dfs(i + 1);
    };
    dfs(0);
    return res;
}
```

13-P2 (22) — see 12-P5.

13-P3 — Permutations (LC 46)

```
vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> res; vector<int> cur;
    vector<bool> used(nums.size(), false);
    function<void()> dfs = [&]() {
        if (cur.size() == nums.size()) { res.push_back(cur); return; }
        for (int i = 0; i < (int)nums.size(); i++) {
            if (used[i]) continue;
            used[i] = true; cur.push_back(nums[i]);
            dfs();
            cur.pop_back(); used[i] = false;
        }
    };
    dfs();
    return res;
}
```

13-P4 — Combination sum (LC 39)

```
vector<vector<int>> combinationSum(vector<int>& a, int target) {
    vector<vector<int>> res; vector<int> cur;
    function<void(int,int)> dfs = [&](int start, int rem) {
        if (rem == 0) { res.push_back(cur); return; }
        for (int i = start; i < (int)a.size(); i++) {
            if (a[i] > rem) continue;
            cur.push_back(a[i]);
            dfs(i, rem - a[i]); // same index → can reuse
            cur.pop_back();
        }
    };
    dfs(0, target);
    return res;
}
```

13-P5 (51), 13-P6 (37), 13-P7 (79)

The chapter contains the full N-Queens code (13-P5). For the Sudoku solver (37), extend it with a `valid(row, col, c)` helper checking row, column, and 3x3 box, and try digits 1–9 in empty cells. For Word Search (79):

```
bool exist(vector<vector<char>>& b, string word) {
    int n = b.size(), m = b[0].size();
    int di[4] = {-1,1,0,0}, dj[4] = {0,0,-1,1};
    function<bool(int,int,int)> dfs = [&](int i, int j, int k) {
        if (k == (int)word.size()) return true;
        if (i < 0 || i >= n || j < 0 || j >= m || b[i][j] != word[k]) return false;
        char c = b[i][j]; b[i][j] = '#'; // mark
        for (int d = 0; d < 4; d++)
            if (dfs(i + di[d], j + dj[d], k + 1)) { b[i][j] = c; return true; }
        b[i][j] = c; // unmark
        return false;
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (dfs(i, j, 0)) return true;
    return false;
}
```

Chapter 14 — Trees

14-P1 — Max depth (LC 104)

```
int maxDepth(TreeNode* r) { return r ? 1 + max(maxDepth(r->left), maxDepth(r->right)) : 0; }
```

14-P2 — Invert (LC 226)

```
TreeNode* invertTree(TreeNode* r) {
    if (!r) return nullptr;
    swap(r->left, r->right);
    invertTree(r->left);
    invertTree(r->right);
    return r;
}
```

14-P3 — Level order (LC 102)

See chapter code.

14-P4 — Symmetric (LC 101)

```
bool isMirror(TreeNode* a, TreeNode* b) {
    if (!a && !b) return true;
    if (!a || !b) return false;
    return a->val == b->val && isMirror(a->left, b->right) && isMirror(a->right, b->left);
}
bool isSymmetric(TreeNode* r) { return r ? isMirror(r->left, r->right) : true; }
```

14-P5 — Validate BST (LC 98)

```
bool valid(TreeNode* r, long long lo, long long hi) {
    if (!r) return true;
    if (r->val <= lo || r->val >= hi) return false;
    return valid(r->left, lo, r->val) && valid(r->right, r->val, hi);
}
bool isValidBST(TreeNode* r) { return valid(r, LLONG_MIN, LLONG_MAX); }
```

14-P6 — Build from preorder & inorder (LC 105)

```
TreeNode* buildTree(vector<int>& pre, vector<int>& in) {
    unordered_map<int,int> pos;
    for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
    int pi = 0;
    function<TreeNode*(int,int)> build = [&](int lo, int hi) {
        if (lo > hi) return (TreeNode*)nullptr;
        int val = pre[pi++];
        TreeNode* root = new TreeNode(val);
        int mid = pos[val];
        root->left = build(lo, mid - 1);
        root->right = build(mid + 1, hi);
        return root;
    };
    return build(0, (int)in.size() - 1);
}
```

14-P7 — LCA (LC 236)

See chapter code.

Chapter 15 — BST

15-P1 (700), 15-P2 (701), 15-P4 (230), 15-P5 (450)

See the chapter code for search, insert, k-th smallest, and delete.

15-P3 — Validate BST (LC 98) — see 14-P5.

15-P6 — Convert to greater tree (LC 538)

```
TreeNode* convertBST(TreeNode* root) {
    int sum = 0;
    function<void(TreeNode*)> rev = [&](TreeNode* r) {
        if (!r) return;
        rev(r->right);
        sum += r->val;
        r->val = sum;
        rev(r->left);
    };
    rev(root);
    return root;
}
```

Chapter 16 — Heaps

16-P1 (215) — see 06-P1.

16-P2 (23) — see the chapter code (merge k lists).

16-P4 (295) — see the chapter code (median finder).

16-P3 — K closest points to origin (LC 973)

```
vector<vector<int>> kClosest(vector<vector<int>>& pts, int k) {
    nth_element(pts.begin(), pts.begin() + k, pts.end(),
```

```

        [](auto& a, auto& b) {
            return a[0]*a[0] + a[1]*a[1] < b[0]*b[0] + b[1]*b[1];
        });
    return vector<vector<int>>(pts.begin(), pts.begin() + k);
}

```

Time $O(n)$ average.

16-P5 — Task scheduler (LC 621)

```

int leastInterval(vector<char>& tasks, int n) {
    int cnt[26] = {0};
    for (char c : tasks) cnt[c - 'A']++;
    int maxF = *max_element(cnt, cnt + 26);
    int maxCnt = 0;
    for (int x : cnt) if (x == maxF) maxCnt++;
    return max((int)tasks.size(), (maxF - 1) * (n + 1) + maxCnt);
}

```

16-P6 — Sort nearly sorted array

See 06-P6.

Chapter 17 — Greedy

17-P1 (55) — see the chapter code (canJump).

17-P2 — Assign cookies (LC 455)

```

int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end()); sort(s.begin(), s.end());
    int i = 0;
    for (int c : s)
        if (i < (int)g.size() && c >= g[i]) i++;
    return i;
}

```

17-P3 — Greedy coin change (canonical systems)

```

int minCoinsGreedy(vector<int> coins, int amount) {
    sort(coins.rbegin(), coins.rend());
    int cnt = 0;
    for (int c : coins) {
        cnt += amount / c;
        amount %= c;
    }
    return amount == 0 ? cnt : -1;
}

```

17-P4 (253) — see the chapter code (meeting rooms).

17-P5 — Min arrows to burst balloons (LC 452)

```

int findMinArrowShots(vector<vector<int>>& pts) {
    sort(pts.begin(), pts.end(), [](auto& a, auto& b){ return a[1] < b[1]; });
    int arrows = 0;
    long long end = LLONG_MIN;
    for (auto& p : pts) {
        if (p[0] > end) { arrows++; end = p[1]; }
    }
    return arrows;
}

```

17-P6 — Jump Game II (LC 45)

```

int jump(vector<int>& a) {
    int jumps = 0, curEnd = 0, farthest = 0;
}

```

```

    for (int i = 0; i < (int)a.size() - 1; i++) {
        farthest = max(farthest, i + a[i]);
        if (i == curEnd) { jumps++; curEnd = farthest; }
    }
    return jumps;
}

```

Time $O(n)$.

17-P7 — Gas station (LC 134)

```

int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
    int total = 0, cur = 0, start = 0;
    for (int i = 0; i < (int)gas.size(); i++) {
        total += gas[i] - cost[i];
        cur += gas[i] - cost[i];
        if (cur < 0) { start = i + 1; cur = 0; }
    }
    return total >= 0 ? start : -1;
}

```

Chapter 18 — Graphs

18-P1 — Path exists (LC 1971)

```

bool validPath(int n, vector<vector<int>>& edges, int s, int d) {
    vector<vector<int>> g(n);
    for (auto& e : edges) { g[e[0]].push_back(e[1]); g[e[1]].push_back(e[0]); }
    vector<bool> vis(n, false);
    queue<int> q; q.push(s); vis[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == d) return true;
        for (int v : g[u]) if (!vis[v]) { vis[v] = true; q.push(v); }
    }
    return false;
}

```

18-P2 (200) — see the chapter code (numIslands).

18-P3 — Clone graph (LC 133)

```

Node* cloneGraph(Node* node) {
    if (!node) return nullptr;
    unordered_map<Node*, Node*> m;
    function<Node*(Node*)> dfs = [&](Node* u) {
        if (m.count(u)) return m[u];
        Node* c = new Node(u->val);
        m[u] = c;
        for (Node* v : u->neighbors) c->neighbors.push_back(dfs(v));
        return c;
    };
    return dfs(node);
}

```

18-P4 — Rotting oranges (LC 994)

```

int orangesRotting(vector<vector<int>>& g) {
    int n = g.size(), m = g[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (g[i][j] == 2) q.push({i, j});
            else if (g[i][j] == 1) fresh++;
        }
    int di[4] = {-1,1,0,0}, dj[4] = {0,0,-1,1}, t = 0;
    while (!q.empty() && fresh) {
        int sz = q.size(); t++;

```

```

    for (int k = 0; k < sz; k++) {
        auto [i, j] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int ni = i + di[d], nj = j + dj[d];
            if (ni < 0 || ni >= n || nj < 0 || nj >= m || g[ni][nj] != 1) continue;
            g[ni][nj] = 2; fresh--;
            q.push({ni, nj});
        }
    }
    return fresh == 0 ? t : -1;
}

```

18-P5 — Course schedule (LC 207)

```

bool canFinish(int n, vector<vector<int>>& prereq) {
    vector<vector<int>> g(n);
    vector<int> indeg(n, 0);
    for (auto& p : prereq) { g[p[1]].push_back(p[0]); indeg[p[0]]++; }
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    int cnt = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); cnt++;
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return cnt == n;
}

```

18-P6 — Course schedule II (LC 210)

```

vector<int> findOrder(int n, vector<vector<int>>& prereq) {
    // identical to topoSort in the chapter; return order if size == n else {}
    vector<vector<int>> g(n);
    vector<int> indeg(n, 0);
    for (auto& p : prereq) { g[p[1]].push_back(p[0]); indeg[p[0]]++; }
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order.size() == (size_t)n ? order : vector<int>();
}

```

18-P7 — Provinces (LC 547)

```

int findCircleNum(vector<vector<int>>& isConnected) {
    int n = isConnected.size(), ans = 0;
    vector<bool> vis(n, false);
    function<void(int)> dfs = [&](int u) {
        vis[u] = true;
        for (int v = 0; v < n; v++)
            if (isConnected[u][v] && !vis[v]) dfs(v);
    };
    for (int i = 0; i < n; i++) if (!vis[i]) { ans++; dfs(i); }
    return ans;
}

```

18-P8 — MST cost (Kruskal)

```

// Use the DSU from Ch. 22 with the kruskal code in Ch. 18.

```

Chapter 19 — DP

19-P1 (70) — see chapter code.

19-P3 (300) — see chapter code.

19-P5 (1143), 19-P7 (5), 19-P9 — see chapter code.

19-P2 — House robber (LC 198)

```
int rob(vector<int>& a) {
    int take = 0, skip = 0;
    for (int x : a) {
        int nt = skip + x;
        skip = max(skip, take);
        take = nt;
    }
    return max(take, skip);
}
```

19-P4 — Coin change (LC 322)

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int w = 1; w <= amount; w++)
        for (int c : coins)
            if (c <= w && dp[w - c] != INT_MAX)
                dp[w] = min(dp[w], dp[w - c] + 1);
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

(Unbounded knapsack: iterate amount ascending.)

19-P6 — Unique paths (LC 62)

```
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[j] += dp[j - 1];
    return dp[n - 1];
}
```

19-P8 — Edit distance (LC 72)

```
int minDistance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    return dp[n][m];
}
```

19-P9 — 0/1 knapsack

```
int knapsack01(const vector<int>& wt, const vector<int>& val, int W) {
    int n = wt.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int w = W; w >= wt[i]; w--)
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
}
```

```
    return dp[W];
}
```

Chapter 20 — Tries

20-P1 (208), 20-P3 (212)

See the chapter code.

20-P2 — Add & search word with wildcard (LC 211)

```
class WordDictionary {
    struct T { T* c[26] = {}; bool end = false; } *root = new T();
    bool dfs(const string& w, int i, T* cur) {
        if (!cur) return false;
        if (i == (int)w.size()) return cur->end;
        if (w[i] != '.') return dfs(w, i + 1, cur->c[w[i] - 'a']);
        for (int k = 0; k < 26; k++)
            if (cur->c[k] && dfs(w, i + 1, cur->c[k])) return true;
        return false;
    }
public:
    void addWord(string w) {
        T* cur = root;
        for (char c : w) {
            if (!cur->c[c - 'a']) cur->c[c - 'a'] = new T();
            cur = cur->c[c - 'a'];
        }
        cur->end = true;
    }
    bool search(string w) { return dfs(w, 0, root); }
};
```

20-P4 — Longest word built by adding letters (LC 720)

```
string longestWord(vector<string>& words) {
    TrieNode* root = new TrieNode();
    for (auto& w : words) insertWord(root, w);
    string best;
    function<void(TrieNode*, string)> dfs = [&](TrieNode* cur, string s) {
        if (s.size() > best.size() || (s.size() == best.size() && s < best)) best = s;
        for (int i = 0; i < 26; i++) {
            if (cur->child[i] && cur->child[i]->isEnd)
                dfs(cur->child[i], s + char('a' + i));
        }
    };
    for (int i = 0; i < 26; i++)
        if (root->child[i] && root->child[i]->isEnd)
            dfs(root->child[i], string(1, char('a' + i)));
    return best;
}
```

Chapter 21 — Bit Manipulation

21-P1 (136) — XOR all: see chapter code.

21-P2 — Power of two (LC 231)

```
bool isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }
```

21-P3 (461)

```
int hammingDistance(int a, int b) { return __builtin_popcount(a ^ b); }
```

21-P4 — Counting bits (LC 338)

```
vector<int> countBits(int n) {
    vector<int> dp(n + 1, 0);
    for (int i = 1; i <= n; i++) dp[i] = dp[i >> 1] + (i & 1);
    return dp;
}
```

21-P5 (260) — see chapter code.

21-P6 — Single number II (LC 137)

```
int singleNumber(vector<int>& a) {
    int res = 0;
    for (int b = 0; b < 32; b++) {
        int cnt = 0;
        for (int x : a) if ((x >> b) & 1) cnt++;
        if (cnt % 3) res |= (1 << b);
    }
    return res;
}
```

21-P7 — Sum without + / - (LC 371)

```
int getSum(int a, int b) {
    while (b) {
        unsigned carry = ((unsigned)a & b) << 1;
        a ^= b;
        b = carry;
    }
    return a;
}
```

Chapter 22 — DSU

22-P1 (547/323)

```
// count components: start with comps = n; decrement on each successful unite.
// DSU from chapter.
```

22-P2 — Accounts merge (LC 721)

```
vector<vector<string>> accountsMerge(vector<vector<string>>& acc) {
    unordered_map<string,int> emailId;
    unordered_map<int,string> emailOwner;
    DSU dsu(acc.size());
    for (int i = 0; i < (int)acc.size(); i++)
        for (int j = 1; j < (int)acc[i].size(); j++) {
            string e = acc[i][j];
            if (!emailId.count(e)) { emailId[e] = i; emailOwner[e] = acc[i][0]; }
            else dsu.unite(emailId[e], i);
        }
    unordered_map<int, set<string>> groups;
    for (auto& [e, ownerIdx] : emailOwner) groups[dsu.find(ownerIdx)].insert(e);
    vector<vector<string>> res;
    for (auto& [root, emails] : groups) {
        vector<string> v{acc[root][0]};
        v.insert(v.end(), emails.begin(), emails.end());
        res.push_back(v);
    }
    return res;
}
```

22-P3 — Redundant connection (LC 684)

```
vector<int> findRedundantConnection(vector<vector<int>>& edges) {
    DSU dsu(edges.size() + 1);
```

```

for (auto& e : edges)
    if (!dsu.unite(e[0], e[1])) return e; // first cycle-creating edge
return {};
}

```

22-P4 — Longest consecutive sequence (LC 128)

```

int longestConsecutive(vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : a) {
        if (s.count(x - 1)) continue; // not a run start
        int len = 1;
        while (s.count(x + len)) len++;
        best = max(best, len);
    }
    return best;
}

```

Time $O(n)$.

22-P5 — Number of islands II (LC 305)

```

vector<int> numIslands2(int m, int n, vector<vector<int>>& pos) {
    DSU dsu(m * n);
    vector<bool> land(m * n, false);
    int cnt = 0;
    vector<int> res;
    int di[4] = {-1, 1, 0, 0}, dj[4] = {0, 0, -1, 1};
    for (auto& p : pos) {
        int id = p[0] * n + p[1];
        if (land[id]) { res.push_back(cnt); continue; }
        land[id] = true; cnt++;
        for (int d = 0; d < 4; d++) {
            int ni = p[0] + di[d], nj = p[1] + dj[d];
            if (ni < 0 || ni >= m || nj < 0 || nj >= n) continue;
            int nid = ni * n + nj;
            if (land[nid] && dsu.unite(id, nid)) cnt--;
        }
        res.push_back(cnt);
    }
    return res;
}

```

Chapter 23 — Fenwick Trees

23-P1, 23-P2

Use the chapter code. Test:

```

Fenwick bit(5);
bit.add(1, 3); bit.add(2, 1); bit.add(3, 4);
assert(bit.sum(3) == 8);

```

Inversions via BIT: see the chapter code.

23-P3 — Count of smaller numbers after self (LC 315)

```

vector<int> countSmaller(vector<int>& a) {
    vector<int> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick bit(vals.size());
    vector<int> res(a.size());
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        int rank = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin() + 1;
        res[i] = bit.sum(rank - 1); // strictly smaller = sum(1..rank-1)
        bit.add(rank, 1);
    }
}

```

```
    return res;
}
```

Time $O(n \log n)$.

23-P4 — K-th smallest dynamic (BIT + binary search)

```
int kthSmallest(const Fenwick& bit, int k, int n) {
    int lo = 1, hi = n;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (bit.sum(mid) >= k) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Chapter 24 — Segment Trees

24-P1, 24-P2, 24-P4, 24-P5

Use the chapter code (sum tree; lazy tree for P4; XOR combine for P5). Test P2:

```
SegTree st({1, 3, 5});
st.update(1, 2);
assert(st.query(0, 2) == 8); // 1 + 2 + 5
```

24-P3 — Count of smaller numbers (segment tree version)

```
// Same idea as 23-P3, but use a segment tree over ranks:
// update(rank, +1), query(1, rank-1) for each element from right to left.
```

Chapter 25 — Advanced Graphs

25-P1 — BFS shortest path (plain)

```
// dist via BFS as in Ch. 18; return dist[target] or -1 if unreachable.
```

25-P2 — Network delay time (LC 743)

```
int networkDelayTime(vector<vector<int>>& times, int n, int k) {
    vector<vector<pair<int,int>>> g(n + 1);
    for (auto& t : times) g[t[0]].push_back({t[1], t[2]});
    vector<long long> dist(n + 1, 1e18);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[k] = 0; pq.push({0, k});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto& [v, w] : g[u])
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }
    }
    long long mx = 0;
    for (int i = 1; i <= n; i++) mx = max(mx, dist[i]);
    return mx == 1e18 ? -1 : (int)mx;
}
```

25-P3 — Cheapest flights within k stops (LC 787)

```
int findCheapestPrice(int n, vector<vector<int>>& fl, int src, int dst, int k) {
    vector<long long> dist(n, 1e18);
    dist[src] = 0;
    for (int stops = 0; stops <= k; stops++) {
        vector<long long> nd = dist;
        for (auto& e : fl) {
            int u = e[0], v = e[1], w = e[2];
```

```

        if (dist[u] != 1e18) nd[v] = min(nd[v], dist[u] + w);
    }
    dist = nd;
}
return dist[dst] == 1e18 ? -1 : (int)dist[dst];
}

```

(Bellman-Ford style, relaxed $k+1$ times.)

25-P4 — City with fewest neighbors $\leq$ threshold (LC 1334)

```

int findTheCity(int n, vector<vector<int>>& edges, int dT) {
    vector<vector<long long>> d(n, vector<long long>(n, 1e18));
    for (int i = 0; i < n; i++) d[i][i] = 0;
    for (auto& e : edges) {
        d[e[0]][e[1]] = e[2];
        d[e[1]][e[0]] = e[2];
    }
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    int city = -1, bestCnt = INT_MAX;
    for (int i = 0; i < n; i++) {
        int cnt = 0;
        for (int j = 0; j < n; j++) if (i != j && d[i][j] <= dT) cnt++;
        if (cnt <= bestCnt) { bestCnt = cnt; city = i; }
    }
    return city;
}

```

25-P5 — SCC count (Kosaraju)

Use the chapter code; the answer is the final `id`.

25-P6 — Critical connections (LC 1192)

```

vector<vector<int>> criticalConnections(int n, vector<vector<int>>& edges) {
    vector<vector<int>> g(n);
    for (auto& e : edges) { g[e[0]].push_back(e[1]); g[e[1]].push_back(e[0]); }
    vector<int> tin(n, -1), low(n);
    int timer = 0;
    vector<vector<int>> res;
    function<void(int,int)> dfs = [&](int u, int parent) {
        tin[u] = low[u] = timer++;
        for (int v : g[u]) {
            if (v == parent) continue;
            if (tin[v] != -1) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) res.push_back({u, v});
            }
        }
    };
    for (int i = 0; i < n; i++) if (tin[i] == -1) dfs(i, -1);
    return res;
}

```

Chapter 26 — Patterns

- **26-P1 (217):** see 01-P1.
- **26-P2 (125):** see 04-P1.
- **26-P3 (1):** see 07-P3.
- **26-P4 (3):** see 09-P2.
- **26-P5 (215):** see 06-P1.
- **26-P6 (200):** see chapter code.
- **26-P7 (198):** see 19-P2.

- **26-P8 (253):** see chapter code.
- **26-P9 (239):** see chapter code.

26-P10 — Word ladder (LC 127)

```
int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
    unordered_set<string> dict(wordList.begin(), wordList.end());
    if (!dict.count(endWord)) return 0;
    queue<string> q; q.push(beginWord);
    int depth = 1;
    while (!q.empty()) {
        int sz = q.size();
        for (int s = 0; s < sz; s++) {
            string w = q.front(); q.pop();
            for (int i = 0; i < (int)w.size(); i++) {
                char orig = w[i];
                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == orig) continue;
                    w[i] = c;
                    if (w == endWord) return depth + 1;
                    if (dict.count(w)) { q.push(w); dict.erase(w); }
                }
                w[i] = orig;
            }
        }
        depth++;
    }
    return 0;
}
```

Time $O(n \cdot L \cdot 26)$, space $O(n)$.

26-P11 — Minimum cost to connect sticks (LC 1167)

```
int connectSticks(vector<int>& sticks) {
    priority_queue<int, vector<int>, greater<int>> pq(sticks.begin(), sticks.end());
    int cost = 0;
    while (pq.size() > 1) {
        int a = pq.top(); pq.pop();
        int b = pq.top(); pq.pop();
        int c = a + b;
        cost += c;
        pq.push(c);
    }
    return cost;
}
```

Time $O(n \log n)$.

Chapter 28 — DSA Checklist & Complexity Cheat Sheet

This is the last chapter. Use it as a **revision sheet**: skim it before an interview or a contest, and use the checklist to self-audit what you can and cannot do yet.

Part A — Complexity Cheat Sheet

A1. Big-O cheat sheet (per operation)

Data structure	Access	Search	Insert	Delete
Array (vector)	$O(1)$	$O(n)$	$O(n)^*$	$O(n)^*$
Sorted array	$O(1)$	$O(\log n)$	$O(n)$	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(1)^{**}$	$O(1)^{**}$

Data structure	Access	Search	Insert	Delete
Stack / Queue / Deque	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash map / set (unordered)	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
Map / set (ordered)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Priority queue (heap)	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$

* at the end it's $O(1)$ amortized; in the middle it's $O(n)$. ** when you already have a pointer to the position.

A2. Sorting complexities

Sort	Average	Worst	Space	Stable
Selection	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Bubble	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes

A3. Graph algorithms

Problem	Algorithm	Complexity
Traversal / components	DFS or BFS	$O(V + E)$
Shortest path, unweighted	BFS	$O(V + E)$
Shortest path, non-negative	Dijkstra	$O((V+E) \log V)$
Shortest path, negatives	Bellman-Ford	$O(V \cdot E)$
All pairs shortest	Floyd-Warshall	$O(V^3)$
MST	Kruskal / Prim	$O(E \log E) / O((V+E) \log V)$
Topological sort	Kahn	$O(V + E)$
SCC	Kosaraju / Tarjan	$O(V + E)$
Bridges / articulation	Tarjan	$O(V + E)$
Max flow	Dinic	$O(V^2 E)$

A4. Common algorithms at a glance

Algorithm	Complexity
Binary search	$O(\log n)$
Lower bound / upper bound	$O(\log n)$
Sliding window / two pointers	$O(n)$
Kadane (max subarray)	$O(n)$
Quickselect (k-th)	$O(n)$ avg
LIS	$O(n \log n)$
LCS	$O(n \cdot m)$
Knapsack	$O(n \cdot W)$
Fibonacci (memoized)	$O(n)$

Algorithm	Complexity
Backtracking	exponential (pruned)
Bitmask DP	$O(2^n \cdot n)$

A5. Input-size → feasible complexity (1 second rule of thumb)

n	Must be ≤
10	$O(n!)$
20	$O(2^n)$
500	$O(n^3)$
5,000	$O(n^2)$
$10^5 - 10^6$	$O(n \log n)$
$10^7 - 10^8$	$O(n)$
any	$O(\log n)$

Part B — STL Fast Reference

```
// containers
vector<int> v; v.push_back(x); v.pop_back(); v[i]; v.size();
string s; s += c; s.substr(l, len); s.find(x) != string::npos;
stack<int> st; st.push(x); st.top(); st.pop();
queue<int> q; q.push(x); q.front(); q.pop();
deque<int> dq; push_back/push_front/pop_back/pop_front/front/back;
priority_queue<int> pq; // max-heap
priority_queue<int, vector<int>, greater<int>> minq; // min-heap
map<Key,Val> m; set<int> s; // ordered,  $O(\log n)$ 
unordered_map<Key,Val> hm; // hash,  $O(1)$  avg

// algorithms
sort(b, e); reverse(b, e); next_permutation(b, e);
lower_bound(b, e, x); upper_bound(b, e, x);
binary_search(b, e, x);
min_element(b, e); max_element(b, e); accumulate(b, e, 0LL);
unique(b, e); // use with erase
count(b, e, x); find(b, e, x);

// fast I/O
ios::sync_with_stdio(false);
cin.tie(nullptr);

// builtins
__builtin_popcount(x); __builtin_popcountll(x);
__builtin_ctz(x); __builtin_clz(x); __builtin_ffs(x);
```

Part C — Interview Checklist (the 10-step self-audit)

C1. Data structure selection

Ask yourself: **what am I doing most?** - Random access by index → array. - "Have I seen it?" / counting / dedupe → hash map. - Need sorted order / nearest neighbors / predecessor-successor → ordered set/map. - Repeatedly get min/max → heap. - Subarray/substring → sliding window / prefix sums. - Sequence → two pointers. - Nesting / matching / next-greater → stack. - FIFO / levels / BFS → queue. - Hierarchy / tree-like → trees. - Relationships / reachability → graph. - "Min cost / count ways / possible" → DP.

C2. Algorithm selection by problem phrase

Problem phrase	Algorithm
"Find the k-th largest/smallest"	heap / quickselect
"Longest substring with..."	sliding window

Problem phrase	Algorithm
"Subarray sum equals k"	prefix sums + hashmap
"Find a pair that..." (sorted)	two pointers
"Next greater / smaller"	monotonic stack
"Shortest path" (unweighted)	BFS
"Shortest path" (weighted)	Dijkstra / Bellman-Ford
"Min cost to connect all"	MST (Kruskal/Prim)
"Order with dependencies"	topological sort
"Count the number of ways / min cost"	DP
"Generate all ..." (small input)	backtracking
"Is it possible to reach / connect"	BFS / DFS / DSU
"Minimum X such that..." (monotonic)	binary search on answer
"Many range queries + updates"	segment tree / Fenwick
"Longest increasing/prefix/suffix"	DP (LIS), trie

C3. The pre-submission bug hunt (read your code top to bottom)

- [] Bounds checks on all indices (no OOB).
- [] `n` vs `n-1` vs `n+1` off-by-ones.
- [] `long long` used for sums / distances / prefix arrays.
- [] Empty input and single-element inputs handled.
- [] Comparator uses `<` not `<=`; `sort` arguments are iterators (`.begin()`, `.end()`).
- [] `unordered_map` presence checked with `count` / `find`, not `[]`.
- [] `pop()` on stack/queue read `top()` / `front()` first.
- [] Visited flags set at the right moment in BFS (when pushing, not popping).
- [] Recursion: base case reached, no infinite recursion, no null deref in trees.
- [] Backtracking: state is undone after each branch.
- [] Sliding window: `left` always advances in the shrink loop.
- [] Modulo/negative handling (`(x % k + k) % k`).
- [] No leftover debugging output.
- [] Time complexity fits the input size (see A5).

Part D — Topic-by-Topic Checklist (track your progress)

For each topic, be honest: can you (1) explain it, (2) implement it from memory, (3) solve the chapter's Easy, (4) solve its Medium, (5) solve its Challenging?

Topic	Explain	Implement	Easy	Medium	Hard
01 STL containers & algorithms	☐	☐	☐	☐	☐
02 Big-O analysis	☐	☐	☐	☐	☐
03 Arrays (prefix sums, Kadane)	☐	☐	☐	☐	☐
04 Strings	☐	☐	☐	☐	☐
05 Searching (binary search, search-on-answer)	☐	☐	☐	☐	☐
06 Sorting (all, stability, inversion count)	☐	☐	☐	☐	☐
07 Hashing (frequency, two-sum, anagram)	☐	☐	☐	☐	☐
08 Two pointers	☐	☐	☐	☐	☐
09 Sliding window	☐	☐	☐	☐	☐
10 Linked lists	☐	☐	☐	☐	☐

Topic	Explain	Implement	Easy	Medium	Hard
11 Stacks/Queues/Deque (monotonic)	☐	☐	☐	☐	☐
12 Recursion	☐	☐	☐	☐	☐
13 Backtracking	☐	☐	☐	☐	☐
14 Trees (traversals, LCA)	☐	☐	☐	☐	☐
15 BST	☐	☐	☐	☐	☐
16 Heaps (top-k, median)	☐	☐	☐	☐	☐
17 Greedy (exchange argument)	☐	☐	☐	☐	☐
18 Graphs (BFS, DFS, topo, MST)	☐	☐	☐	☐	☐
19 Dynamic programming	☐	☐	☐	☐	☐
20 Tries	☐	☐	☐	☐	☐
21 Bit manipulation	☐	☐	☐	☐	☐
22 DSU	☐	☐	☐	☐	☐
23 Fenwick tree	☐	☐	☐	☐	☐
24 Segment tree (+ lazy)	☐	☐	☐	☐	☐
25 Advanced graphs	☐	☐	☐	☐	☐
26 Problem-solving patterns	☐	☐	☐	☐	☐

Part E — Final advice

1. **Consistency beats intensity.** 30 focused minutes daily beats 6 hours once a week.
2. **Write every algorithm by hand** at least 3 times from memory. Muscle memory matters.
3. **Explain out loud** — if you can't explain it, you don't know it. This is the single best interview-prep technique.
4. **Solve the chapter problems** before reading Chapter 27's solutions.
5. **Revisit this checklist weekly.** Mark progress honestly.
6. When stuck on a problem for 30+ minutes, use Chapter 26's framework and the hint section — then redo the problem from scratch a day later.

Good luck. You now have every tool you need — go practice.